

Kubernetes: From First Pod to Production

A working book on Kubernetes — what the control loop actually does, how to run workloads

35 sections · approximately 54,265 words

Johan De Clercq

Generated from the markdown sources

Contents

Chapter 1 — Why Kubernetes	3
Chapter 2 — Declarative reconciliation	7
Chapter 3 — Cluster anatomy	13
Chapter 4 — kubectl and the resource model.....	19
Chapter 5 — Pods	26
Chapter 6 — Deployments and ReplicaSets	33
Chapter 7 — Configuration and secrets	39
Chapter 8 — Resources, requests and QoS	46
Chapter 9 — Health and lifecycle.....	52
Chapter 10 — The other workload kinds.....	57
Chapter 11 — Services	64
Chapter 12 — DNS and service discovery	70
Chapter 13 — Ingress and Gateway API	76
Chapter 14 — NetworkPolicy.....	83
Chapter 15 — Storage.....	89
Chapter 16 — The debugging mindset	95
Chapter 17 — Pods that won't run	100
Chapter 18 — Getting inside	107
Chapter 19 — Network diagnosis.....	112
Chapter 20 — Resources and eviction.....	118
Chapter 21 — Nodes and the control plane	124
Chapter 22 — Cookbook: symptom → cause → fix.....	131
Chapter 23 — Security	140
Chapter 24 — Admission control and supply chain	148
Chapter 25 — Scaling and disruption.....	154
Chapter 26 — Observability	160
Chapter 27 — Deployment strategies and GitOps.....	166
Chapter 28 — Multi-tenancy	172
Chapter 29 — Packaging: Helm and Kustomize	178
Chapter 30 — Operators and CRDs	183
Chapter 31 — The ecosystem	189
Chapter 32 — Anti-patterns	194
Appendix A — kubectl cheatsheet.....	203
Appendix B — Glossary	208
Appendix C — Further reading	211

Chapter 1 — Why Kubernetes

The problem it actually solves

Containers solved packaging. An image is a reproducible artifact that runs the same everywhere, which is a genuine advance and the subject of the companion book. What containers did not solve is everything that happens *after* you have more than one machine and more than a handful of containers:

- A host dies at 03:00. Which of its containers should run where now, and who decides?
- You want to deploy without downtime. Which containers stop, in what order, and what if the new ones are broken?
- Traffic triples on Friday. Who notices, and who starts more replicas?
- Forty services need to find each other, and their addresses change constantly.
- Fifteen teams deploy to shared machines. Who stops one team's runaway job from starving everyone else?

Each of these has a manual answer, and the manual answers are scripts. Every organisation that ran containers at scale before orchestration wrote those scripts, and they all converged on the same thing: a loop that compares what should be running against what is running, and fixes the difference.

Kubernetes is that loop, generalised and given an API. That is the whole idea, and Chapter 2 is devoted to it.

What you get

Concretely, and stated as capabilities rather than features:

Declarative desired state. You describe what should exist. Controllers make it so, and keep making it so. You do not run deployment steps; you change a record and the system converges. This is genuinely different from `docker run`, and it takes a while to stop thinking imperatively.

Self-healing. A pod dies, a controller notices and replaces it. A node dies, its pods are rescheduled elsewhere. Measured on this book's cluster, deleting a pod from a three-replica Deployment produced a replacement in under six seconds, unprompted.

Rolling updates and rollbacks as first-class operations, with health gating so a broken version stops the rollout rather than replacing everything.

Service discovery and load balancing built in. A stable name and virtual IP in front of a changing set of pods.

Bin-packing. The scheduler places workloads on nodes according to their declared resource needs, which is a genuine efficiency win at any scale beyond a few machines.

Horizontal autoscaling on CPU, memory or custom metrics.

A uniform API for everything extensible with your own resource types. This is the underrated one: it means storage, certificates, DNS records and databases can all be managed the same way as workloads, which is what the operator pattern (Chapter 30) is built on.

An enormous ecosystem. Whatever you need to do, something exists. This is a real benefit and, as the next section argues, also a cost.

What it costs

Honesty here matters more than enthusiasm, because the failure mode of Kubernetes adoption is not technical — it is a small team spending its engineering capacity on a platform instead of a product.

Conceptual surface. Pods, ReplicaSets, Deployments, Services, EndpointSlices, Ingress, ConfigMaps, Secrets, ServiceAccounts, Roles, RoleBindings, PersistentVolumes, PersistentVolumeClaims, StorageClasses, StatefulSets, DaemonSets, Jobs, CronJobs, HPAs, PDBs, NetworkPolicies, CRDs, admission webhooks. A default cluster exposes **71 API resource types** before you install anything. You will meet most of them.

Operational burden. A cluster is infrastructure. It needs upgrading — control plane and nodes, on a support window measured in months, not years. It needs monitoring, certificate rotation, etcd backups, and someone who understands it at 03:00. Managed control planes (EKS, GKE, AKS) remove a large part of this and not all of it: you still own the nodes, the workloads, and every decision above the API.

Debugging has more layers. In the Docker book, a failing container had four places to look. Here there are five, and one of them — the control plane — can itself be the problem. An application that will not start might be failing because of an image, a probe, a resource request no node can satisfy, an admission webhook, a missing RBAC permission, a NetworkPolicy, or a taint. Part IV exists because of this.

Cost. Control plane fees, nodes that must be over-provisioned for headroom, and the engineering time. A three-node cluster running one service costs meaningfully more than a VM running one service.

The ecosystem is a cost too. For any problem there are six projects, three of which are abandoned and two of which are commercial products with an open core. Evaluating them is work, and every one you adopt is another thing to upgrade.

When not to use it

The companion book's Chapter 30 laid out the decision; here is the short version from the other side.

Do not adopt Kubernetes because you might need to scale. You will not need to scale in the way you imagine, and the version of your system that eventually does need to scale will not be the version you are building now.

Do not adopt it for one service. A Deployment, a Service and an Ingress to run one container is a great deal of apparatus. A managed container service — Cloud Run, ECS Fargate, Container Apps — gives you rolling deploys, autoscaling, TLS and health checks without a cluster, and is the right answer far more often than its reputation suggests.

Do not adopt it to solve a problem you have not had yet. The good reasons are present constraints: you already run several hosts and coordinate them with scripts; deploy downtime is unacceptable; a node failure must not be an outage; many teams need isolated, self-service deploys.

Do not adopt it if nobody will own it. A cluster without a maintainer degrades. Version skew accumulates, certificates expire, and the first serious incident finds nobody who understands the system.

The honest summary: Kubernetes is very good at the problems it exists for, and those problems arrive later than most teams expect.

What Kubernetes is not

Not a PaaS. It does not build your code, manage your database, or give you a `git push` deploy. Those are things you assemble on top, which is why platform engineering exists as a discipline.

Not a way to avoid understanding your system. It automates decisions you must still be able to make. A pod that will not schedule is a resource-allocation question, and the scheduler will not answer it for you.

Not a security boundary by default. Pods on a node share a kernel, exactly as containers do. Namespaces are an *organisational* boundary with optional policy attached, not a hard multi-tenancy boundary. Chapter 23 covers what to do; Chapter 28 covers the limits.

Not portable in the way people mean. The API is portable; your manifests will run anywhere. Your *system* depends on storage classes, load balancer implementations, ingress controllers, node types and cloud IAM, and those differ. "No vendor lock-in" is overstated — it is better than the alternative, not free.

Not a replacement for containers. Everything in the Docker book still applies. Your images, layers, healthchecks, non-root users, signal handling and logging behaviour carry over unchanged, and Kubernetes will punish you for getting them wrong more visibly than Docker did.

How this book is organised

Part I builds the model: reconciliation, cluster anatomy, and the resource system. Part II runs workloads. Part III connects and stores. Part IV is debugging, which is where most of your time will actually go. Part V is what production requires. Part VI is the ecosystem and the mistakes.

The chapters assume Linux nodes and a recent cluster (v1.30+). Everything was checked against **v1.37.0**.

Try it

If you want to follow along, a three-node cluster takes about a minute:

```
kind create cluster --config examples/cluster/kind-cluster.yaml
```

```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
k8sbook-control-plane	Ready	control-plane	29s	v1.37.0
k8sbook-worker	Ready	<none>	13s	v1.37.0
k8sbook-worker2	Ready	<none>	13s	v1.37.0

Now look at the size of the thing you just started. This is every resource type the API serves, before you install anything at all:

```
kubectl api-resources --no-headers | wc -l
```

```
71
```

And the control plane components running as pods on the control-plane node:

```
kubectl get pods -n kube-system
```

Note `etcd`, `kube-apiserver`, `kube-controller-manager`, `kube-scheduler`, plus `kube-proxy` and a CNI plugin on every node, and CoreDNS. Chapter 3 explains what each does. For now the point is simply that a "minimal" cluster is already a distributed system with seven or eight moving parts — which is the cost side of this chapter, made concrete.

Finally, watch self-healing, since it is the capability everything else rests on:

```
kubectl create deployment recon --image=nginx:alpine --replicas=2
```

```
kubectl get pods -l app=recon
```

```
kubectl delete pod -l app=recon --field-selector status.phase=Running --wait=false  
| head -1
```

Within a few seconds a replacement appears with a new name and an age of zero. Nothing issued a "create a pod" command; a controller noticed a difference and closed it.

Clean up:

```
kubectl delete deployment recon
```

Takeaways

- Kubernetes is a control loop with an API: describe desired state, controllers converge towards it continuously.
- The real capabilities are self-healing, rolling updates, service discovery, bin-packing, autoscaling, and a uniform extensible API.
- The real costs are conceptual surface (71 API types out of the box), cluster operations, five debugging layers instead of four, and money.
- Adopt for a **present constraint**, not anticipated scale. For one service, a managed container service is usually better.
- It is not a PaaS, not a hard security boundary, and not as portable as claimed.
- Everything you know about containers still applies — and bad container practice hurts more here.

Next: Chapter 2 — Declarative reconciliation

Chapter 2 — Declarative reconciliation

If you take one idea from this book, take this one. Everything Kubernetes does — rollouts, self-healing, autoscaling, certificate management, the entire operator ecosystem — is the same mechanism applied to different objects.

`kubectl apply` **is not a command**

The instinct carried over from Docker is that `kubectl apply` *does* something, the way `docker run` does. It does not. It writes a record.

```
kubectl apply -f 01-deployment.yaml
```

That request means: "the cluster should contain a Deployment named `pingd` with these properties." The API server validates it, stores it in etcd, and returns. **No pod has been created.** Nothing has been scheduled. The API server does not create pods; it does not know how.

What happens next is that several independent controllers, each watching for changes to things they care about, notice and act:

1. The **Deployment controller** sees a Deployment with no matching ReplicaSet, and creates one.
2. The **ReplicaSet controller** sees a ReplicaSet wanting 3 pods and finding 0, and creates 3 Pod objects — still just records, with no node assigned.
3. The **scheduler** sees Pods with an empty `spec.nodeName`, picks nodes for them, and writes the choice back.
4. The **kubelet** on each chosen node sees a Pod assigned to it, and tells containerd to pull the image and start the container.
5. The kubelet reports status back; the ReplicaSet and Deployment controllers update their own status.

Five actors, each doing one job, none of them talking to each other directly. They communicate only by reading and writing objects through the API server. That is the architecture, and it explains almost every behaviour you will encounter.

The control loop

Each controller runs the same loop, forever:

```
observe desired state (the spec, from the API)
observe actual state (the status, from the API or the world)
if they differ: take one step to close the gap
repeat
```

Three properties follow, and they are worth stating because they explain a lot of otherwise surprising behaviour.

It is level-triggered, not edge-triggered. A controller does not react to "a pod was deleted". It periodically observes that there are 2 pods where there should be 3, and creates one. If it misses an event, restarts, or is

disconnected for a minute, it recovers by simply looking again. This is why Kubernetes is robust to controllers crashing — there is no queue of missed work to replay, only a current difference to close.

It is continuous. Reconciliation does not stop after the rollout finishes. Delete a pod a week later and it comes back, because the loop never stopped comparing.

It converges rather than executing steps. If you change replicas from 3 to 10 and then, two seconds later, to 5, you do not get 10 pods and then a scale-down. The controller reads current desired state each time round, so it converges on 5.

Measured

Delete a pod from a healthy 3-replica Deployment and time the response:

```
replacement pod object created after: 0.44 s
all replicas Ready again after:      4.81 s
```

Under half a second from deletion to a new Pod object existing. Nothing was instructed to do that; the ReplicaSet controller observed 2 where it wanted 3.

The events record each actor's contribution:

Killing	pingd-...-fmfvv	Stopping container api
SuccessfulCreate	pingd-5975cc6496	Created pod: pingd-...-67prp
Pulled	pingd-...-67prp	Container image "pingd:latest" already
present		
Created	pingd-...-67prp	Container created
Started	pingd-...-67prp	Container started
Unhealthy	pingd-...-67prp	Startup probe failed: connect: connection
refused		

Note the last line. A failed startup probe during boot is **normal** — the container is up before the server is listening, and the probe retries. Chapter 9 explains why that is what `startupProbe` is for, and Chapter 17 explains how to tell this benign case from a real failure.

Spec and status

Every Kubernetes object has the same shape:

```
apiVersion: apps/v1      # which API group and version
kind: Deployment         # what type of thing
metadata:                # name, namespace, labels, annotations, uid
  name: pingd
spec:                    # what you want — you write this
  replicas: 3
status:                  # what is — controllers write this
  readyReplicas: 3
```

You write spec. Controllers write status. Editing `status` by hand is meaningless; the controller will overwrite it on the next pass. This division is absolute and is the clearest signal of whether a field is an input or an output.

Look at what the API server added to a Deployment you never wrote:

```
kubectl get deployment pingd -o json | jq '.metadata | keys, .status | keys'
```

```
metadata: annotations, creationTimestamp, generation, labels, name,
          namespace, resourceVersion, uid
status:   availableReplicas, conditions, observedGeneration, readyReplicas,
          replicas, terminatingReplicas, updatedReplicas
```

Four of those are load-bearing:

`uid` — a unique identifier for this object instance. Delete and recreate a Deployment with the same name and it gets a new uid; that is how controllers know it is a different object.

`generation` **and** `observedGeneration` — `generation` increments each time you change the *spec*. `observedGeneration` is how far the controller has got. When they are equal, the controller has seen your latest change. When they differ, it has not caught up yet, which is exactly what `kubectl rollout status` is waiting for.

`resourceVersion` — changes on every write, and is the basis of optimistic concurrency. Measured, adding an annotation:

```
before: 1006
after:  1211
```

When you `kubectl edit`, the version you fetched is sent back; if someone else wrote in the meantime, the API server rejects your update with a conflict rather than silently losing their change. It is also what `watch` uses to resume a stream without missing events.

`conditions` — the standard way controllers report. Not a single status field but a list of independent assertions:

```
kubectl get deployment pingd -o jsonpath='{range .status.conditions[*]}{.type}
={.status} reason={.reason}{"\n"}{end}'
```

```
Available=True    reason=MinimumReplicasAvailable
Progressing=True  reason=NewReplicaSetAvailable
```

Conditions are the first thing to read when something is wrong, on any object type. They carry a `reason` and usually a `message` explaining precisely what a controller is unhappy about.

Ownership and cascading deletion

Objects created by controllers carry an `ownerReferences` field pointing at their creator:

```
kubectl get pods -l app=pingd -o jsonpath='{range .items[*]}{.metadata.name}{
owner="}{.metadata.ownerReferences[0].kind}{" / "}{.metadata.ownerReferences[0].name}
{"\n"}{end}'
```

```
pingd-5975cc6496-fmfvv owner=ReplicaSet/pingd-5975cc6496
pingd-5975cc6496-mzrzk owner=ReplicaSet/pingd-5975cc6496
pingd-5975cc6496-wxcdc owner=ReplicaSet/pingd-5975cc6496
```

And the ReplicaSet in turn is owned by the Deployment. This chain is what makes `kubectl delete deployment pingd` remove the pods too: the **garbage collector** is another controller, which deletes objects whose owner no longer exists.

It also explains a class of confusion. Delete a ReplicaSet directly and the Deployment controller creates a new one within a second, because from its point of view a required object went missing. You cannot delete the middle of an ownership chain and have it stay deleted — you have to change the desired state at the top.

To deliberately keep the children:

```
kubectl delete deployment pingd --cascade=orphan
```

The pods survive, ownerless, and nothing manages them. Occasionally useful in a migration; a mess if done by accident.

Why things sometimes "come back"

A short list of behaviours that make sense only through this lens:

- **A deleted pod returns.** Its ReplicaSet still wants it.
- **An edited pod reverts.** You edited a pod the ReplicaSet manages; it does not reconcile pod *contents*, but any replacement comes from the template, so your change vanishes at the next restart. Edit the Deployment, not the pod.
- **A scaled Deployment un-scales.** Something else — an HPA (Chapter 25), or a GitOps controller (Chapter 27) — also has an opinion about `replicas` and is writing it back. Two controllers fighting over one field is a genuine and confusing failure mode.
- **A deleted namespace hangs in Terminating.** A finalizer on some object inside it has not completed. Chapter 21 covers finalizers.
- **Your change had no effect.** Check `generation` versus `observedGeneration`: if they differ, no controller has processed it, which usually means the responsible controller is down.

Imperative commands still exist

`kubectl create`, `kubectl scale`, `kubectl set image`, `kubectl expose` all work by writing objects, and they are fine for experimentation:

```
kubectl scale deployment pingd --replicas=5
```

But they leave no record anywhere except the cluster. The next `kubectl apply` from your manifests will set `replicas` back to 3, silently. That is not a bug — it is the system doing exactly what you asked, twice, with different desired states.

The production discipline is: **manifests in git are the desired state, and the cluster is a derivative**. Chapter 27 makes that concrete with GitOps. For now, treat imperative commands as debugging tools, not deployment tools.

A useful middle ground is generating manifests rather than applying them:

```
kubectl create deployment pingd --image=pingd:latest --dry-run=client -o yaml > deployment.yaml
```

Try it

Watch the ownership chain assemble itself from a single object:

```
kubectl apply -f examples/manifests/01-deployment.yaml
```

```
kubectl get deployment,replicaset,pods -l app=pingd
```

You created one object; three kinds exist.

Prove that controllers are continuous rather than one-shot. Delete a pod and watch:

```
kubectl delete pod -l app=pingd --field-selector status.phase=Running --wait=false | head -1; kubectl get pods -l app=pingd -w
```

(Ctrl-C when it settles.) Now try to defeat it by deleting the ReplicaSet:

```
kubectl delete rs -l app=pingd && sleep 2 && kubectl get rs -l app=pingd
```

A new ReplicaSet, seconds old. The Deployment still wants one.

Watch generation track your changes:

```
kubectl get deployment pingd -o jsonpath='{gen={.metadata.generation} observed={.status.observedGeneration}}{"\n"}'
```

```
kubectl scale deployment pingd --replicas=4 && kubectl get deployment pingd -o jsonpath='{gen={.metadata.generation} observed={.status.observedGeneration}}{"\n"}'
```

Then see the imperative/declarative conflict for yourself — `apply` silently reverts your scale:

```
kubectl apply -f examples/manifests/01-deployment.yaml && kubectl get deployment pingd -o jsonpath='{replicas={.spec.replicas}}{"\n"}'
```

Back to 3. Nothing warned you.

Takeaways

- `kubectl apply` writes a record. Controllers do the work, asynchronously, by watching for differences.

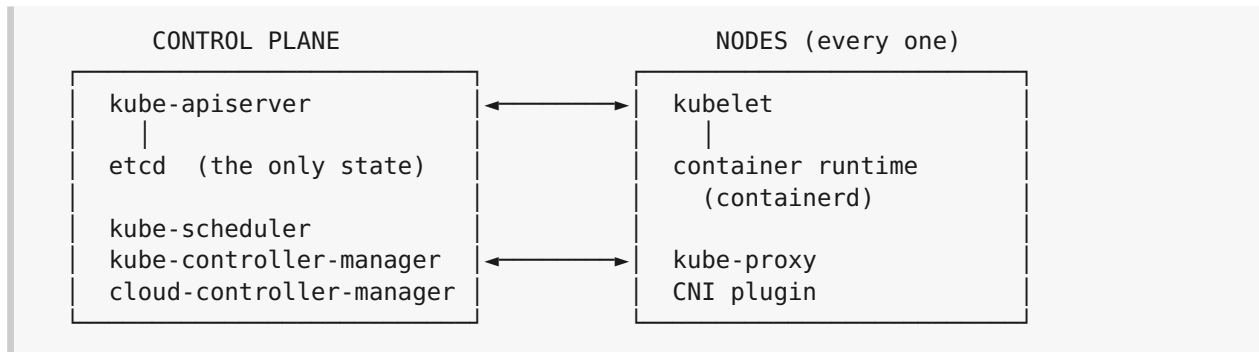
- Five actors turn a Deployment into a running container: Deployment controller, ReplicaSet controller, scheduler, kubelet, and the status path back. They communicate only through the API server.
 - Control loops are **level-triggered and continuous** — they observe current state rather than reacting to events, which is why they survive restarts and missed messages.
 - You write `spec`; controllers write `status`. `generation` vs `observedGeneration` tells you whether your change has been seen; `conditions` tell you what a controller thinks.
 - `ownerReferences` drive cascading deletion. You cannot delete the middle of the chain and have it stay deleted.
 - Imperative commands write desired state with no record. `apply` will overwrite them silently. Manifests in git are the source of truth.
-

Previous: Chapter 1 — Why Kubernetes · Next: Chapter 3 — Cluster anatomy

Chapter 3 — Cluster anatomy

Chapter 2 described five actors. This chapter says where they live, what each one owns, and what breaks when each fails — which is the knowledge Part IV depends on.

The two halves



Everything talks to the API server. Nothing talks to anything else. The scheduler does not call the kubelet; the kubelet does not call the scheduler. This hub-and-spoke design is why components can be restarted, upgraded and replaced independently, and why a single component failing degrades the cluster in a *specific* way rather than breaking everything.

On this book's cluster, the control plane runs as pods on the control-plane node:

```
kubectl get pods -n kube-system
```

```

etcd-k8sbook-control-plane           Running
kube-apiserver-k8sbook-control-plane  Running
kube-controller-manager-k8sbook-control-plane  Running
kube-scheduler-k8sbook-control-plane  Running
kube-proxy-5z5br / -kvkkt / -vwlf2      Running   (one per node)
kindnet-m7t7c / -rplnh / -54tzh        Running   (CNI, one per node)
coredns-559f6c778d-htbxc / -tcpbb      Running
  
```

Those four control-plane pods are **static pods**: manifests in `/etc/kubernetes/manifests` on the node, started by the kubelet directly rather than by a controller. This solves a bootstrapping problem — the API server cannot be scheduled by a system that requires the API server. Chapter 21 uses this: you can fix a broken API server by editing a file on the node, because the kubelet is watching that directory.

On managed clusters (EKS, GKE, AKS) the control plane is invisible — you will not see these pods, cannot access etcd, and the provider handles upgrades. Most of this chapter still matters for understanding behaviour; the operational parts do not.

kube-apiserver

The front door and the only component that talks to etcd. Everything else — kubectl, the controllers, the kubelets, your operators — reaches state through it.

It does five things to every request: **authentication** (who are you), **authorization** (RBAC — may you), **admission** (mutating then validating webhooks, Chapter 24), **validation** against the schema, and **persistence** to etcd.

That ordering matters for debugging. A request rejected for RBAC never reaches admission; a request rejected by a webhook never reaches etcd. When something "will not apply", the error message tells you which stage refused it, and they are very different problems.

It is stateless and horizontally scalable — production clusters run three behind a load balancer.

When it fails: `kubectl` stops working entirely, and no controller can observe or act. But **running pods keep running**. The kubelet continues managing containers it already knows about, and containers do not stop. An API server outage is a control-plane outage, not a data-plane outage. This distinction is worth internalising before your first incident.

etcd

A distributed key-value store, and the **only** stateful component. Every object in the cluster is a key in etcd. Lose etcd without a backup and you have lost the cluster's definition of itself — nodes and containers keep running, but nothing knows what should exist.

It also provides the **watch** primitive: clients stream changes rather than polling, which is what makes level-triggered controllers efficient at scale.

Practical facts:

- Run an odd number of members (3 or 5) for quorum. With 3, one may fail; with 5, two.
- It is **latency-sensitive to disk fsync**. Slow disks are the most common cause of a mysteriously sluggish cluster — the symptom is high API latency across everything, and the cause is not obviously etcd at all.
- Objects have practical size limits (~1 MiB default), which is why very large ConfigMaps and Secrets cause trouble.
- **Back it up**. Chapter 21 has the command. A managed control plane does this for you; a self-hosted one does not unless you set it up.

kube-scheduler

Watches for Pods with no `spec.nodeName` and assigns each one a node. That is its whole job — it does not start containers, and once it has written the node name it never looks at that pod again.

It runs two phases:

Filtering — eliminate nodes that cannot run this pod. Insufficient allocatable CPU or memory, taints not tolerated, node selectors or affinity unmatched, no available ports, volume topology constraints, unschedulable nodes.

Scoring — rank the survivors. Least-requested resources, image already present on the node, spreading across zones and hosts, affinity preferences.

Two consequences that cause real confusion:

Scheduling uses requests, not actual usage. A node with 8 idle CPUs whose pods have *requested* all 8 will refuse a new pod requesting 1, even though nothing is using anything. Conversely, a node can be genuinely overloaded while accepting more pods because the running ones requested very little. Chapter 8 is about this, and Chapter 17 covers the resulting `Pending`.

The decision is a point-in-time snapshot. The scheduler does not rebalance. A pod placed on a busy node stays there when the cluster empties out, and nothing moves it. Rebalancing requires a separate tool (`descheduler`) or a pod restart.

When it fails: existing pods are unaffected; new pods stay `Pending` forever with no node assigned. A pod stuck `Pending` with *no events at all* is the signature of an absent scheduler, as opposed to a scheduler that ran and found nowhere to put it (which produces a `FailedScheduling` event explaining why).

kube-controller-manager

A single binary running dozens of controllers in separate goroutines: `Deployment`, `ReplicaSet`, `StatefulSet`, `DaemonSet`, `Job`, `CronJob`, `Node`, `ServiceAccount`, `EndpointSlice`, `PersistentVolume`, `namespace`, garbage collection, and more.

Each is an independent control loop of the kind Chapter 2 described.

When it fails: nothing self-heals. Deleted pods are not replaced, rollouts stall, `EndpointSlices` stop updating so `Services` keep sending traffic to dead pods, and namespaces will not finish deleting. Existing healthy traffic is unaffected, which makes this failure quiet and nasty — everything looks fine until something needs to change.

The **cloud-controller-manager** is the provider-specific half: provisioning load balancers for `type: LoadBalancer` `Services`, attaching cloud disks, and labelling nodes with zone and instance type. On `kind` there is none, which is why a `LoadBalancer` `Service` stays `<pending>` forever on a local cluster (Chapter 11).

kubelet

The agent on every node, and the component that actually makes containers exist. It:

- watches the API server for Pods assigned to *its* node;
- tells the container runtime (`containerd`) to pull images and start containers, via CRI;
- runs **probes** and reports results (Chapter 9);
- reports node and pod status back;
- manages volume mounts;
- enforces eviction when the node is under resource pressure (Chapter 20);
- runs **static pods** from its manifest directory.

Note who runs probes: **the kubelet, locally**. Probes are not run from the control plane, which is why a liveness probe still restarts your container during an API server outage, and why probe traffic never crosses the network between nodes.

When it fails: the node goes `NotReady` after a grace period (~40 s of missed heartbeats). Its containers *keep running* — nothing stops them — but nothing manages them either. After a further timeout (5 minutes by

default) the node controller marks the pods for deletion and they are rescheduled elsewhere, which means for those five minutes the workload may be running in two places. Chapter 21 covers this properly; it is the source of the "split brain" worry people have about node failure, and the mitigation is application-level.

kube-proxy and the CNI plugin

Two different things, frequently conflated.

The CNI plugin (Calico, Cilium, Flannel, or `kindnet` here) gives pods their IP addresses and makes pod-to-pod traffic route across nodes. Kubernetes itself does not implement pod networking — it defines the requirements and delegates. The requirement is strong and worth stating: **every pod gets a unique IP, and every pod can reach every other pod directly, without NAT.** No port mapping, no NAT between pods, which is a much simpler model than Docker's default bridge.

Your CNI choice determines whether NetworkPolicy works at all (Chapter 14), and its performance characteristics.

kube-proxy implements *Services*: it watches Services and EndpointSlices and programs the node so that traffic to a Service's virtual IP is load-balanced to a pod IP. In `iptables` mode it writes iptables rules; in `ipvs` mode it uses the kernel's load balancer, which scales better with many Services. Some CNIs (Cilium) replace kube-proxy entirely with eBPF.

When kube-proxy fails on a node: pods on that node cannot reach Service IPs — but *can* reach pod IPs directly. That asymmetry is a strong diagnostic signal (Chapter 19).

CoreDNS

Cluster DNS, running as an ordinary Deployment in `kube-system`. It resolves Service names to ClusterIPs, and pod DNS is configured to point at it.

When it fails: name resolution breaks, so almost everything appears broken while IP-based traffic works fine. Because it is a normal Deployment on normal nodes, it can be evicted, throttled or OOM-killed like anything else — and a CPU-throttled CoreDNS produces intermittent, latency-shaped failures across the entire cluster that look like anything but DNS. Chapter 12 covers this.

The data plane keeps running

The single most useful operational fact in this chapter, so it gets its own section:

If the entire control plane disappears, running pods keep serving traffic.

Containers keep running. kube-proxy's existing rules keep routing. CoreDNS keeps resolving, if its pods are healthy. What you lose is *change*: no new pods, no rescheduling, no scaling, no rollouts, no self-healing, and no `kubectl`.

This means a control plane outage is serious but not immediately customer-facing, and it changes how you triage. It also means "the cluster is down" needs qualifying before you can act on it.

Try it

Look at your control plane:

```
kubectl get pods -n kube-system -o custom-  
columns='NAME:.metadata.name,NODE:.spec.nodeName,STATUS:.status.phase'
```

Confirm the control-plane components are static pods — note the `-<nodename>` suffix and the absence of a ReplicaSet owner:

```
kubectl get pod -n kube-system kube-apiserver-k8sbook-control-plane -o  
jsonpath='{.metadata.ownerReferences[0].kind}{"\n"}'
```

Node

Owned by the *Node*, not a controller — the signature of a static pod. See them on disk:

```
docker exec k8sbook-control-plane ls /etc/kubernetes/manifests/
```

See what the scheduler decided, and that it used requests:

```
kubectl get pods -o wide -l app=pingd
```

```
kubectl describe node k8sbook-worker | sed -n '/Allocated resources/,/^Events/p'
```

That last block shows requests versus allocatable — the numbers the scheduler actually uses, which are unrelated to current utilisation.

Watch the API server's request path reject something at the authorization stage:

```
kubectl auth can-i delete nodes --as=system:serviceaccount:default:default
```

no

Prove pod-to-pod networking needs no NAT — connect straight to a pod IP, bypassing Services entirely:

```
POD_IP=$(kubectl get pod -l app=pingd -o jsonpath='{.items[0].status.podIP}') &&  
kubectl run direct --rm -it --restart=Never --image=curlimages/curl:8.11.1 --quiet  
-- curl -sS "http://$POD_IP:8000/"
```

Finally, see the node heartbeat that determines `Ready`:

```
kubectl get node k8sbook-worker -o jsonpath='{range .status.conditions[*]}{.type}  
={.status} {(.reason)}{"\n"}{end}'
```

Takeaways

- Everything talks to the API server; nothing talks to anything else. Components fail independently and in characteristic ways.
 - **etcd is the only state.** Back it up. It is fsync-latency-sensitive, and slow disks present as cluster-wide API slowness.
 - The scheduler assigns nodes using **requests**, once, and never rebalances.
 - The controller-manager is where self-healing lives; when it is down, everything looks fine until something needs to change.
 - The kubelet runs probes **locally** and makes containers exist. A **NotReady** node keeps running its containers for ~5 minutes before they are rescheduled elsewhere.
 - CNI gives pods IPs and NAT-free pod-to-pod routing; kube-proxy implements Service VIPs. "Pod IP works, Service IP does not" points at kube-proxy.
 - Static pods are owned by the Node and read from a directory on disk — which is how you fix a broken control plane.
 - **A control plane outage stops change, not traffic.**
-

Previous: Chapter 2 — Declarative reconciliation · Next: Chapter 4 — kubectl and the resource model

Chapter 4 — kubectl and the resource model

Kubernetes is an API with a uniform object model. Once you know the shape of that model, resources you have never met behave predictably — which is what makes the ecosystem's hundreds of custom resource types tractable rather than overwhelming.

Everything is an object

Same five top-level fields, every time:

```
apiVersion: apps/v1
kind: Deployment
metadata: {}
spec: {}
status: {}
```

`apiVersion` is `group/version`, except for the original core resources (Pod, Service, ConfigMap, Secret, Node, Namespace), which are in the empty group and just say `v1`. That inconsistency is historical and permanent.

On this cluster:

```
kubectl api-resources --no-headers | wc -l      # 71 types
kubectl api-versions | wc -l                    # 23 group/versions
```

Of the 71 types, **34 are namespaced and 37 are cluster-scoped**:

```
kubectl api-resources --namespaced=true --no-headers | wc -l    # 34
kubectl api-resources --namespaced=false --no-headers | wc -l   # 37
```

Cluster-scoped means it exists once for the whole cluster — Nodes, Namespaces, PersistentVolumes, StorageClasses, ClusterRoles, CustomResourceDefinitions. Attempting `kubectl get nodes -n default` produces a warning, because the namespace is meaningless:

```
Warning: resource 'nodes' is not namespace scoped
```

Knowing which is which matters for RBAC (Chapter 23): a Role grants access within one namespace and simply *cannot* grant access to a cluster-scoped resource, no matter how you write it. You need a ClusterRole.

Discovering the API

Three commands worth knowing before anything else, because they replace searching the web for field names.

```
kubectl api-resources
```

Every type, its short name, its API group, and whether it is namespaced. The `SHORTNAMES` column is where `po`, `svc`, `deploy`, `rs`, `ns`, `cm`, `sa`, `pvc` come from.

```
kubectl explain deployment.spec.strategy
```

```
GROUP:      apps
KIND:       Deployment
VERSION:    v1

FIELD: strategy <DeploymentStrategy>
```

Documentation for any field of any type, generated from the API schema — including custom resources you installed five minutes ago. `--recursive` prints the whole subtree. This is the authoritative answer to "what fields does this thing have", and it is always correct for *your* cluster version, which a web search is not.

```
kubectl explain deployment.spec --recursive | head -40
```

Namespaces

A namespace is a name-scoping mechanism with policy attachment points. It is **not** a security boundary on its own (Chapter 28), and it does not isolate the network unless you add NetworkPolicy (Chapter 14).

```
kubectl get namespaces
```

```
kubectl get pods -n kube-system
kubectl get pods --all-namespaces
```

Every cluster has `default` (do not use it for real work), `kube-system` (control plane), `kube-public` (world-readable cluster info) and `kube-node-lease` (node heartbeats).

Set a default so you stop typing `-n`:

```
kubectl config set-context --current --namespace=myapp
```

Namespaces give you: name uniqueness, a target for RBAC, a target for ResourceQuota and LimitRange, and a scope for NetworkPolicy. That is a lot, but note what is missing — nothing about node isolation or kernel isolation. Pods from different namespaces run side by side on the same nodes, sharing a kernel.

Labels and selectors

This is the mechanism that ties Kubernetes together, and it is worth more attention than it usually gets.

Labels are arbitrary key-value pairs for identification. **Selectors** query them. The loose coupling between almost all Kubernetes objects is a label selector:

- A Service finds its pods by selector — it knows nothing about your Deployment.
- A ReplicaSet owns pods by selector.
- A NetworkPolicy targets pods by selector.
- A PodDisruptionBudget, an HPA target, a topology spread constraint — all selectors.

```
kubectl get pods -l app=pingd
kubectl get pods -l 'app in (pingd, web)'
kubectl get pods -l 'app=pingd,version!=canary'
kubectl get pods -l '!canary'           # label absent
```

The consequence that bites: **a Service selecting labels no pod has is not an error**. It exists, has a ClusterIP, and routes to nothing. Traffic to it fails, and nothing anywhere says "your selector is wrong". Chapter 19 makes checking for this the first step in Service debugging.

Recommended labels — worth adopting because tooling understands them:

```
metadata:
  labels:
    app.kubernetes.io/name: pingd
    app.kubernetes.io/instance: pingd-prod
    app.kubernetes.io/version: "1.4.2"
    app.kubernetes.io/component: api
    app.kubernetes.io/part-of: platform
```

Annotations look similar and are for something else entirely: non-identifying metadata, not queryable, often large, and frequently written by tools rather than humans — checksums to force rollouts, ingress controller configuration, `kubectl.kubernetes.io/last-applied-configuration`. If you want to select on it, it is a label; if you want to attach data to it, it is an annotation.

The kubectl verbs

```
kubectl get pods           # list
kubectl get pod NAME -o yaml # full object
kubectl describe pod NAME  # human summary + events
kubectl logs NAME          # container output
kubectl exec -it NAME -- sh # run a command inside
kubectl apply -f manifest.yaml # declarative write
kubectl delete -f manifest.yaml
kubectl edit deployment NAME # fetch, edit, write back
kubectl port-forward svc/pingd 8080:80 # local tunnel
```

Three of those deserve elaboration.

`describe` **versus** `get -o yaml`. `describe` gives you a human summary *plus the object's recent events*, which `get -o yaml` does not include. For debugging, `describe` is almost always the right first command — the events are usually the answer.

`logs` **flags you will use constantly**:

```
kubectl logs -f --tail=100 --timestamps deploy/pingd
kubectl logs POD -c CONTAINER           # multi-container pod
kubectl logs POD --previous              # the container that just crashed ←
crucial
kubectl logs -l app=pingd --prefix --tail=20 # across all matching pods
kubectl logs POD --since=10m
```

`--previous` is the one people miss. When a container is crash-looping, `kubectl logs` shows the *current* attempt, which may have produced nothing yet; `--previous` shows the instance that actually died, with the error in it.

`-o` output formats:

```
kubectl get pods -o wide                # + node and pod IP
kubectl get pods -o json | jq '...'
kubectl get pods -o jsonpath='{.items[*].spec.nodeName}'
kubectl get pods -o custom-columns='NAME:.metadata.name,NODE:.spec.nodeName,QOS:.status.qosClass'
kubectl get pods -o name                # pod/x - feeds other commands
```

`custom-columns` and `jsonpath` are how you stop scrolling. Most examples in this book use them.

Events

Events are the cluster's narration of what controllers did, and they are the single most underused debugging resource.

```
kubectl get events --sort-by=.lastTimestamp
```

```
kubectl get events --field-selector involvedObject.name=POD_NAME
```

```
kubectl events --for pod/POD_NAME --watch
```

Two critical caveats. **Events expire**, by default after one hour — so "there are no events" on an old problem means nothing. And `kubectl get events` **is not sorted by default**, which makes it nearly useless without `--sort-by`; that is a long-standing wart, not something you are doing wrong.

For anything you need to keep, events must be shipped somewhere (Chapter 26).

Dry runs and diffs

Two commands that prevent a lot of damage:

```
kubectl apply -f manifest.yaml --dry-run=server
```

Sends the object to the API server for full validation — schema, admission webhooks, defaulting — without persisting it. Much stronger than `--dry-run=client`, which only validates locally and misses everything interesting.

```
kubectl diff -f manifest.yaml
```

Shows exactly what applying would change against live state. Run this before any `apply` you are unsure about; it is the difference between confidence and hoping.

Server-side apply and field ownership

Modern `kubectl apply` uses **server-side apply**, where the API server tracks which manager owns which field:

```
kubectl get deployment pingd -o yaml | grep -A5 managedFields | head
```

This is why you sometimes get:

```
error: Apply failed with 1 conflict: conflict with "kubectl-edit": .spec.replicas
```

Another manager — an HPA, a GitOps controller, or your own earlier `kubectl edit` — owns that field. The error is Kubernetes preventing two sources of truth from silently fighting, which is a feature. `--force-conflicts` takes ownership, and you should understand *what* you are taking it from before using it.

Contexts

```
kubectl config get-contexts
kubectl config use-context CLUSTER
kubectl config current-context
```

`kubectl` reads `~/.kube/config`, which may hold many clusters. **Running the right command against the wrong cluster is a genuine and common production incident.** Mitigations worth adopting: put the current context in your shell prompt (`kube-ps1`, `starship`), use `kubectx/kubens`, and keep production in a separate `kubeconfig` file selected explicitly via `KUBECONFIG`.

Try it

Explore the API rather than searching the web for it:

```
kubectl api-resources | head -20
```

```
kubectl explain pod.spec.containers.resources
```

```
kubectl explain deployment.spec.strategy.rollingUpdate
```

See the namespaced/cluster-scoped split:

```
kubectl api-resources --namespaced=false --no-headers | wc -l; kubectl api-resources --namespaced=true --no-headers | wc -l
```

Watch what `describe` gives you that `get -o yaml` does not:

```
kubectl describe pod -l app=pingd | tail -15
```

Those trailing events are the payoff.

Prove that a Service with a wrong selector fails silently — this is Chapter 19's most common bug, created deliberately:

```
kubectl create service clusterip broken --tcp=80:8000 --dry-run=client -o yaml |
kubectl apply -f -
```

```
kubectl get svc broken
```

It has a ClusterIP and looks completely healthy. Now look for endpoints:

```
kubectl get endpointslice -l kubernetes.io/service-name=broken
```

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
broken-hq48j	IPv4	<unset>	<unset>	0s

An EndpointSlice is created — it is simply empty. That is the tell: `ENDPOINTS` showing `<unset>` rather than a list of pod IPs. No error, no warning, no event — just a Service that silently discards traffic.

```
kubectl delete svc broken
```

Finally, use `diff` before an `apply`:

```
kubectl diff -f examples/manifests/01-deployment.yaml || true
```

Takeaways

- Every object has `apiVersion`, `kind`, `metadata`, `spec`, `status`. Learn the shape once and every custom resource is familiar.
- `kubectl api-resources` and `kubectl explain` are the authoritative documentation for *your* cluster version. Use them instead of searching.
- 71 types on a bare cluster: 34 namespaced, 37 cluster-scoped. The distinction determines whether a Role or ClusterRole is even possible.
- Namespaces scope names, RBAC, quotas and policy. They are not a security or network boundary by themselves.
- **Label selectors are the coupling between nearly all objects**, and a selector matching nothing is never an error — it just silently does nothing.
- `describe` includes events; `get -o yaml` does not. Events expire after an hour and are unsorted by default.
- `kubectl logs --previous` shows the container that actually crashed.
- `--dry-run=server` and `kubectl diff` before applying. Server-side apply conflicts mean something else owns that field — find out what before forcing.
- Wrong-context accidents are real. Put the context in your prompt.

[Previous: Chapter 3 — Cluster anatomy](#) · [Next: Chapter 5 — Pods](#)

Chapter 5 — Pods

The Pod is the atom. Kubernetes does not schedule containers; it schedules Pods. Every higher-level workload type — Deployment, StatefulSet, Job, DaemonSet — exists to create and manage Pods, and every one of them contains a pod template.

What a Pod is

A group of containers that share a network namespace, IPC namespace and storage volumes, scheduled together on one node, sharing a lifecycle.

Concretely, containers in a Pod:

- share one **IP address** and one port space, so they reach each other on `localhost`;
- can share **volumes**;
- are always **co-located** on the same node;
- are created and destroyed **together**.

They do *not* share a filesystem root (each has its own image) and, by default, do *not* share a PID namespace.

Measured on a two-container Pod. The sidecar reaching the server over `localhost`:

```
kubectl exec multi -c sidecar -- wget -qO- http://localhost:8080/
```

```
prepared by init at 12:21:15
```

Same network namespace, no Service involved. But their process views are separate:

```
kubectl exec multi -c server -- ps -o pid,comm
kubectl exec multi -c sidecar -- ps -o pid,comm
```

```
server:  PID 1 httpd
sidecar: PID 1 sleep
```

Each container is PID 1 in its own namespace. Enable sharing explicitly with `shareProcessNamespace: true` when you need one container to see or signal another's processes — useful for debugging sidecars, and a security consideration since it also lets them read each other's `/proc`.

Why the Pod exists at all

The abstraction is not arbitrary. Some things genuinely need to be co-located with shared network and storage — a log shipper reading a volume the app writes, a proxy terminating TLS on the app's behalf, a credential refresher writing to a shared tmpfs. Making the *pod* rather than the container the schedulable unit means "these processes must live together" is expressible.

The Docker book's advice about one process per container becomes, here: **one concern per container, one cohesive unit per Pod**. If two containers do not need to share a network namespace or a volume, they probably want to be separate Pods.

The infrastructure container

Every Pod has a hidden `pause` container that holds the namespaces. It does nothing but sleep and reap. Application containers join *its* namespaces, which is why individual containers in a Pod can restart without the Pod losing its IP address.

That detail explains an important behaviour: a crash-looping container does not change the Pod's IP, and the Pod is not rescheduled. Restarts happen in place, on the same node.

You rarely create Pods directly

```
apiVersion: v1
kind: Pod
metadata:
  name: standalone
spec:
  containers:
  - name: app
    image: pingd:latest
```

This works and you should almost never do it. A bare Pod has **nothing watching it**. If it dies, it stays dead. If its node dies, it is gone. There is no controller with a desired state that includes it.

Use a Deployment (Chapter 6) for services, a Job for batch work, a DaemonSet for per-node agents. Bare Pods are for debugging and one-off experiments — which is exactly what `kubectl run --rm -it` is for:

```
kubectl run tmp --rm -it --restart=Never --image=alpine -- sh
```

Init containers

Containers that run to completion, **in order**, before any app container starts. If one fails, the Pod restarts it according to the restart policy; app containers do not start until all init containers have succeeded.

```
spec:
  initContainers:
  - name: setup
    image: busybox:1.37
    command: ['sh', '-c', 'echo "prepared at $(date -u +%T)" > /work/index.html']
    volumeMounts:
    - name: shared
      mountPath: /work
  containers:
  - name: server
    image: busybox:1.37
    command: ['sh', '-c', 'httpd -f -p 8080 -h /work']
    volumeMounts:
    - name: shared
      mountPath: /work
```

The app container sees the init container's output:

```
kubectl exec multi -c server -- cat /work/index.html
```

```
prepared at 12:21:15
```

Legitimate uses: waiting for a dependency to be reachable, running database migrations, fetching configuration or secrets, setting file ownership on a volume, cloning a git repo.

Two properties make them useful beyond "run something first": init containers can use a **different image** with tools the app image lacks (keeping the runtime image minimal), and they run with the Pod's volumes already mounted.

The waiting pattern, which replaces application-level startup retries in some designs:

```
initContainers:
- name: wait-for-db
  image: busybox:1.37
  command: ['sh', '-c', 'until nc -z db 5432; do echo waiting; sleep 2; done']
```

Use it, but do not let it replace retry logic in the application (Chapter 9) — it helps at startup and does nothing when the database restarts later.

Sidecar containers

A sidecar runs *alongside* the app for the Pod's lifetime. The classic problem was that ordinary containers all start in parallel with no ordering, and a sidecar that must be ready first — a proxy, a credential agent — created race conditions.

Since v1.29 (stable in v1.33), sidecars are **init containers with** `restartPolicy: Always`:

```
spec:
  initContainers:
  - name: proxy
    image: envoyproxy/envoy:v1.32-latest
    restartPolicy: Always      # ← this makes it a sidecar
  containers:
  - name: app
    image: pingd:latest
```

That gives you what everyone wanted: the sidecar starts *before* app containers, keeps running alongside them, and shuts down *after* them. It also fixes the long-standing Job problem where a never-exiting sidecar prevented the Job from ever completing.

If you are reading older material describing sidecars as ordinary containers with startup races and Job hangs, this is the fix, and it is worth adopting.

The Pod lifecycle

`status.phase` has five values, and it is coarser than people expect:

Phase	Meaning
Pending	Accepted, but not all containers running. Scheduling, pulling images, or init containers running.
Running	Bound to a node, all containers created, at least one running.
Succeeded	All containers terminated successfully; will not restart.
Failed	All containers terminated, at least one in failure.
Unknown	The node cannot be reached.

Running **does not mean working**. It means containers exist. Readiness (Chapter 9) is a separate thing, which is why `kubectl get pods` shows both `STATUS` and `READY`:

NAME	READY	STATUS	RESTARTS	AGE
pingd-5975cc6496-fmfvv	1/1	Running	0	2m

`READY 1/1` is the number that determines whether traffic is sent. A pod can sit at `0/1 Running` indefinitely, healthy-looking and receiving nothing.

The real detail is in `status.containerStatuses`, and `describe` is the fastest way to it:

```
kubectl describe pod POD | sed -n '/Containers:/,/Conditions:/p'
```

Look for `State`, `Last State` (the previous run, if it restarted), `Reason`, and `Exit Code`. Chapter 17 works through these.

Restart policy

```
spec:
  restartPolicy: Always      # default. Also: OnFailure, Never
```

It applies to **containers within the Pod**, not to the Pod itself — this trips people up. `restartPolicy: Always` means the kubelet restarts a failed container **in place**, on the same node, with the same Pod IP. Nothing reschedules.

Deployments require `Always`. Jobs use `OnFailure` or `Never`.

Restarts use exponential backoff: 10 s, 20 s, 40 s, up to 5 minutes. That backoff is what `CrashLoopBackOff` is (Chapter 17) — the Pod is not in an error state, it is *waiting* before the next attempt.

Resources, probes, security context

Every Pod should set resource requests and limits (Chapter 8), probes (Chapter 9) and a security context (Chapter 23). They belong in this chapter's mental model but each has its own chapter, so here is only the shape:

```
spec:
  securityContext:                # pod-level
    runAsNonRoot: true
    runAsUser: 10001
    fsGroup: 10001
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: api
      resources:
        requests: { cpu: 50m, memory: 96Mi }
        limits:   { cpu: 500m, memory: 192Mi }
      livenessProbe:
        httpGet: { path: /healthz, port: http }
      securityContext:          # container-level, overrides pod-level
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities: { drop: ["ALL"] }
```

Pod-level `securityContext` applies to all containers; container-level overrides it for one. Some fields exist only at one level — `fsGroup` is pod-only, `capabilities` and `readOnlyRootFilesystem` are container-only — which is a common source of "why is this field rejected".

Scheduling controls

Where a Pod runs, in increasing order of expressiveness:

```
spec:
  nodeSelector:                # simplest: exact label match
  disktype: ssd
```

```
spec:
  affinity:
    nodeAffinity:              # expressive: operators, soft preferences
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: topology.kubernetes.io/zone
                operator: In
                values: ["eu-west-1a", "eu-west-1b"]
    podAntiAffinity:           # spread replicas across nodes
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchLabels: { app: pingd }
            topologyKey: kubernetes.io/hostname
```

Note `IgnoredDuringExecution` in those names: the rules apply **at scheduling time only**. If a node's labels change afterwards, nothing moves. There is no "RequiredDuringExecution" — it has been proposed for years and does not exist.

Tolerations let a Pod schedule onto a *tainted* node. Taints repel; tolerations permit:

```
kubectl taint nodes k8sbook-worker2 gpu=true:NoSchedule
```

```
spec:
  tolerations:
  - key: gpu
    operator: Equal
    value: "true"
    effect: NoSchedule
```

Control-plane nodes are tainted by default, which is why your workloads land on workers without you asking. Chapter 20 covers the `NoExecute` taints the node controller applies during failures, and Chapter 25 covers `topologySpreadConstraints`, which is the modern and better answer to spreading replicas.

Try it

Build a Pod that demonstrates all three sharing behaviours:

```
kubectl apply -f examples/manifests/05-multi-container.yaml
```

```
kubectl wait --for=condition=Ready pod/multi --timeout=60s
```

Shared network namespace — the sidecar reaches the server on localhost:

```
kubectl exec multi -c sidecar -- wget -qO- http://localhost:8080/
```

Separate PID namespaces — each container is its own PID 1:

```
kubectl exec multi -c server -- ps -o pid,comm; kubectl exec multi -c sidecar --
ps -o pid,comm
```

Shared volume, written by the init container before either app container started:

```
kubectl exec multi -c sidecar -- cat /work/index.html
```

See the whole lifecycle including init containers:

```
kubectl describe pod multi | sed -n '/Init Containers:/,/Conditions:/p'
```

Prove a bare Pod has nothing watching it:

```
kubectl delete pod multi && sleep 5 && kubectl get pod multi
```

```
Error from server (NotFound): pods "multi" not found
```

Gone, permanently. Compare with a Deployment's pod, which returns in under a second (Chapter 2).

Finally, watch scheduling constraints take effect:

```
kubectl taint nodes k8sbook-worker2 demo=true:NoSchedule
```

```
kubectl run tainted-test --image=alpine --restart=Never --overrides='{ "spec":  
{"nodeSelector":{"kubernetes.io/hostname":"k8sbook-worker2"}}}' -- sleep 60
```

```
sleep 3 && kubectl get pod tainted-test -o wide && kubectl describe pod tainted-  
test | grep -A3 Events
```

Pending, with a FailedScheduling event naming the taint. Clean up:

```
kubectl delete pod tainted-test --force --grace-period=0 2>/dev/null; kubectl  
taint nodes k8sbook-worker2 demo-
```

Takeaways

- A Pod is containers sharing a network namespace, IPC and volumes, co-scheduled and co-terminated. They do **not** share a PID namespace unless you ask.
- The `pause` container holds the namespaces, which is why a container can restart in place without the Pod losing its IP.
- Do not create bare Pods for real workloads — nothing watches them.
- Init containers run to completion, in order, before app containers, and may use a different image with tools the runtime image lacks.
- **Sidecars are now init containers with** `restartPolicy: Always` — correct ordering, and Jobs can finally complete.
- Running $\neq$ working. `READY 1/1` is what determines traffic.
- `restartPolicy` restarts containers **in place** on the same node; it never reschedules. `CrashLoopBackOff` is the backoff timer, not an error state.
- Affinity and node selectors apply **at scheduling time only** — nothing moves when labels change later.

Previous: Chapter 4 — kubectl and the resource model · Next: Chapter 6 — Deployments and ReplicaSets

Chapter 6 — Deployments and ReplicaSets

A Deployment is how you run a stateless service. It manages ReplicaSets, which manage Pods, and the indirection exists for exactly one reason: **rollouts**.

The two-level structure

```
Deployment —manages→ ReplicaSet —manages→ Pods
(rollout strategy)      (replica count)
```

- The **ReplicaSet controller** does one thing: keep N pods matching a selector alive. It has no concept of versions or updates.
- The **Deployment controller** creates a *new* ReplicaSet whenever the pod template changes, then shifts replicas from old to new according to your strategy.

You can see both after an update:

```
kubectl get rs -l app=pingd -o custom-
columns='NAME:.metadata.name,DESIRED:.spec.replicas,CURRENT:.status.replicas,REVISION:.metadata.annotations.deployment\.kubernetes\.io/revision'
```

NAME	DESIRED	CURRENT	REVISION
pingd-5975cc6496	0	0	1
pingd-79fd4dd7cc	3	3	2

The old ReplicaSet is **kept at zero replicas**, not deleted. That is what makes rollback instant: rolling back does not rebuild anything, it just scales the old ReplicaSet back up. `revisionHistoryLimit` (default 10) controls how many are retained.

The hash in the name (5975cc6496) is computed from the pod template. Identical templates produce identical hashes, which is why re-applying an unchanged manifest does not trigger a rollout.

Rollouts

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

- `maxSurge` — how many pods above `replicas` may exist during the update.
- `maxUnavailable` — how many below `replicas` may be unavailable.

Both accept counts or percentages. The defaults are 25% for each, which means a default Deployment **goes below capacity during every rollout**. For a service under load, that is usually wrong.

`maxSurge: 1`, `maxUnavailable: 0` is the safe production setting: start one new pod, wait for it to be *ready*, then remove one old pod. Capacity never dips. The cost is that you need room for one extra pod and the rollout is slightly slower.

`maxUnavailable: 0` with `maxSurge: 0` is invalid — nothing could ever happen — and the API server rejects it.

Watch a rollout:

```
kubectl rollout status deployment/pingd
```

```
deployment "pingd" successfully rolled out
```

This blocks until complete and **exits non-zero on failure**, which makes it the right thing to put in a deploy script. Without it, your pipeline reports success the instant the API accepts the object, long before any pod is running.

Readiness gates the rollout. A new pod that never becomes ready stops the rollout rather than replacing everything with something broken — but only if you have a readiness probe. Without one, a pod is "ready" as soon as its container starts, and a completely broken version rolls out to 100% happily. **This is the most important reason to have readiness probes** (Chapter 9).

Rollouts also have a deadline:

```
spec:
  progressDeadlineSeconds: 600
```

After that with no progress, the Deployment gets `Progressing=False` with reason `ProgressDeadlineExceeded`. Note what it does *not* do: **it does not roll back automatically**. The rollout simply stops, half-done, and waits for you. Automatic rollback requires a higher-level tool (Chapter 27).

Rollback

```
kubectl rollout history deployment/pingd
```

```
REVISION  CHANGE-CAUSE
1          kubectl set image deployment/pingd api=pingd:latest --record=true
2          kubectl set image deployment/pingd api=pingd:latest --record=true
```

```
kubectl rollout undo deployment/pingd
```

```
kubectl rollout undo deployment/pingd --to-revision=3
```

Measured: after an `undo`, the environment variable set in revision 2 was gone and the value from revision 1 was back:

```
PINGD_VERSION now = 1.0.0
```

Two caveats. `CHANGE-CAUSE` comes from an annotation set by the deprecated `--record` flag, and as the output above shows, it is frequently useless — both revisions have the same description because `set image` was run twice with different arguments elsewhere. Set it yourself if you want it to mean anything:

```
kubectl annotate deployment/pingd kubernetes.io/change-cause="Deploy 1.4.2 (commit a1b2c3d)"
```

And more importantly: **if you deploy from git**, `kubectl rollout undo` **puts the cluster out of sync with your manifests**. The next `apply` or GitOps sync re-applies the broken version. Rollback in git, not in the cluster — or accept that `undo` is an emergency stop that you must immediately follow with a git revert.

Pausing

```
kubectl rollout pause deployment/pingd
```

Make several changes without each triggering a rollout, then:

```
kubectl rollout resume deployment/pingd
```

Also useful mid-rollout: pause when you see errors, investigate with old and new pods both running, then resume or undo.

The selector trap

```
spec:
  selector:
    matchLabels:
      app: pingd
  template:
    metadata:
      labels:
        app: pingd      # must match the selector
```

`spec.selector` **is immutable**. Try to change it:

```
The Deployment "pingd" is invalid:
* spec.template.metadata.labels: Invalid value: {"app":"pingd"}: `selector` does not match template `labels`
* spec.selector: Invalid value: {"matchLabels":{"app":"pingd2"}}: field is immutable
```

You cannot change it, ever. The only path is delete and recreate the Deployment, which means downtime unless you do a careful parallel migration.

So **choose selector labels that will never need to change**. Specifically: do not put a version in the selector. `app: pingd` is right; `app: pingd, version: 1.4.2` guarantees you must delete the Deployment on every release.

Keep the selector minimal and stable, and put everything descriptive in the template's labels — those *can* change:

```
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: pingd      # stable forever
  template:
    metadata:
      labels:
        app.kubernetes.io/name: pingd    # required: matches selector
        app.kubernetes.io/version: "1.4.2" # free to change
```

A second trap: **two Deployments with overlapping selectors fight**. Each ReplicaSet adopts any pod matching its selector, so they will repeatedly delete each other's pods. The symptom is pods being created and destroyed continuously with no obvious cause. Selectors must be disjoint.

Scaling

```
kubectl scale deployment pingd --replicas=5
```

```
kubectl autoscale deployment pingd --min=3 --max=10 --cpu-percent=70
```

If an HPA manages a Deployment, **remove replicas from your manifest** — otherwise every `apply` resets the count the HPA just calculated, and the two fight. Chapter 25 covers this properly; it is a common and confusing production problem.

Restarting without changing anything

There is no `kubectl restart deployment`. There is:

```
kubectl rollout restart deployment/pingd
```

It works by setting an annotation (`kubectl.kubernetes.io/restartedAt`) on the pod template, which changes the template hash, which creates a new ReplicaSet, which rolls pods gracefully. Use it to pick up a changed ConfigMap or Secret (Chapter 7), or to clear a bad in-memory state.

Deleting pods to "restart" a Deployment works too, but it is abrupt and ignores your rollout strategy. Prefer `rollout restart`.

Deployment vs the alternatives

Use a **Deployment** when pods are interchangeable: stateless services, workers, anything where "replica 2" means nothing in particular.

Use a **StatefulSet** (Chapter 10) when pods need stable identity, stable storage, or ordered startup — databases, clustered systems, anything doing leader election.

Use a **DemonSet** for one pod per node: log shippers, node exporters, CNI agents.

Use a **Job** or **CronJob** for work that finishes.

The mistake worth avoiding is a StatefulSet used for a stateless service because it "sounds more serious". It is slower to roll out, harder to scale, and gives you ordering guarantees you do not need.

Try it

Watch a rollout create a second ReplicaSet:

```
kubectl get rs -l app=pingd
```

```
kubectl set env deployment/pingd PINGD_VERSION=2.0.0 && kubectl rollout status deployment/pingd
```

```
kubectl get rs -l app=pingd -o custom-columns='NAME:.metadata.name,DESIRED:.spec.replicas,REVISION:.metadata.annotations.deployment\.kubernetes\.io/revision'
```

Two ReplicaSets, the old one scaled to 0 and retained. Roll back and confirm:

```
kubectl rollout undo deployment/pingd && kubectl rollout status deployment/pingd
```

```
kubectl get deployment pingd -o jsonpath='{.spec.template.spec.containers[0].env[0].value}'
```

Prove the selector is immutable:

```
kubectl patch deployment pingd --type=json -p='[{"op":"replace","path":"/spec/selector/matchLabels/app","value":"pingd2"}]'
```

```
* spec.selector: Invalid value: {"matchLabels":{"app":"pingd2"}}: field is immutable
```

Watch capacity during a rollout with `maxUnavailable: 0` — ready count never drops below 3:

```
kubectl rollout restart deployment/pingd & for i in $(seq 1 12); do kubectl get deployment pingd -o jsonpath='ready={.status.readyReplicas} updated={.status.updatedReplicas} total={.status.replicas}'; sleep 2; done
```

Now see the failure mode that readiness probes prevent. Deploy an image that does not exist:

```
kubectl set image deployment/pingd api=pingd:does-not-exist
```

```
sleep 15 && kubectl get pods -l app=pingd && kubectl rollout status deployment/pingd --timeout=20s
```

The old pods are still serving; the new one is stuck `ImagePullBackOff`; `rollout status` times out non-zero. Nothing broke, because the rollout refused to proceed. Recover:

```
kubectl rollout undo deployment/pingd && kubectl rollout status deployment/pingd
```

Takeaways

- Deployment → ReplicaSet → Pods. The indirection exists so old ReplicaSets can be retained at zero replicas, making rollback instant.
- The default strategy is 25%/25%, which **goes below capacity during rollouts**. Use `maxSurge: 1`, `maxUnavailable: 0` in production.
- **Readiness probes gate rollouts**. Without one, a broken version rolls out completely.
- `progressDeadlineSeconds` stops a stalled rollout but does **not** roll back automatically.
- `spec.selector` **is immutable**. Never put a version in it. Overlapping selectors between Deployments cause pods to be destroyed continuously.
- `kubectl rollout undo` desynchronises the cluster from git — treat it as an emergency stop, then fix git.
- Remove `replicas` from manifests when an HPA owns it.
- `kubectl rollout restart` is the graceful way to restart everything.

Previous: Chapter 5 — Pods · Next: Chapter 7 — Configuration and secrets

Chapter 7 — Configuration and secrets

Build once, configure per environment. That principle carries over from containers unchanged; what changes is that Kubernetes gives you first-class objects for it, with behaviours that are not obvious and cause real outages.

ConfigMaps

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: pingd-config
data:
  LOG_LEVEL: "INFO"
  PINGD_GREETING: "pong"
  app.yaml: |
    feature_flags:
      new_router: false
    timeouts:
      upstream_seconds: 5
```

Two shapes in one object: simple key-value pairs, and whole files as multi-line values. Which you use determines how you consume it.

Consuming as environment variables

```
containers:
- name: api
  envFrom:
  - configMapRef:
    name: pingd-config
  env:
  - name: LOG_LEVEL
    valueFrom:
      configMapKeyRef:
        name: pingd-config
        key: LOG_LEVEL
```

`envFrom` injects every key; `configMapKeyRef` picks one and can rename it. Note that `envFrom` will silently skip keys that are not valid environment variable names — so `app.yaml` above does not become an env var, and nothing tells you.

Consuming as files

```

    volumeMounts:
      - name: config
        mountPath: /etc/pingd
        readOnly: true
  volumes:
    - name: config
      configMap:
        name: pingd-config
        items:
          - key: app.yaml
            path: app.yaml

```

Without `items`, every key becomes a file in that directory. With `items`, you choose.

Mounting a ConfigMap over a directory hides the image's contents at that path, exactly like a bind mount in Docker. To add a file to an existing directory, use `subPath` — but read the next section first, because `subPath` has a serious catch.

The update behaviour that catches everyone

This is the most important thing in the chapter.

Environment variables are frozen when the container starts. Volume-mounted files update in place.

Measured. A pod reading the same ConfigMap key both ways, before and after the ConfigMap was changed:

```

before change: env=hello file=hello
after change:  env=hello file=CHANGED

```

The file updated; the environment variable did not, and never will for the life of that container. The kubelet refreshes mounted ConfigMap and Secret volumes periodically (the sync period plus cache TTL — roughly a minute in practice; the change above appeared within 75 seconds).

The consequences:

- **Changing a ConfigMap does not restart anything.** No rollout, no notification. If your app reads config via env vars, the change has no effect until the pods restart for some unrelated reason — possibly days later, possibly during an incident, producing a version of your service nobody deployed.
- `subPath` **mounts do not update at all.** They are resolved once. This is the catch: `subPath` is the natural way to place a single config file into an existing directory, and it silently opts you out of updates.

Three ways to handle it:

1. Restart deliberately after a change.

```
kubectl rollout restart deployment/pingd
```

2. Make the change force a rollout, by putting a hash of the config into the pod template annotation. Helm does this idiomatically:


```
spec:
  template:
    metadata:
      annotations:
        checksum/config: {{
include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
```

Changing the ConfigMap changes the annotation, which changes the template hash, which triggers a normal rolling update. This is the best answer for most cases.

3. Use immutable ConfigMaps and version their names — `pingd-config-v2` — so a config change is necessarily a Deployment change:

```
immutable: true
```

Immutable ConfigMaps also reduce API server load significantly at scale, because the kubelet stops watching them.

Secrets

Structurally almost identical to ConfigMaps, with base64-encoded values:

```
apiVersion: v1
kind: Secret
metadata:
  name: pingd-secret
type: Opaque
stringData:
  DATABASE_PASSWORD: "not-a-real-password"
```

`stringData` takes plaintext and encodes it for you on write; `data` requires you to encode it yourself. Always use `stringData` in manifests you write by hand.

Secrets are not secret

The single most important fact, and it surprises people who assume the name means something:

```
kubectl get secret demo-secret -o jsonpath='{.data.DB_PASSWORD}'
```

```
aHVudGVyMg==
```

```
kubectl get secret demo-secret -o jsonpath='{.data.DB_PASSWORD}' | base64 -d
```

```
hunter2
```

Base64 is an encoding, not encryption. Anyone with `get secrets` in the namespace reads every secret in it. And by default, etcd stores them the same way — unencrypted at rest — so anyone with an etcd backup has them all.

What actually protects Secrets:

Encryption at rest. Configure `EncryptionConfiguration` on the API server so Secrets are encrypted in etcd, ideally with a KMS provider so the key is not on the node. Managed clusters usually offer this as a checkbox; verify it is on rather than assuming.

RBAC. Very few subjects should have `get/list` on Secrets. Note that `list` on Secrets in a namespace is equivalent to reading all of them.

An external secret store. Vault, AWS Secrets Manager, GCP Secret Manager. Either the External Secrets Operator syncs into Kubernetes Secrets, or the Secrets Store CSI driver mounts them directly without creating a Secret object at all. The latter is stronger.

Do not commit Secrets to git. For GitOps, use Sealed Secrets (encrypted with a cluster-held key, safe to commit) or SOPS, or keep secrets entirely outside git with the External Secrets Operator.

Prefer files over environment variables

The Docker book's argument applies here with more force, because Kubernetes makes the leakage easier:

```
kubectll describe pod POD          # env var *names* and configMapRef sources
kubectll get pod POD -o yaml       # inline env values, in full
```

Environment variables are also inherited by child processes and captured by crash handlers. Mounted secret files are read only by code that opens them, and can be `defaultMode: 0400`.

```
volumes:
- name: secret
  secret:
    secretName: pingd-secret
    defaultMode: 0400
```

Secret volumes are mounted as **tmpfs**, so they never touch the node's disk.

The downward API

Exposes pod metadata to the container — useful for logging, metrics labelling and sharding:

```
env:
- name: POD_NAME
  valueFrom:
    fieldRef:
      fieldPath: metadata.name
- name: NODE_NAME
  valueFrom:
    fieldRef:
      fieldPath: spec.nodeName
- name: POD_IP
  valueFrom:
    fieldRef:
      fieldPath: status.podIP
- name: MEMORY_LIMIT
  valueFrom:
    resourceFieldRef:
      containerName: api
      resource: limits.memory
```

That last one is genuinely valuable: it lets a runtime size itself from its actual limit rather than from the node's total memory. It is the Kubernetes-native fix for the problem Chapter 8 describes.

Confirm what a pod received:

```
kubectl exec deploy/pingd -- env | grep -E 'POD_|NODE_'
```

Ordering and precedence

Later `env` entries win over `envFrom`, so you can set defaults in bulk and override individually:

```
envFrom:
- configMapRef:
    name: pingd-config      # LOG_LEVEL=INFO
env:
- name: LOG_LEVEL
  value: "DEBUG"           # wins
```

Variables can reference earlier ones with `$(VAR)`, but only ones defined earlier in the same list — not shell expansion, and not arbitrary ordering:

```
env:
- name: HOST
  value: "db"
- name: DATABASE_URL
  value: "postgres://$(HOST):5432/pingd"
```

Missing references

A missing ConfigMap or Secret key does not fail at apply time. The Deployment is accepted, the pod is created, and it sits in `CreateContainerConfigError`:

```
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
pingd-xxxxx	0/1	CreateContainerConfigError	0	10s

```
kubectl describe pod POD | grep -A3 Events
```

```
Error: configmap "pingd-config" not found
```

Chapter 17 covers this state. Make it optional if it genuinely is:

```
envFrom:
- configMapRef:
    name: optional-config
    optional: true
```

Try it

Prove Secrets are just encoded:

```
kubectl create secret generic demo-secret --from-literal=DB_PASSWORD='hunter2'
```

```
kubectl get secret demo-secret -o jsonpath='{.data.DB_PASSWORD}' | base64 -d; echo
```

Now the update-behaviour experiment, which is the one worth running yourself. Create a ConfigMap and a pod that reads the same key both ways:

```
kubectl create configmap demo-cm --from-literal=GREETING=hello
```

```
kubectl apply -f examples/manifests/07-configmap-update-demo.yaml
```

```
kubectl wait --for=condition=Ready pod/cmtest --timeout=90s && kubectl logs cmtest --tail=1
```

```
env=hello file=hello
```

Change the ConfigMap:

```
kubectl create configmap demo-cm --from-literal=GREETING=CHANGED --dry-run=client -o yaml | kubectl apply -f -
```

Wait about 75 seconds for the kubelet to resync the volume, then look again:

```
sleep 75 && kubectl logs cmtest --tail=1
```

```
env=hello file=CHANGED
```

The file changed. The environment variable did not, and will not until the container restarts. Clean up:

```
kubectl delete pod cmtest --force --grace-period=0 2>/dev/null; kubectl delete cm demo-cm secret demo-secret
```

Finally, see the downward API in the running example:

```
kubectl exec deploy/pingd -- env | grep -E 'POD_NAME|NODE_NAME'
```

Takeaways

- ConfigMaps and Secrets are consumed as env vars or as files, and the choice determines update behaviour.
- **Env vars freeze at container start; volume-mounted files update in place** (~1 minute). Measured: `env=hello file=CHANGED` after a config change.
- subPath **mounts never update**. It is the natural way to place one file, and it silently opts out of refresh.
- Changing a ConfigMap restarts nothing. Use a config checksum annotation to force a rollout, or version the ConfigMap name and mark it immutable.
- **Secrets are base64, not encrypted**, and unencrypted in etcd by default. Enable encryption at rest, restrict RBAC (`list` on Secrets reads them all), and prefer an external store.
- Prefer secret **files** (tmpfs, mode 0400) over env vars.
- The downward API can pass the container's real memory limit into the runtime — the fix for Chapter 8's sizing problem.
- Missing ConfigMap/Secret references fail at container creation, not at apply.

Previous: Chapter 6 — Deployments and ReplicaSets · Next: Chapter 8 — Resources, requests and QoS

Chapter 8 — Resources, requests and QoS

Two numbers per container decide where it runs, how it behaves under pressure, and whether it survives a node running short of memory. They are the most consequential fields in a pod spec and the most commonly left out.

```
resources:
  requests:
    cpu: 50m
    memory: 96Mi
  limits:
    cpu: 500m
    memory: 192Mi
```

Requests and limits are different things

Requests are for the scheduler. "This container needs at least this much." The scheduler sums the requests of all pods on a node and will not place a pod whose requests do not fit in what remains. Requests are also the basis of QoS and eviction ordering.

Limits are for the kernel. "This container may not exceed this." Enforced by cgroups on the node, exactly as in the Docker book.

The critical consequence, which causes more confusion than anything else in this chapter:

Scheduling is based on requests, never on actual usage.

```
kubectl describe node k8sbook-worker | sed -n '/Allocated resources/,/Events/p'
```

```
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
Resource       Requests      Limits
-----
cpu            200m (0%)    700m (3%)
memory        178Mi (0%)   320Mi (1%)
```

Those percentages are of *allocatable*, and they are what the scheduler sees. A node whose pods have requested everything will refuse new pods even if it is completely idle. A node running at 95% CPU will happily accept more pods if those pods requested little. Neither is a bug; both surprise people.

Note the parenthetical the node itself prints: limits may exceed 100%. The cluster is deliberately overcommitted, which is the whole point of separating the two numbers.

CPU

CPU is measured in cores; **m** means millicores, so **500m** is half a core. It is a **compressible** resource: exceeding the limit does not kill you, it throttles you.

The mechanism is exactly what the Docker book measured — a cgroup bandwidth quota, not a slower core. `limits.cpu: 500m` becomes a quota of 50 ms per 100 ms period. A multi-threaded process burns that in a

few milliseconds and then freezes for the rest of the period, producing latency cliffs while average CPU utilisation looks modest.

`requests.cpu` translates into a **CPU share weight**, which only matters under contention. Under contention, a pod requesting 500m gets roughly five times the CPU of one requesting 100m.

The recurring argument about whether to set CPU limits at all is worth stating fairly:

For CPU limits: predictable behaviour, no noisy neighbours, and workloads cannot come to depend on burst capacity that will not exist when the node is busy.

Against CPU limits: throttling causes latency even when the node is idle, which is pure waste. Requests alone already provide fair sharing under contention.

The defensible position for most latency-sensitive services is **always set requests; set CPU limits only where you need hard isolation or predictable capacity planning**. For batch work, limits are fine. For the SIP load-generation case, CPU limits are actively harmful — RTP pacing is destroyed by 90 ms throttle stalls, and `nr_throttled` is the first thing to check when jitter measurements look wrong.

Always set **memory** limits, though. Memory is not compressible.

Memory

Memory is **incompressible**: you cannot throttle a process's memory, you can only kill it. Exceed `limits.memory` and the kernel OOM-kills the container.

What an OOM kill actually looks like

This is where measurement contradicted the standard advice, so it needs care.

A container with `limits.memory: 128Mi` that allocates until it dies:

```
kubectl get pod oom3 -o jsonpath='{.status.containerStatuses[0].lastState}'
```

```
{"terminated":{"exitCode":
137,"reason":"Error","startedAt":"...","finishedAt":"..."}}
```

`reason: Error`, **not** `reason: OOMKilled`. And yet the node's kernel log is unambiguous:

```
docker exec k8sbook-worker dmesg -T | grep -i 'killed process'
```

```
[Mon Aug 31 12:27:39 2026] Memory cgroup out of memory: Killed process 160986
(python)
total-vm:157584kB, anon-rss:130100kB, ... oom_score_adj:996
```

It was definitively OOM-killed by the memory cgroup, and the pod status did not say so. Repeated across restarts, the result was the same every time.

This appears to be the cgroup v2 `memory.oom.group` path: the whole container cgroup is killed at once, and on this stack (Kubernetes v1.37, containerd 2.3.4, cgroup v2) it surfaces as a generic `Error`. `OOMKilled` **does**

still appear in many environments — plenty of clusters report it correctly — so treat this as "the absence of `OOMKilled` does not rule out an OOM kill", not as "the field is gone".

The reliable diagnosis, in order:

1. **Exit code 137**. Always present for a SIGKILL. This is the primary signal.
2. **Memory usage sitting at the limit** before death (`kubectl top pod`, or metrics).
3. **Node kernel logs** — `Memory cgroup out of memory` names the cgroup and the process.
4. `reason: OOMKilled` when your stack reports it, which is a confirmation rather than a requirement.

Chapter 20 builds the full procedure. The practical takeaway for now: **do not conclude "not an OOM" just because the reason says Error**.

Sizing memory

Set `requests.memory` to a realistic steady-state figure and `limits.memory` with headroom for peaks. Runtimes that size themselves from the host rather than the cgroup need telling — the downward API (Chapter 7) is the clean way:

```
env:
  - name: MEMORY_LIMIT
    valueFrom:
      resourceFieldRef:
        containerName: api
        resource: limits.memory
```

Modern JVMs read cgroup limits themselves; use `-XX:MaxRAMPercentage=75` rather than a fixed `-Xmx` so it tracks the limit. Node needs `--max-old-space-size` set explicitly. Python has no heap limit and will allocate until killed.

QoS classes

Kubernetes derives a QoS class from your requests and limits. You do not set it; it is computed. Measured:

```
kubectl get pods -o custom-columns='NAME:.metadata.name,QOS:.status.qosClass'
```

```
qos-guaranteed    Guaranteed
qos-burstable     Burstable
qos-besteffort    BestEffort
```

Class	Condition	Eviction order
Guaranteed	Every container has requests equal to limits, for both CPU and memory	Last
Burstable	At least one request set, but not equal to limits	Middle
BestEffort	No requests or limits at all	First

When a node runs short of memory, the kubelet evicts BestEffort pods first, then Burstable pods exceeding their requests, and Guaranteed pods last. Chapter 20 covers eviction in detail.

BestEffort is the class you get by forgetting. A pod with no resources block is first against the wall on a busy node, and it is also invisible to the scheduler's accounting, which means it contributes to overcommit without being counted. Set requests on everything.

Guaranteed additionally unlocks exclusive CPU pinning via the static CPU manager policy, which matters for latency-sensitive workloads — the Kubernetes equivalent of the Docker book's `--cpuset-cpus` advice.

Other resources

Ephemeral storage — the container's writable layer, `emptyDir` volumes and logs. Worth limiting, because a pod that fills the node's disk causes `DiskPressure` and evicts *everything else*:

```
resources:
  requests: { ephemeral-storage: 1Gi }
  limits:   { ephemeral-storage: 2Gi }
```

Extended resources — GPUs and similar, advertised by device plugins. Only `limits` is meaningful; requests are set equal automatically, and they are not overcommittable:

```
resources:
  limits:
    nvidia.com/gpu: 1
```

Defaults and governance

Two namespace-scoped objects make good behaviour the default (Chapter 28):

LimitRange supplies defaults to pods that specify nothing, and can enforce minimums and maximums:

```
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
spec:
  limits:
    - type: Container
      default:      { cpu: 500m, memory: 256Mi }
      defaultRequest: { cpu: 50m,  memory: 64Mi }
      max:          { cpu: "4",   memory: 4Gi }
```

This is the cheapest way to eliminate BestEffort pods across a namespace.

ResourceQuota caps total consumption per namespace, and can require that requests and limits are set at all:

```

apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-quota
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    limits.cpu: "20"
    limits.memory: 40Gi
    pods: "50"

```

Note the interaction that catches people: **once a ResourceQuota on requests.cpu exists in a namespace, every pod in it must specify that request** or creation is rejected. Combine with a LimitRange so existing manifests keep working.

Sizing in practice

Measure rather than guess:

```
kubectl top pods --containers
```

```
kubectl top nodes
```

(`kubectl top` needs metrics-server; Chapter 26 covers installing it.)

Reasonable starting points: set `requests` at roughly the p50–p75 of observed usage, and `limits.memory` at the p99 plus headroom. Then look at whether you are throttling (`container_cpu_cfs_throttled_periods_total`) and adjust.

The **Vertical Pod Autoscaler** in recommendation mode does this for you without changing anything, and is worth running purely as an advisor (Chapter 25).

Try it

Create one pod of each QoS class and see the classification:

```
kubectl apply -f examples/manifests/08-qos.yaml
```

```

kubectl get pods -l demo=qos -o custom-
columns='NAME:.metadata.name,QOS:.status.qosClass'

```

Now reproduce the OOM finding. A container limited to 128Mi that allocates until killed:

```
kubectl apply -f examples/manifests/08-oom.yaml
```

```

sleep 40 && kubectl get pod oom3 -o
jsonpath='{.status.containerStatuses[0].lastState}{"\n"}'

```

```
{"terminated":{"exitCode":137,"reason":"Error",...}}
```

Exit 137, reason `Error`. Now confirm from the node that it really was the OOM killer:

```
docker exec k8sbook-worker dmesg -T | grep -i 'memory cgroup out of memory' | tail -2
```

If your cluster reports `reason: OOMKilled` instead, good — but note that exit 137 was correct either way.

See that scheduling uses requests, not usage:

```
kubectl describe node k8sbook-worker | sed -n '/Allocated resources/,/Events/p'
```

Compare with actual usage, which is a completely different number:

```
kubectl top node k8sbook-worker 2>/dev/null || echo "(needs metrics-server – Chapter 26)"
```

Clean up:

```
kubectl delete -f examples/manifests/08-qos.yaml -f examples/manifests/08-oom.yaml --force --grace-period=0
```

Takeaways

- **Requests are for the scheduler; limits are for the kernel.** Scheduling never considers actual usage.
- CPU is compressible (throttled); memory is not (killed). Always set memory limits; set CPU limits deliberately, and avoid them for latency-sensitive work.
- CPU limits are a bandwidth quota — multi-threaded workloads freeze for the rest of each period, causing latency cliffs at low average utilisation.
- **Measured on v1.37/containerd 2.3.4/cgroup v2, an OOM kill reported `reason: Error`, not `OOMKilled`.** Exit code 137 plus the node's `Memory cgroup out of memory` log are the reliable signals. Do not rule out OOM because the reason field disagrees.
- QoS is derived: `requests==limits` → Guaranteed; some set → Burstable; **nothing set** → **BestEffort**, **evicted first**. Forgetting resources puts you in the worst class.
- Guaranteed unlocks exclusive CPU pinning for latency-sensitive workloads.
- Limit ephemeral storage, or one pod can evict everything on the node.
- LimitRange supplies defaults; ResourceQuota caps a namespace and *forces* requests to be specified once it exists.

Previous: Chapter 7 — Configuration and secrets · Next: Chapter 9 — Health and lifecycle

Chapter 9 — Health and lifecycle

Kubernetes decides two things about your container continuously: should it be restarted, and should it receive traffic. Those are different questions with different answers, and conflating them causes cascading outages.

Three probes

Probe	Question	On failure	Runs
startupProbe	Has it finished booting?	Restart the container	Until first success, then never again
livenessProbe	Is it wedged?	Restart the container	Forever, after startup succeeds
readinessProbe	Can it serve right now?	Marked not-ready in EndpointSlices	Forever

A precision worth having early: a not-ready pod is not *removed* from its EndpointSlice. It stays listed with `conditions.ready: false`, and kube-proxy declines to route to it. Measured during a stalled rollout with three healthy pods and one unready one:

```
true true true false
```

This matters for debugging (Chapter 19): an EndpointSlice containing your pod does not mean your pod is receiving traffic. Read the conditions, not just the addresses.

All three are executed by the **kubelet on the node**, not by the control plane. Probe traffic never crosses the network between nodes, and probes keep working during an API server outage.

```
startupProbe:
  httpGet: { path: /healthz, port: http }
  failureThreshold: 30
  periodSeconds: 1           # allows up to 30s to boot
livenessProbe:
  httpGet: { path: /healthz, port: http }
  periodSeconds: 10
  failureThreshold: 3        # ~30s of failure before restart
readinessProbe:
  httpGet: { path: /healthz, port: http }
  periodSeconds: 5
  failureThreshold: 2        # ~10s before traffic stops
```

Liveness and readiness must differ

The single most consequential design decision in this chapter.

Liveness must not check dependencies. If your liveness probe queries the database, then a database blip fails liveness on every replica of every service simultaneously, and Kubernetes restarts your entire fleet — adding a thundering herd of reconnections to a database that was about to recover. You have converted a brief dependency wobble into a full outage, and the restarts make recovery slower.

Liveness answers one question: *is this process stuck in a way only a restart fixes?* For most services the honest answer is "that essentially never happens", and the correct liveness probe is a trivial endpoint that returns 200 if the event loop is turning.

Readiness is where dependencies belong. If the database is unreachable, this replica cannot serve, so remove it from the Service — but do not restart it, because restarting will not bring the database back.

The `pingd` example keeps them separate: `/healthz` checks nothing but itself, `/readyz` checks the database.

A caveat even for readiness: if *every* replica's readiness depends on a shared dependency, a dependency outage empties the Service entirely and you serve nothing rather than degraded responses. Consider whether serving errors is better than serving nothing.

Startup probes exist for slow starters

Before `startupProbe`, a slow-booting application needed a long `initialDelaySeconds` on its liveness probe — which then delayed detection of real hangs for the whole life of the container.

`startupProbe` decouples them: be generous during boot, aggressive afterwards. The Deployment in this book allows `30 × 1s` for startup, then checks liveness every 10 s.

While a startup probe is running, liveness and readiness are suppressed. This is why the benign event from Chapter 2 appears on every normal start:

```
Unhealthy   Startup probe failed: dial tcp 10.244.2.7:8000: connect: connection
refused
```

The container process exists before the server is listening. The probe retries and succeeds. **A handful of startup-probe failures during boot is normal**; a `failureThreshold` exhausted is not.

Probe types

```
httpGet: { path: /healthz, port: http, httpHeaders: [{name: Host, value:
example.com}] }
tcpSocket: { port: 5432 }
exec:      { command: ["sh", "-c", "pg_isready -U pingd"] }
grpc:      { port: 9000 }
```

Prefer `httpGet`. `exec` probes fork a process on every check — at 3 replicas × every 10 s that is trivial, at 3000 replicas it is a measurable load on your nodes, and `exec` probes have been the cause of real cluster-wide CPU problems. `tcpSocket` only proves something is listening, which is weak evidence.

Use a **named port** (`port: http`) rather than a number, so changing the container port does not silently break the probe.

Configuration mistakes that cause outages

Timeout too short. `timeoutSeconds` defaults to **1 second**. A service that occasionally takes 1.2 s to answer its health endpoint under load will be restarted, under load, making the load worse. This is a classic self-inflicted cascading failure.

Period plus threshold too aggressive. `periodSeconds: 5`, `failureThreshold: 1` restarts on a single blip. Give real applications room: `failureThreshold: 3` at minimum.

A liveness endpoint that does real work. If `/healthz` queries the database, renders a template, or acquires a contended lock, it will fail under exactly the load conditions where restarting is worst.

Probes with no resource headroom. A CPU-throttled container (Chapter 8) fails its own liveness probe because it cannot get scheduled to answer it, gets restarted, and starts the cycle again.

Graceful shutdown, and the race nobody expects

When a pod is deleted, two things happen **in parallel**:

1. The kubelet sends **SIGTERM** to the containers and starts the `terminationGracePeriodSeconds` clock.
2. The endpoint controller removes the pod from `EndpointSlices`, and then every kube-proxy on every node must update its rules.

Step 2 is **eventually consistent and not instant**. For a short window — typically tens to hundreds of milliseconds, longer on large clusters — the pod is shutting down while nodes are still routing new connections to it.

The result is dropped requests during every deployment, and it is the most common cause of "we get 502s during rollouts" when everything else looks correct.

The fix is to make the pod keep serving for a moment after receiving SIGTERM:

```
lifecycle:
  preStop:
    exec:
      command: ["sh", "-c", "sleep 5"]
```

The `preStop` hook runs **before** SIGTERM is sent, and the container keeps serving during it. Five seconds is enough for endpoint propagation nearly everywhere. It feels wrong — deliberately sleeping — and it is the standard, correct fix.

Better still, if your application can: fail readiness immediately on SIGTERM, keep serving in-flight and new requests for a few seconds, then exit. That is what the `sleep` approximates without application changes.

The full sequence:

```
pod deleted
├─ preStop hook runs (container still serving)
├─ endpoint removal propagates to kube-proxy
├─ SIGTERM to PID 1
├─ application drains in-flight work
├─ application exits → done
└─ ...or grace period expires → SIGKILL (exit 137)
```

└─ overlap here is the point

```
spec:
  terminationGracePeriodSeconds: 30
```

Set this longer than your longest reasonable request. If your pods always take exactly the full grace period to terminate, they are being SIGKILLed — which means PID 1 is not handling SIGTERM, and everything the Docker book says about exec-form CMD and signal handlers applies unchanged.

Restart backoff

A container that keeps failing is restarted with exponential backoff: 10 s, 20 s, 40 s, capped at 5 minutes. During the wait the pod shows `CrashLoopBackOff` — which is a *timer*, not an error condition (Chapter 17).

The backoff resets after the container has run successfully for 10 minutes.

Pod readiness gates

Occasionally you need readiness to depend on something outside the container — a cloud load balancer having actually registered the pod, for instance:

```
spec:
  readinessGates:
    - conditionType: "example.com/lb-registered"
```

The pod is not `Ready` until some controller sets that condition to `True`. Cloud load balancer controllers use this to prevent traffic being sent before the external LB knows about the pod — a real source of deployment errors on AWS in particular.

Try it

Look at the probe configuration on the running example:

```
kubectl get deployment pingd -o
jsonpath='{.spec.template.spec.containers[0].startupProbe.failureThreshold}
x{.spec.template.spec.containers[0].startupProbe.periodSeconds}s
liveness={.spec.template.spec.containers[0].livenessProbe.periodSeconds}s
readiness={.spec.template.spec.containers[0].readinessProbe.periodSeconds}s{"\n"}'
```

```
startup=30x1s liveness=10s readiness=5s
```

Confirm all three replicas are in the Service:

```
kubectl get endpointslice -l kubernetes.io/service-name=pingd -o
jsonpath='{.items[0].endpoints[*].conditions.ready}{"\n"}'
```

```
true true true
```

Now point readiness at a path that does not exist, and watch what a broken rollout looks like when readiness is doing its job:

```
kubectl patch deployment pingd --type=json -p='[{"op":"replace","path":"/spec/
template/spec/containers/0/readinessProbe/httpGet/path","value":"/nope"}]'
```

```
sleep 25 && kubectl get pods -l app=pingd
```

```
pingd-5975cc6496-5sxvh    1/1    Running    0    9m6s
pingd-5975cc6496-6hzpx    1/1    Running    0    9m11s
pingd-5975cc6496-9bsdg    1/1    Running    0    9m2s
pingd-68894bbf58-f68kq    0/1    Running    0    25s    ← new pod, never ready
```

The new pod is Running but 0/1. Because `maxUnavailable: 0`, the three old pods are untouched and still serving — the rollout **stalled instead of taking the service down**. Look at the endpoint conditions:

```
kubectl get endpointslice -l kubernetes.io/service-name=pingd -o
jsonpath='{.items[0].endpoints[*].conditions.ready}'{"\n"}
```

```
true true true false
```

Four endpoints, three routable. The broken pod is listed and excluded. Undo:

```
kubectl rollout undo deployment/pingd && kubectl rollout status deployment/pingd
```

Finally, watch a graceful termination and time it:

```
POD=$(kubectl get pod -l app=pingd -o jsonpath='{.items[0].metadata.name}') &&
time kubectl delete pod $POD
```

Well under the 30 s grace period, because `pingd` handles SIGTERM. A pod that always takes exactly `terminationGracePeriodSeconds` is being killed, not stopping.

Takeaways

- Three probes, three jobs. The kubelet runs them locally.
- **Liveness must not check dependencies.** A dependency-checking liveness probe turns a database blip into a fleet-wide restart storm.
- Readiness is where dependencies belong — but if every replica depends on one thing, an outage empties the Service entirely.
- `startupProbe` lets you be generous at boot and aggressive afterwards. A few startup-probe failures during boot are normal.
- `timeoutSeconds` **defaults to 1 second**. This restarts healthy-but-slow services under load.
- Prefer `httpGet` with a **named port**; `exec` probes fork a process every period.
- Endpoint removal and SIGTERM happen **in parallel**, so a terminating pod still receives new connections briefly. A `preStop` sleep of ~5 s is the standard fix for rollout 502s.
- Pods always taking the full grace period means PID 1 is not handling SIGTERM.
- `CrashLoopBackOff` is a backoff timer, not a state.

Chapter 10 — The other workload kinds

Deployments cover stateless services, which is most of what most clusters run. Four other controllers cover the rest, and choosing the wrong one produces problems that are hard to back out of later.

Job

Runs pods until a specified number complete successfully, then stops.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: migrate
spec:
  backoffLimit: 4
  completions: 1
  template:
    spec:
      restartPolicy: OnFailure      # or Never. Never `Always`.
      containers:
      - name: migrate
        image: pingd:latest
        command: ["alembic", "upgrade", "head"]
```

Measured on a trivial job:

```
kubectl create job demo-job --image=busybox:1.37 -- sh -c 'echo working; sleep 3;
echo done'
```

```
kubectl get job demo-job -o custom-
columns='NAME:.metadata.name,COMPLETIONS:.status.succeeded,STATUS:.status.condition
s[0].type'
```

NAME	COMPLETIONS	STATUS
demo-job	1	SuccessCriteriaMet

```
kubectl get pod -l job-name=demo-job
```

NAME	STATUS	RESTART POLICY
demo-job-lfw4m	Succeeded	Never

Note that **the completed pod is retained**. That is deliberate — its logs are the record of what happened — and it is also how clusters accumulate thousands of `Succeeded` pods. Clean up automatically:

```
spec:
  ttlSecondsAfterFinished: 3600
```

Set this on every Job. Without it, the pods stay until something deletes them.

The fields that matter:

- `backoffLimit` (default 6) — how many pod failures before the Job is marked `Failed`. Failures back off exponentially, capped at 6 minutes.
- `activeDeadlineSeconds` — a wall-clock cap on the whole Job. This overrides `backoffLimit`; when it expires the Job fails and running pods are killed. Essential for anything that could hang.
- `completions` **and** `parallelism` — for parallel work. `completions: 10, parallelism: 3` runs ten tasks, three at a time.
- `completionMode: Indexed` — each pod gets `JOB_COMPLETION_INDEX` in its environment, so workers can shard deterministically. This is what makes Jobs usable for partitioned batch work, and it is underused.

`restartPolicy` must be `OnFailure` or `Never` — `Always` is rejected, because a Job that always restarts can never complete. The distinction: `OnFailure` restarts the container in place (same pod, keeps the node and any local scratch); `Never` creates a new pod (fresh everything, and you keep the failed pod for inspection). For debugging, `Never` is more useful.

Sidecars used to break Jobs: a never-exiting sidecar meant the pod never completed. The native sidecar mechanism from Chapter 5 (`initContainers` with `restartPolicy: Always`) fixes this — sidecars are terminated once the main containers finish.

For the LLM evaluation runner case, the Job fields that matter are `activeDeadlineSeconds` (an API call that hangs must not hold a pod forever), `backoffLimit: 0` where retries would double-charge for inference, and `Indexed` completion mode to shard a test suite across workers deterministically.

CronJob

Creates Jobs on a schedule.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-report
spec:
  schedule: "0 3 * * *"
  timeZone: "Europe/Brussels"
  concurrencyPolicy: Forbid
  startingDeadlineSeconds: 300
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      ttlSecondsAfterFinished: 3600
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: report
              image: pingd:latest
              command: ["python", "-m", "reports.nightly"]
```

The fields that cause incidents when omitted:

`concurrencyPolicy` defaults to `Allow`. If a run takes longer than the interval, you get overlapping executions — two copies of your nightly report writing to the same table. Use `Forbid` (skip if one is running) or `Replace` (kill the old one) unless overlap is genuinely safe.

`timeZone` — without it, schedules are interpreted in the **controller manager's** time zone, usually UTC. A "3am" job running at 4am or 2am depending on daylight saving is a recurring surprise. Set it explicitly.

`startingDeadlineSeconds` — if the controller was down at the scheduled time, how late is too late to still run? Without it, a controller outage can trigger a burst of missed runs at once. Note that if more than 100 schedules are missed, the `CronJob` stops scheduling entirely and logs an error — a genuinely surprising failure mode that leaves you with a `CronJob` that silently never runs again.

History limits — defaults are 3 and 1. Reasonable, but Jobs plus their pods still accumulate; combine with `ttlSecondsAfterFinished`.

`CronJob` guarantees are **at-least-once, approximately**. A job may run twice or be skipped. Make the work idempotent; do not build anything requiring exactly-once semantics on a `CronJob` alone.

DaemonSet

One pod per node, automatically, including nodes added later.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
spec:
  selector:
    matchLabels: { app: node-exporter }
  template:
    metadata:
      labels: { app: node-exporter }
    spec:
      tolerations:
        - operator: Exists                # run on every node, including tainted ones
      containers:
        - name: node-exporter
          image: quay.io/prometheus/node-exporter:v1.8.2
```

```
kubectl get ds -n kube-system kube-proxy
```

NAME	DESIRED	READY
kube-proxy	3	3

Three nodes, three pods, maintained automatically.

Used for things that are properly per-node: log shippers, metrics exporters, CNI agents, storage drivers, security agents. Restrict to a subset with `nodeSelector` — GPU monitoring only on GPU nodes.

The **tolerations** point matters: by default a `DaemonSet` will not schedule onto tainted nodes, including control-plane nodes. Infrastructure `DaemonSets` usually want `tolerations: [{operator: Exists}]` to run

everywhere regardless of taints. Forgetting this means your monitoring silently has a hole exactly where you need it.

DaemonSets support `RollingUpdate` with `maxUnavailable`, and updating one means touching every node — do it carefully on large clusters.

StatefulSet

For workloads where pods are **not** interchangeable.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres-headless      # required: a headless Service
  replicas: 3
  selector:
    matchLabels: { app: postgres }
  template:
    metadata:
      labels: { app: postgres }
    spec:
      containers:
        - name: postgres
          image: postgres:17-alpine
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
      volumeClaimTemplates:            # one PVC per pod, created automatically
        - metadata:
            name: data
          spec:
            accessModes: ["ReadWriteOnce"]
            resources:
              requests:
                storage: 10Gi
```

What you get that a Deployment does not give you:

Stable, ordinal names. `postgres-0`, `postgres-1`, `postgres-2` — not random suffixes. A pod that is replaced comes back with the *same* name.

Stable network identity. With the required headless Service, each pod gets a predictable DNS name: `postgres-0.postgres-headless.default.svc.cluster.local`. This is what lets cluster members find each other.

Stable storage. `volumeClaimTemplates` creates one PVC per pod, and the PVC follows the pod name. `postgres-0` always gets `data-postgres-0`, even after rescheduling to another node.

Ordered operations. Pods are created 0, 1, 2 — each waiting for the previous to be Ready — and deleted in reverse. Set `podManagementPolicy: Parallel` if you do not need this and want faster scaling.

The consequences people are unprepared for:

- **PVCs are not deleted** when you delete the StatefulSet or scale it down. That is deliberate (it is your data), and it means scaling from 5 to 3 and back to 5 reattaches the original volumes. `persistentVolumeClaimRetentionPolicy` (stable since v1.32) lets you change this.
- **Rollouts are slow**, one pod at a time in reverse ordinal order, waiting for readiness at each step.
- **A stuck pod blocks everything**. If `postgres-1` will not become Ready, the rollout stops there permanently. `podManagementPolicy: Parallel` and partitioned rollouts help.

Do not use a StatefulSet for a stateless service. It is slower to roll out, slower to scale, and gives you guarantees you are paying for and not using. The only reasons are stable identity, stable per-pod storage, or ordered startup.

And the larger question: **should you run the database in Kubernetes at all?** A managed database removes an entire category of operational risk — backups, failover, upgrades, point-in-time recovery — for money. If you do run it yourself, use a mature **operator** (CloudNativePG, Zalando, Percona) rather than a hand-written StatefulSet; the operator is where the backup, failover and upgrade logic lives, and that logic is the actual product. Chapter 30 covers operators.

ReplicaSet

Mentioned for completeness. You do not create these directly — Deployments manage them (Chapter 6). The only time you interact with one is reading `kubectl get rs` output during a rollout.

Choosing

Need	Use
Stateless service, interchangeable replicas	Deployment
Work that finishes	Job
Work that finishes, on a schedule	CronJob
Exactly one pod per node	DaemonSet
Stable identity, per-pod storage, or ordered startup	StatefulSet
A database	A managed service , or a StatefulSet driven by an operator

Try it

Run a Job and see what it leaves behind:

```
kubectl create job demo-job --image=busybox:1.37 -- sh -c 'echo working; sleep 3;
echo done'
```

```
sleep 12 && kubectl get job,pods -l job-name=demo-job
```

The pod is Succeeded, not deleted — its logs are still readable:

```
kubectl logs -l job-name=demo-job
```

```
kubectl delete job demo-job
```

Deleting the Job deletes its pods, via the ownership chain from Chapter 2.

Now a Job that fails, to see `backoffLimit` at work:

```
kubectl create job failjob --image=busybox:1.37 -- sh -c 'echo attempting; exit 1'
--dry-run=client -o yaml | sed 's/^spec:/spec:\n  backoffLimit: 2/' | kubectl
apply -f -
```

```
sleep 45 && kubectl get job failjob -o custom-
columns='NAME:.metadata.name,FAILED:.status.failed,CONDITION:.status.conditions[0].
type'
```

```
kubectl delete job failjob
```

Look at a real DaemonSet and confirm it tracks node count:

```
kubectl get ds -n kube-system -o custom-
columns='NAME:.metadata.name,DESIRED:.status.desiredNumberScheduled,READY:.status.n
umberReady'
```

kube-proxy shows 3 desired on a 3-node cluster. Note which DaemonSets tolerate the control-plane taint and which do not:

```
kubectl get ds -n kube-system kube-proxy -o
jsonpath='{.spec.template.spec.tolerations[*].operator}{"\n"}'
```

Finally, see StatefulSet identity. This uses the local-path provisioner kind ships with:

```
kubectl apply -f examples/manifests/10-statefulset.yaml
```

```
kubectl rollout status statefulset/web --timeout=180s && kubectl get pods -l
app=web
```

Ordinal names, `web-0` and `web-1`, and one PVC each:

```
kubectl get pvc -l app=web
```

Delete a pod and watch it return **with the same name and the same volume**:

```
kubectl delete pod web-0 && sleep 15 && kubectl get pod web-0 && kubectl get pvc -
l app=web
```

Clean up — note the PVCs survive deliberately. **destructive** on the second command:

```
kubectl delete -f examples/manifests/10-statefulset.yaml
```

```
kubectl delete pvc -l app=web
```

Takeaways

- **Job:** set `ttlSecondsAfterFinished` on every one, or completed pods accumulate forever. `activeDeadlineSeconds` for anything that could hang. `Indexed` mode for sharded work.
- **CronJob:** `concurrencyPolicy` defaults to `Allow` (overlapping runs), `timeZone` defaults to the controller's, and missing 100 schedules stops it permanently. Make the work idempotent.
- **DaemonSet:** one per node automatically. Infrastructure DaemonSets need `tolerations: [{operator: Exists}]` or they silently skip tainted nodes.
- **StatefulSet:** stable names, stable DNS, stable per-pod PVCs, ordered operations. PVCs survive deletion on purpose; a stuck pod blocks the whole rollout.
- Do not use a StatefulSet for a stateless service.
- For databases, prefer a managed service, or an operator over a hand-rolled StatefulSet.

Previous: Chapter 9 — Health and lifecycle · Next: Chapter 11 — Services

Chapter 11 — Services

Pods are ephemeral and their IPs change constantly. A Service is a stable name and virtual IP in front of a changing set of pods, and it is the only reason anything in a cluster can reliably talk to anything else.

The selector is the whole mechanism

```
apiVersion: v1
kind: Service
metadata:
  name: pingd
spec:
  type: ClusterIP
  selector:
    app: pingd
  ports:
    - name: http
      port: 80           # the Service's port
      targetPort: http   # the pod's port (named)
```

A Service knows nothing about your Deployment. It matches labels. Whatever has those labels receives traffic — a Deployment's pods, a StatefulSet's pods, a bare pod you created by hand, or nothing at all.

That last case is the one to remember: a selector matching nothing is not an error (Chapter 4). The Service exists, has a ClusterIP, and silently discards traffic.

Use a **named** `targetPort` (`http`) rather than a number. Change the container port later and the Service follows; hard-code `8000` and you have a silent mismatch.

EndpointSlices

The Service is the abstraction; **EndpointSlices** are the actual list of backends, maintained by the endpoint controller.

```
kubectl get endpointslice -l kubernetes.io/service-name=pingd -o
jsonpath='{.items[0].endpoints[*].addresses[0]}'
```

```
10.244.2.5 10.244.1.4 10.244.2.6
```

These are pod IPs, and they are the first thing to check when a Service does not work (Chapter 19). Also check the readiness conditions, because a listed endpoint is not necessarily a routable one:

```
kubectl get endpointslice -l kubernetes.io/service-name=pingd -o
jsonpath='{.items[0].endpoints[*].conditions.ready}'
```

```
true true true
```

Not-ready pods stay listed with `ready: false` (Chapter 9). Reading only the addresses will mislead you.

EndpointSlices replaced the older `Endpoints` object, which put every backend in a single object and became a scalability problem — one pod change rewrote an object listing thousands of addresses, and every kube-proxy in the cluster received the whole thing. Slices cap at 100 endpoints each. `kubectl get endpoints` still works via a compatibility shim, but EndpointSlices are what actually drives routing.

The four types

ClusterIP (default)

A virtual IP reachable only inside the cluster.

```
clusterIP=10.96.184.46 type=ClusterIP
```

The ClusterIP is **not attached to any network interface**. Nothing listens on it. It exists only as a set of rules that kube-proxy programs on every node: packets to `10.96.184.46:80` are rewritten to one of the pod IPs. You cannot ping it meaningfully, and `tcpdump` on the ClusterIP shows nothing — a genuinely confusing property when debugging.

Load balancing is **per connection**, not per request. A long-lived HTTP/1.1 keepalive connection or an HTTP/2 connection stays pinned to one pod. This is why gRPC clients famously do not balance across a ClusterIP Service: one connection, one backend, forever. The answers are client-side load balancing with a headless Service, or a proxy/service mesh that understands HTTP/2.

NodePort

Opens the same port on **every node**, forwarding to the Service.

```
kubectl expose deployment pingd --name=pingd-np --type=NodePort --port=80 --target-port=8000
```

```
nodePort=32580
```

Reachable at `<any-node-ip>:32580`, from outside the cluster. The range is 30000–32767 by default.

NodePort is a building block rather than a destination. Its problems: unmemorable high ports, you must know node IPs, and no health-aware routing in front of it. It exists mainly so cloud load balancers have something to target, and for local clusters.

`externalTrafficPolicy` matters here:

- **Cluster** (default) — any node accepts the traffic and forwards to a pod anywhere, which costs an extra hop and **loses the client source IP** (it is SNATed).
- **Local** — only nodes running a pod accept traffic; the source IP is preserved and the extra hop is gone. But nodes with no pod blackhole the traffic, so it needs a load balancer that health-checks node ports.

"Why does my application see the node's IP instead of the client's?" is this setting.

LoadBalancer

Asks the cloud provider for an external load balancer pointing at the NodePort.

```
kubectl get svc pingd-lb
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
pingd-lb	LoadBalancer	10.96.171.20	<pending>	80:32068/TCP

<pending> **forever on this cluster**, because kind has no cloud-controller-manager (Chapter 3). That is not a bug; there is nothing to provision a load balancer. On a local cluster use `kubectl port-forward` or an Ingress, or install MetalLB if you want real LoadBalancer semantics.

The other thing to know: **one LoadBalancer Service is one cloud load balancer**, each with a monthly cost and an IP address. Twenty microservices exposed this way is twenty load balancers. This is the main practical argument for an Ingress or Gateway (Chapter 13), which puts many services behind one.

ExternalName

A CNAME, with no proxying and no endpoints:

```
spec:
  type: ExternalName
  externalName: db.example.com
```

Useful for referring to something outside the cluster by an in-cluster name, so applications can use `db` in every environment and the mapping changes per cluster.

Headless Services

`clusterIP: None`. No virtual IP, no proxying — DNS returns **the pod IPs directly**.

Measured against the two Services in this book's example:

```
nslookup pingd          # normal ClusterIP Service
nslookup pingd-headless # headless
```

```
pingd-headless.default.svc.cluster.local
Address: 10.244.1.8
Address: 10.244.2.12
Address: 10.244.2.11
```

Three A records, one per pod, instead of a single VIP.

Use a headless Service when:

- the client should do its own load balancing (gRPC, database drivers with connection pools);
- you need per-pod DNS names — a StatefulSet requires a headless Service for `web-0.web-headless.default.svc.cluster.local` (Chapter 10);
- something needs to discover all peers, like a clustered database forming a quorum.

The caveat: DNS clients cache, and many take the first record. Round-robin DNS is not load balancing (the same warning as the Docker book's Chapter 11).

Multi-port Services and ports for the SIP case

```
ports:
  - name: http
    port: 80
    targetPort: http
  - name: metrics
    port: 9090
    targetPort: metrics
```

Names are **required** once there is more than one port.

UDP is supported (`protocol: UDP`), and SCTP exists but with patchy support. Two things are worth knowing for anything media-related:

A Service cannot express a large dynamic port range. RTP typically needs thousands of UDP ports, and Services enumerate ports individually. The practical answers are `hostNetwork: true` (Chapter 5's scheduling constraints then apply, and you lose port isolation), or `hostPort` on specific ports, or a specialised ingress that understands the protocol.

kube-proxy's UDP conntrack handling has historically been a source of stale entries, where a UDP "connection" keeps being sent to a pod that no longer exists. If you see UDP traffic going nowhere after a pod restart, that is the first thing to check.

For SIP specifically, the NAT problem from the Docker book returns in a new form: pod IPs are routable within the cluster but not outside it, so SDP advertising a pod IP fails for external peers exactly as it did with Docker's bridge. `hostNetwork` plus the application's public-address override remains the working answer.

Session affinity

```
spec:
  sessionAffinity: ClientIP
  sessionAffinityConfig:
    clientIP:
      timeoutSeconds: 10800
```

Pins a client IP to one backend. Crude — it is source-IP based, so everyone behind one NAT is one client — and it defeats even load distribution. If you need sticky sessions, do it at the HTTP layer with an Ingress (cookie-based), and better still make your application stateless.

Traffic distribution

```
spec:
  trafficDistribution: PreferClose
```

A newer, simpler alternative to topology-aware hints: prefer endpoints in the same zone, falling back cluster-wide. This reduces cross-zone traffic charges, which are a real cost on cloud providers. Note it is a *preference*, not a guarantee.

Try it

Look at the Service and its backends:

```
kubectl get svc pingd -o wide
```

```
kubectl get endpointslice -l kubernetes.io/service-name=pingd -o custom-  
columns='NAME:.metadata.name,ENDPOINTS:.endpoints[*].addresses[0],READY:.endpoints[  
*].conditions.ready'
```

Reach it by name from another pod:

```
kubectl run t --rm -it --restart=Never --image=curlimages/curl:8.11.1 --quiet --  
curl -sS http://pingd/
```

Compare a ClusterIP with a headless Service — a single VIP versus three pod IPs:

```
kubectl run t --rm -it --restart=Never --image=busybox:1.37 --quiet -- sh -c  
'nslookup pingd | tail -3; echo ---; nslookup pingd-headless | tail -6'
```

Create a NodePort and reach it via a node IP:

```
kubectl expose deployment pingd --name=pingd-np --type=NodePort --port=80 --target-  
port=8000
```

```
NP=$(kubectl get svc pingd-np -o jsonpath='{.spec.ports[0].nodePort}') && NODE=$(  
(kubectl get node k8sbook-worker -o jsonpath='{.status.addresses[0].address}') && k  
ubectl run t --rm -it --restart=Never --image=curlimages/curl:8.11.1 --quiet -- cur  
l -sS "http://$NODE:$NP/"
```

Watch a LoadBalancer stay pending with no cloud controller:

```
kubectl expose deployment pingd --name=pingd-lb --type=LoadBalancer --port=80 --  
target-port=8000 && sleep 5 && kubectl get svc pingd-lb
```

```
pingd-lb    LoadBalancer    10.96.171.20    <pending>    80:32068/TCP
```

Now create the classic silent failure — a Service whose selector matches nothing:

```
kubectl create service clusterip nobody --tcp=80:8000
```

```
kubectl get svc nobody && kubectl get endpointslice -l kubernetes.io/service-  
name=nobody
```

Healthy-looking Service, `<unset>` endpoints. Clean up:

```
kubectl delete svc pingd-np pingd-lb nobody
```

Takeaways

- A Service matches **labels**, not Deployments. A selector matching nothing is silent.
- Use named `targetPort`s so a container port change cannot break routing.
- **EndpointSlices are the real backend list.** Check both addresses and `conditions.ready`.
- A ClusterIP is not on any interface — it is only kube-proxy rules, which is why you cannot tcpdump it.
- Load balancing is **per connection**. HTTP/2 and gRPC pin to one pod; use a headless Service with client-side balancing or a proxy.
- NodePort loses the client source IP unless `externalTrafficPolicy: Local`.
- One LoadBalancer Service is one cloud load balancer, with a bill. Use Ingress for many services.
- `<pending>` EXTERNAL-IP means no cloud-controller-manager — expected locally.
- Headless Services return pod IPs and are required for StatefulSet per-pod DNS.
- Services cannot express large dynamic UDP ranges; media workloads need `hostNetwork`.

Previous: Chapter 10 — The other workload kinds · Next: Chapter 12 — DNS and service discovery

Chapter 12 — DNS and service discovery

Every pod is configured to use cluster DNS, and almost every inter-service call in a cluster starts with a DNS lookup. It is also a component that fails in ways that look like anything but DNS, and whose default configuration wastes a surprising amount of traffic.

What a pod gets

```
kubectl run t --rm -it --restart=Never --image=busybox:1.37 -- cat /etc/resolv.conf
```

```
search default.svc.cluster.local svc.cluster.local cluster.local
nameserver 10.96.0.10
options ndots:5
```

Three parts, all of which matter:

- `nameserver 10.96.0.10` — the ClusterIP of the `kube-dns` Service, which fronts CoreDNS. (The Service is still called `kube-dns` for compatibility, though CoreDNS replaced `kube-dns` years ago.)
- `search ...` — domains appended to unqualified names, in order. Note the first entry is the **pod's own namespace**, which is why a bare `pingd` resolves within a namespace and not across namespaces.
- `options ndots:5` — the setting that costs you.

Names

```
<service>                # same namespace
<service>.<namespace>     # cross-namespace
<service>.<namespace>.svc  # more explicit
<service>.<namespace>.svc.cluster.local  # fully qualified
<service>.<namespace>.svc.cluster.local. # absolute – no search walk
```

For headless Services, each pod gets its own record, which is what StatefulSets depend on:

```
<pod>.<service>.<namespace>.svc.cluster.local
web-0.web-headless.default.svc.cluster.local
```

`ndots:5`, and why external lookups are slow

This is the practical problem, and it is worth understanding precisely.

`ndots:5` means: **if a name contains fewer than 5 dots, try the search domains first.**

`github.com` has one dot. So a lookup for `github.com` from a pod does this:

```
github.com.default.svc.cluster.local -> NXDOMAIN
github.com.svc.cluster.local         -> NXDOMAIN
github.com.cluster.local              -> NXDOMAIN
github.com                           -> answer
```

Measured on this cluster — the first two, confirmed NXDOMAIN:

```
github.com.default.svc.cluster.local    -> NXDOMAIN
github.com.svc.cluster.local            -> NXDOMAIN
```

Four queries to resolve one name, three of them guaranteed to fail. And because most resolvers query both A and AAAA, it is typically eight packets. Multiply by every external call your services make.

The consequences: added latency on every external request, three times the necessary load on CoreDNS, and — at scale — CoreDNS becoming a bottleneck for reasons that look like a DNS problem but are really a configuration default.

In-cluster names are unaffected: `pingd` matches on the *first* search domain, so a short in-namespace name is optimal. The waste is entirely on external names.

Three fixes:

1. Use absolute names for external hosts. A trailing dot ends the search:

```
env:
- name: API_ENDPOINT
  value: "https://api.example.com./v1"
```

Effective, and it looks like a typo to everyone who reads it. Comment it.

2. Lower ndots per pod:

```
spec:
  dnsConfig:
    options:
      - name: ndots
        value: "2"
```

With `ndots:2`, `github.com` (1 dot) is still searched, but `api.example.com` (2 dots) is tried absolute first. Going below 2 breaks short in-cluster names, so 2 is usually the floor.

3. Run NodeLocal DNSCache — a DaemonSet caching DNS on each node, which removes most of the cross-node traffic and the conntrack pressure. This is the standard answer for large clusters and worth deploying before you need it.

CoreDNS

An ordinary Deployment in `kube-system`, configured by a ConfigMap:

```
kubectl get cm -n kube-system coredns -o jsonpath='{.data.Corefile}'
```

```

.:53 {
  errors
  health { lameduck 5s }
  ready
  kubernetes cluster.local in-addr.arpa ip6.arpa {
    pods insecure
    fallthrough in-addr.arpa ip6.arpa
    ttl 30
  }
  prometheus :9153
  forward . /etc/resolv.conf { max_concurrent 1000 }
  cache 30 { disable success cluster.local
              disable denial cluster.local }
  loop
  reload
  loadbalance
}

```

Reading it: the `kubernetes` plugin answers for `cluster.local` with a **30-second TTL**; `forward` sends everything else to the node's own resolvers; `cache 30` caches for 30 seconds but explicitly **disables caching for `cluster.local`**, since the `kubernetes` plugin is authoritative and already fast.

That 30-second TTL is worth knowing: a client that caches DNS aggressively can keep using a Service's old ClusterIP or a headless Service's stale pod IPs for far longer than intended. JVMs with `networkaddress.cache.ttl=-1` are the classic offender.

Useful additions, edited into that ConfigMap:

```

# Send an internal zone to a specific upstream
corp.internal:53 {
  forward . 10.0.0.53
}

```

```

# Rewrite one name to another
rewrite name legacy-api.default.svc.cluster.local pingd.default.svc.cluster.local

```

CoreDNS reloads automatically (the `reload` plugin) within a minute or two of a ConfigMap change.

CoreDNS is a failure domain

Because it is a normal Deployment on normal nodes, it can be evicted, CPU-throttled or OOM-killed like anything else. When it degrades, **every service in the cluster experiences intermittent, latency-shaped failures**, and almost nobody's first hypothesis is DNS.

Practical protections:

- Give it resource **requests** so it is not BestEffort (Chapter 8) — this is the single most common CoreDNS mistake.
- Do not set aggressive CPU limits on it; throttled DNS is a cluster-wide latency source.
- Spread replicas across nodes with anti-affinity or topology spread.
- Scale replicas with cluster size — the rough guidance is 2 minimum, plus one per ~16 nodes, and watch the metrics rather than trusting the formula.

- Add a PodDisruptionBudget (Chapter 25) so a node drain cannot take all replicas at once.
- Alert on `coredns_dns_responses_total{rcode="SERVFAIL"}` and request duration.

Per-pod DNS configuration

```
spec:
  dnsPolicy: ClusterFirst          # default
  dnsConfig:
    nameservers: ["10.0.0.53"]
    searches: ["corp.internal"]
    options:
      - name: ndots
        value: "2"
```

`dnsPolicy` values:

- `ClusterFirst` (default) — cluster DNS, forwarding external names upstream.
- `ClusterFirstWithHostNet` — what you need with `hostNetwork: true`. **This is a real trap:** a `hostNetwork` pod with the default policy uses the *node's* resolver and cannot resolve any Service name at all. If a host-networked pod cannot find `pingd`, this is why — and it applies directly to the SIP workload from Chapter 11, which needs host networking.
- `Default` — inherit the node's resolvers, ignoring cluster DNS.
- `None` — use only what you specify in `dnsConfig`.

Debugging DNS

The sequence, in order:

```
kubectl exec -it POD -- cat /etc/resolv.conf
```

Wrong nameserver or missing search domains means a pod-level misconfiguration or a `hostNetwork` policy problem.

```
kubectl run t --rm -it --restart=Never --image=nicolaka/netshoot -- dig
pingd.default.svc.cluster.local
```

Resolving from a fresh pod but not from yours narrows it to your pod's configuration or its resolver library.

```
kubectl get svc -n kube-system kube-dns
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns --tail=50
```

```
kubectl get endpointslice -n kube-system -l kubernetes.io/service-name=kube-dns
```

No endpoints for `kube-dns` means CoreDNS pods are not ready, and *nothing in the cluster can resolve anything*.

Two special cases worth recognising:

Intermittent failures, usually ~5 seconds. Classic musl/glibc parallel A+AAAA race against conntrack, producing occasional lost responses and a 5-second resolver timeout. Alpine images are more affected. Mitigations: `single-request-reopen` in `dnsConfig`, `NodeLocal DNSCache`, or a CNI that avoids the conntrack path.

`Name` does not resolve **for a Service that exists**. Check the namespace — a bare name only resolves within the pod's own namespace, because that is the first search domain.

Try it

Look at what your pods actually get:

```
kubectl run t --rm -it --restart=Never --image=busybox:1.37 --quiet -- cat /etc/resolv.conf
```

Watch the search walk waste three queries on an external name:

```
kubectl run t --rm -it --restart=Never --image=nicolaka/netshoot --quiet -- sh -c
'for s in default.svc.cluster.local svc.cluster.local cluster.local; do printf
"github.com.%s -> " "$s"; dig +short +time=1 "github.com.$s" | head -1 | grep -q .
&& echo ANSWER || echo NXDOMAIN; done'
```

```
github.com.default.svc.cluster.local -> NXDOMAIN
github.com.svc.cluster.local         -> NXDOMAIN
github.com.cluster.local              -> NXDOMAIN
```

Then confirm the trailing dot skips it:

```
kubectl run t --rm -it --restart=Never --image=nicolaka/netshoot --quiet -- sh -c
'time dig +short github.com. | head -1'
```

Resolve a Service the four different ways:

```
kubectl run t --rm -it --restart=Never --image=nicolaka/netshoot --quiet -- sh -c
'for n in pingd pingd.default pingd.default.svc pingd.default.svc.cluster.local; do
printf "%-40s -> " "$n"; dig +short "$n"; done'
```

See per-pod DNS records from a headless Service:

```
kubectl run t --rm -it --restart=Never --image=nicolaka/netshoot --quiet -- dig
+short pingd-headless.default.svc.cluster.local
```

Check CoreDNS health, and specifically whether it has resource requests:

```
kubectl get deploy -n kube-system coredns -o
jsonpath='{.spec.template.spec.containers[0].resources}{"\n"}'
```

```
kubectl get endpointslice -n kube-system -l kubernetes.io/service-name=kube-dns -o
jsonpath='{.items[0].endpoints[*].addresses[0]}{"\n"}'
```

Takeaways

- Pods get `nameserver <kube-dns ClusterIP>`, a search list starting with **their own namespace**, and `ndots:5`.
 - `ndots:5` **makes every external lookup cost four queries, three of them NXDOMAIN**. Fix with trailing-dot absolute names, `ndots:2` via `dnsConfig`, or `NodeLocal DNSCache`.
 - A bare Service name only resolves inside the pod's namespace. Cross-namespace needs `svc.namespace`.
 - CoreDNS is a normal Deployment and therefore a cluster-wide failure domain. Give it requests, avoid tight CPU limits, spread it, add a PDB.
 - Records have a 30-second TTL; aggressively caching clients will hold stale addresses.
 - `hostNetwork: true` **needs** `dnsPolicy: ClusterFirstWithHostNet`, or Service names do not resolve at all.
 - Intermittent ~5-second DNS failures are the A/AAAA conntrack race, not your application.
-

Previous: Chapter 11 — Services · Next: Chapter 13 — Ingress and Gateway API

Chapter 13 — Ingress and Gateway API

Chapter 11 ended on a cost problem: one `LoadBalancer` Service is one cloud load balancer, with a monthly bill and an IP address each. Ingress solves that — one entry point, HTTP routing to many Services — and introduces problems of its own that the Gateway API exists to fix.

Ingress needs a controller

An Ingress object on its own does nothing. It is a *description* of desired routing; a **controller** must be running to implement it. Install ingress-nginx, Traefik, HAProxy, Contour, or your cloud's controller, or your Ingress objects sit there inert.

```
kubectl get ingressclass
```

NAME	CONTROLLER	PARAMETERS	AGE
nginx	k8s.io/ingress-nginx	<none>	22s

No IngressClass means no controller, which means nothing will happen. This is the first thing to check when an Ingress "does not work".

An Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: pingd
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
spec:
  ingressClassName: nginx
  rules:
    - host: pingd.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: pingd
                port:
                  number: 80
    - http:
        paths:
          - path: /api(/|$)(.*)
            pathType: ImplementationSpecific
            backend:
              service:
                name: pingd
                port:
                  number: 80
```

Measured against the running cluster:

```
curl -H 'Host: pingd.local' http://localhost:8180/
```

```
{"service":"pingd","version":"1.0.0","reply":"pong"}
```

```
curl http://localhost:8180/api/
```

```
{"service":"pingd","version":"1.0.0","reply":"pong"}
```

```
curl -o /dev/null -w '%{http_code}\n' -H 'Host: nope.local' http://localhost:8180/
```

```
404
```

Host-based routing, path-based routing with a rewrite, and an unmatched host falling through to the controller's default backend.

pathType matters

- **Prefix** — matches on path *segments*. `/api` matches `/api/v1` but **not** `/apifoo`. This is what you usually want.
- **Exact** — exact string match.
- **ImplementationSpecific** — whatever the controller does, which for nginx means regular expressions. Required for the capture groups in the rewrite example above.

Getting **Prefix** semantics wrong is a common source of "why does this path not route" — it is segment-based, not string-based.

TLS

```
spec:
  tls:
    - hosts: ["pingd.example.com"]
      secretName: pingd-tls
```

The Secret must be type: `kubernetes.io/tls` with `tls.crt` and `tls.key` keys, **in the same namespace as the Ingress**. Cross-namespace certificate references do not work, and this is a recurring annoyance when one team owns certificates.

In practice, use **cert-manager** rather than managing certificates by hand:

```
metadata:
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
```

cert-manager sees the annotation, obtains a certificate via ACME, writes the Secret, and renews it. It is one of the few pieces of cluster software that is unambiguously worth installing on day one.

The annotation problem

Look again at the manifest above:

```
annotations:
  nginx.ingress.kubernetes.io/rewrite-target: /$2
```

The Ingress *spec* covers host and path routing and TLS. Everything else — rewrites, timeouts, request size limits, rate limiting, authentication, CORS, canary weighting, sticky sessions, custom headers — is **controller-specific annotations**.

The consequences are serious:

- **Not portable.** Switching from ingress-nginx to Traefik means rewriting every annotation.
- **Not validated.** A misspelled annotation is silently ignored. There is no error, no event, no warning — the behaviour you asked for just does not happen, and you discover it in production.
- **Not typed.** Values are strings, parsed by the controller.
- **Not delegatable.** Annotations are all-or-nothing on one object, so you cannot let an application team set a timeout while preventing them changing authentication.

ingress-nginx alone has well over a hundred annotations. This is the reason the Gateway API exists.

A security note

Some ingress-nginx annotations allow arbitrary nginx configuration snippets, which has been the basis of real CVEs — a user with permission to create an Ingress in any namespace could achieve code execution in the controller, which typically runs with broad cluster access. Modern versions disable snippets by default (`allow-snippet-annotations: false`). **Keep it disabled**, keep the controller patched, and treat "can create Ingress" as a privileged permission (Chapter 23).

Gateway API

The successor, and the direction the ecosystem is moving. It is a set of CRDs rather than a built-in type, so it must be installed:

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.2.1/standard-install.yaml
```

The central idea is **role separation** into three resources:

```

# Infrastructure team: what load balancers exist
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: production
spec:
  controllerName: example.com/gateway-controller
---
# Cluster operator: listeners, TLS, and who may attach
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: external
  namespace: infra
spec:
  gatewayClassName: production
  listeners:
    - name: https
      protocol: HTTPS
      port: 443
      tls:
        certificateRefs:
          - name: wildcard-tls
  allowedRoutes:
    namespaces:
      from: Selector
      selector:
        matchLabels: { gateway-access: "true" }
---
# Application team: routing rules, in their own namespace
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: pingd
  namespace: apps
spec:
  parentRefs:
    - name: external
      namespace: infra
  hostnames: ["pingd.example.com"]
  rules:
    - matches:
        - path: { type: PathPrefix, value: /api }
      filters:
        - type: RequestHeaderModifier
          requestHeaderModifier:
            add: [{name: X-Env, value: prod}]
      backendRefs:
        - name: pingd
          port: 80

```

What it fixes:

Typed, portable configuration. Header manipulation, redirects, rewrites, request mirroring and traffic splitting are **fields in the spec**, validated by the API server. A typo is rejected at apply time rather than silently ignored.

Real role separation. The infrastructure team owns the Gateway and controls which namespaces may attach routes. Application teams own HTTPRoutes in their own namespaces and cannot change TLS or listener configuration. This is genuinely not expressible with Ingress.

Native traffic splitting — canary deployments without annotations:

```
backendRefs:
  - name: pingd-stable
    port: 80
    weight: 90
  - name: pingd-canary
    port: 80
    weight: 10
```

Protocols beyond HTTP. TCPRoute, UDPRoute, TLSRoute, GRPCRoute are part of the model rather than bolted on.

Cross-namespace references, governed by ReferenceGrant — so a route in one namespace can target a Service in another, with the target namespace's explicit consent.

Should you adopt it?

The honest position as of Kubernetes v1.37:

- The **core** (GatewayClass, Gateway, HTTPRoute) is stable (v1) and implemented by Istio, Envoy Gateway, Cilium, Traefik, Contour, NGINX Gateway Fabric, Kong and the major cloud controllers.
- Some **extended** features remain experimental and implementation-dependent.
- Ingress is **not deprecated** and will keep working indefinitely. There is no forced migration.

For a **new** cluster or a new ingress deployment, Gateway API is the better choice. For a working ingress-nginx setup, migrate when you have a reason — multi-team routing, canary requirements, or annotation sprawl you can no longer reason about.

Debugging Ingress

The failure this book hit while being written is instructive. The first requests returned:

```
curl: (56) Recv failure: Connection reset by peer
```

The Ingress existed, the Service had endpoints, the controller was Running. The problem was that **the controller pod had been scheduled onto a node where external traffic never arrives** — it uses hostPort: 80, and only one node had the host port mapped.

```
kubectl get pod -n ingress-nginx -l app.kubernetes.io/component=controller \
-o custom-columns='NAME:.metadata.name,NODE:.spec.nodeName,HOSTPORT:.spec.containers[0].ports[*].hostPort'
```

NAME	NODE	HOSTPORT
ingress-nginx-controller-596f5b6bcf-c6rzc	k8sbook-worker2	80,443

Pinning it to the right node fixed it immediately. The general lesson: **an Ingress problem is often not an Ingress problem** — it is where the controller runs, whether traffic reaches it, or whether the backend Service has endpoints.

The debugging order:

```
kubectl get ingressclass
# is there a controller at all?
kubectl describe ingress pingd           # events, and resolved
backends
kubectl get endpointslice -l kubernetes.io/service-name=pingd # does the backend
have pods?
kubectl logs -n ingress-nginx deploy/ingress-nginx-controller --tail=50
```

`kubectl describe ingress` is particularly useful because it shows whether the controller resolved your backend Service, and emits events when it could not.

Try it

Confirm a controller exists:

```
kubectl get ingressclass
```

Apply the Ingress and check it was accepted:

```
kubectl apply -f examples/manifests/13-ingress.yaml && sleep 10 && kubectl get
ingress pingd
```

Host-based routing:

```
curl -sS -H 'Host: pingd.local' http://localhost:8180/
```

Path-based routing with a rewrite:

```
curl -sS http://localhost:8180/api/
```

An unmatched host falls through to the default backend:

```
curl -sS -o /dev/null -w 'status=%{http_code}\n' -H 'Host: nope.local' http://
localhost:8180/
```

TLS on the mapped HTTPS port (self-signed, hence `-k`):

```
curl -sSk -o /dev/null -w 'https status=%{http_code}\n' -H 'Host: pingd.local'
https://localhost:8443/
```

Now see what `describe` tells you about backend resolution:

```
kubectl describe ingress pingd | tail -20
```

And prove the annotation problem — a misspelled annotation is accepted silently:

```
kubectl annotate ingress pingd nginx.ingress.kubernetes.io/rewrite-targett=/wrong
--overwrite
```

No error, no event, no effect. That is the whole argument for Gateway API in one command.

```
kubectl annotate ingress pingd nginx.ingress.kubernetes.io/rewrite-targett-
```

Takeaways

- An Ingress without a controller does nothing. Check `kubectl get ingressclass` first.
- `pathType: Prefix` matches **path segments**, not string prefixes.
- TLS Secrets must be in the same namespace as the Ingress. Use `cert-manager`.
- **Everything beyond host/path/TLS is controller-specific annotations:** unportable, untyped, and **silently ignored when misspelled**.
- Keep `ingress-nginx` snippet annotations disabled and the controller patched; "can create Ingress" is a privileged permission.
- **Gateway API** makes routing typed and validated, separates infrastructure from application roles, and supports traffic splitting and non-HTTP protocols natively. Core is stable; prefer it for new deployments.
- Ingress failures are often really "the controller is not where the traffic arrives" or "the backend Service has no endpoints".

Previous: Chapter 12 — DNS and service discovery · Next: Chapter 14 — NetworkPolicy

Chapter 14 — NetworkPolicy

By default, every pod in a cluster can reach every other pod, in every namespace. No firewall, no segmentation, no restrictions. A compromised frontend can talk directly to your database, your internal admin service and the cloud metadata endpoint.

NetworkPolicy is how you change that. It is also the feature most often assumed to be working when it is not.

The prerequisite nobody checks

NetworkPolicy objects do nothing unless your CNI plugin implements them. The API accepts them, `kubectl get networkpolicy` lists them, and if your CNI ignores policy they have precisely zero effect. There is no warning.

Flannel, notably, does not enforce NetworkPolicy. Calico, Cilium, Antrea, Weave and kindnet do.

So before writing any policy, **test that enforcement works.** The procedure is in Try it below, and it takes a minute. Deploying policies onto a cluster that ignores them is worse than having none, because you believe you are protected.

The model

Three rules that explain all the confusing behaviour:

- 1. Policies are additive allow-lists.** There is no `deny` rule. A policy grants permission; multiple policies selecting a pod are unioned.
- 2. A pod is "selected" if any policy's podSelector matches it.** Once selected for a direction (Ingress or Egress), **everything not explicitly allowed in that direction is denied.** An unselected pod is completely unrestricted.
- 3. Ingress and Egress are independent.** A policy with `policyTypes: [Ingress]` does not restrict outbound traffic at all.

The consequence people trip over: a policy that selects a pod but lists **no rules** is a *deny-all* for that direction. That is not a special syntax — it is rule 2 with an empty allow-list.

Default deny

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector: {}           # every pod in this namespace
  policyTypes: [Ingress]    # no ingress rules → deny all inbound
```

`podSelector: {}` means *all pods in the namespace*. Note that NetworkPolicy is namespaced — this affects one namespace, and you need it in every namespace you want protected.

Then allow what is needed:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-pingd-from-client
spec:
  podSelector:
    matchLabels: { app: pingd }
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels: { role: client }
      ports:
        - protocol: TCP
          port: 8000
```

Measured on this cluster, with both policies applied:

```
no label      -> blocked
role=client   -> 200
role=other    -> blocked
```

Traffic to `pingd` succeeds only from pods labelled `role=client`, on port 8000. Both the Service ClusterIP path and a direct pod-IP connection behave identically, which is expected: kube-proxy DNATs to the pod IP before policy is evaluated, so policy sees the same source and destination either way.

Note the port is the pod's port (8000), not the Service's port (80). Policy operates on pod-level traffic after DNAT. Writing `port: 80` here silently blocks everything, and it is one of the most common NetworkPolicy mistakes.

Selecting sources

Four kinds of `from/to` peer, and the distinction between two of them causes real misconfiguration:

```

ingress:
- from:
  # (a) pods in the SAME namespace
  - podSelector:
    matchLabels: { role: client }

  # (b) ALL pods in namespaces matching this label
  - namespaceSelector:
    matchLabels: { kubernetes.io/metadata.name: monitoring }

  # (c) specific pods in specific namespaces ← note: ONE list item
  - namespaceSelector:
    matchLabels: { team: platform }
    podSelector:
    matchLabels: { app: prometheus }

  # (d) IP ranges (for traffic from outside the cluster)
  - ipBlock:
    cidr: 10.0.0.0/8
    except: ["10.5.0.0/16"]

```

YAML list structure is the trap. In (c), `namespaceSelector` and `podSelector` are in the *same* list item, meaning "pods matching X **in** namespaces matching Y" — an AND. Put them in separate list items (each with its own `-`) and you get an OR: "all pods in those namespaces, **or** those pods in any namespace". That is dramatically more permissive, and the YAML difference is a single character.

The label `kubernetes.io/metadata.name` is automatically set on every namespace, so you can select a namespace by name without adding labels yourself.

Egress

More valuable than ingress for containing a compromise, and much more disruptive to get wrong.

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: pingd-egress
spec:
  podSelector:
    matchLabels: { app: pingd }
  policyTypes: [Egress]
  egress:
    # DNS – required, and the thing everyone forgets
    - to:
      - namespaceSelector:
          matchLabels: { kubernetes.io/metadata.name: kube-system }
        podSelector:
          matchLabels: { k8s-app: kube-dns }
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
    # the database
    - to:
      - podSelector:
          matchLabels: { app: postgres }
      ports:
        - protocol: TCP
          port: 5432

```

Always allow DNS explicitly. An egress policy without a DNS rule breaks name resolution for the selected pods, and the symptom — everything failing with resolution errors — rarely points people at the network policy they just applied. Both UDP and TCP port 53; large responses fall back to TCP.

Egress policy is also how you block access to the **cloud metadata endpoint** (169.254.169.254), which is a genuine privilege-escalation path on cloud providers:

```

egress:
  - to:
    - ipBlock:
        cidr: 0.0.0.0/0
        except:
          - 169.254.169.254/32

```

What NetworkPolicy cannot do

Being clear about the limits, because it is often oversold:

- **No L7 awareness.** It is IPs, ports and protocols. You cannot allow `GET /api/public` while denying `POST /admin`. That needs a service mesh or an application-layer proxy.
- **No egress-by-DNS-name.** `ipBlock` takes CIDRs. Allowing `"api.stripe.com"` is not expressible, because the addresses behind a name change. Several CNIs offer this as a proprietary extension (Cilium's `CiliumNetworkPolicy` has `toFQDNs`), but it is not standard.
- **No logging in the standard API.** You cannot see what was denied. Most CNIs provide this separately (Cilium's Hubble, Calico's flow logs), and you will want it — silent drops are miserable to debug.

- **Node-local traffic is inconsistent.** Traffic from a node's own kubelet — health probes, for instance — may bypass policy depending on the CNI. Do not rely on policy to block it.
- **Not a strong tenant boundary.** It restricts network reachability. Pods still share a kernel (Chapter 28).

A workable adoption path

Applying default-deny to a running cluster breaks things immediately and in ways that are hard to trace. A safer sequence:

1. **Verify your CNI enforces policy** at all.
2. **Start in one non-critical namespace.**
3. **Write egress policies first** — they are more valuable and their failures are more obvious.
4. **Always include DNS.**
5. **Apply default-deny last**, after the allow rules exist.
6. **Get flow logs** from your CNI before you need them.
7. **Test from both sides** — that allowed traffic works, and that denied traffic is denied. Only testing the first half is how you end up with policies that permit everything.

For generated starting points, `kubectl get networkpolicy -o yaml` on a mature cluster and tools like Cilium's policy editor help; do not deploy generated policy unread.

Try it

First, verify enforcement. Baseline — any pod can reach `pingd`:

```
kubectl run t --rm -it --restart=Never --image=curlimages/curl:8.11.1 --quiet --
curl -sS -m5 http://pingd/
```

```
{"service":"pingd","version":"1.0.0","reply":"pong"}
```

Apply default-deny plus a narrow allow:

```
kubectl apply -f examples/manifests/14-networkpolicy.yaml && sleep 6
```

Unlabelled pod — should now fail:

```
kubectl run t --rm -it --restart=Never --image=curlimages/curl:8.11.1 --quiet --
curl -sS -m6 http://pingd/
```

If that **succeeds**, your CNI is not enforcing policy and everything else in this chapter is decorative on your cluster. Find out before relying on it.

Labelled pod — should succeed:

```
kubectl run t --rm -it --restart=Never --labels='role=client' --image=curlimages/
curl:8.11.1 --quiet -- curl -sS -m6 -o /dev/null -w '%{http_code}\n' http://pingd/
```

200

Wrong label — should fail, proving it is the label and not merely "having a label":

```
kubectl run t --rm -it --restart=Never --labels='role=other' --image=curlimages/
curl:8.11.1 --quiet -- curl -sS -m6 http://pingd/
```

Confirm egress is untouched, since we only set `policyTypes: [Ingress]`:

```
kubectl run t --rm -it --restart=Never --labels='role=client' --image=curlimages/
curl:8.11.1 --quiet -- curl -sS -m6 -o /dev/null -w 'external=%{http_code}\n' https
://example.com/
```

Inspect what a policy resolved to:

```
kubectl describe networkpolicy allow-pingd-from-client
```

Clean up — **leaving default-deny in place will break the rest of the book's examples:**

```
kubectl delete -f examples/manifests/14-networkpolicy.yaml
```

Takeaways

- Default is **allow everything, everywhere, across namespaces**.
- **NetworkPolicy does nothing unless your CNI implements it.** Flannel does not. Test enforcement before trusting it.
- Policies are additive allow-lists. Selecting a pod with no rules for a direction denies that direction entirely.
- **Use the pod's port, not the Service's port.** Policy is evaluated after DNAT.
- `namespaceSelector` and `podSelector` in the *same list item* is AND; in separate items it is OR, and far more permissive. One character of YAML.
- **Always allow DNS in egress policies** (UDP and TCP 53), or name resolution silently breaks.
- Egress policy can block the cloud metadata endpoint — a real escalation path.
- No L7 rules, no DNS-name egress, no logging in the standard API. Get flow logs from your CNI.
- Adopt incrementally: allow rules first, default-deny last, test both directions.

Previous: Chapter 13 — Ingress and Gateway API · Next: Chapter 15 — Storage

Chapter 15 — Storage

Containers are ephemeral; data is not. Kubernetes separates *what a pod asks for* from *what actually provides it*, which makes manifests portable across clusters but adds a layer of indirection that is worth understanding before it fails.

Volumes that are not persistent

Not everything needs a PersistentVolume.

`emptyDir` — a directory created when the pod is scheduled and deleted when the pod is removed. Shared between containers in the pod (Chapter 5).

```
volumes:
- name: tmp
  emptyDir: {}
- name: cache
  emptyDir:
    medium: Memory          # tmpfs - RAM, not disk
    sizeLimit: 256Mi
```

This is what the `pingd` Deployment uses to give a `readOnlyRootFilesystem` container a writable `/tmp` — the same pattern as the Docker book's `--read-only --tmpfs /tmp`.

Two cautions: `emptyDir` consumes the node's **ephemeral storage**, and a pod filling the disk triggers `DiskPressure` and evicts other pods (Chapter 20) — set `sizeLimit`. And `medium: Memory` counts against the container's **memory limit**, so a large `tmpfs` can get you OOM-killed.

`configMap`, `secret`, `downwardAPI`, `projected` — covered in Chapter 7. `projected` combines several into one directory, which is how service account tokens are mounted.

`hostPath` — mounts a path from the node.

```
volumes:
- name: docker-sock
  hostPath:
    path: /var/run/containerd/containerd.sock
    type: Socket
```

`hostPath` is a **privilege escalation waiting to happen**. A pod that can mount `/var/lib/kubelet` or the container runtime socket effectively owns the node, and mounting `/etc` or `/` gives host root. Legitimate uses are node-level agents (log shippers, CSI drivers, monitoring) — and those are exactly what an attacker would like to impersonate. Restrict it with Pod Security Standards or a policy engine (Chapters 23–24). For pod-local scratch, use `emptyDir`; for durable data, use a PVC.

PersistentVolumes and PersistentVolumeClaims

The core abstraction:

- A **PersistentVolumeClaim (PVC)** is a request: "I need 10Gi, ReadWriteOnce."

- A **PersistentVolume (PV)** is the actual storage.
- A **StorageClass** describes *how* to provision PVs dynamically.

Pods reference PVCs, never PVs. That indirection is what makes a manifest portable: the same PVC gets an EBS volume on AWS, a PD on GCP, and a local directory on kind.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pingd-data
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: standard
  resources:
    requests:
      storage: 10Gi
```

```
volumeMounts:
- name: data
  mountPath: /var/lib/pingd
volumes:
- name: data
  persistentVolumeClaim:
    claimName: pingd-data
```

StorageClasses

```
kubectl get storageclass
```

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION			
standard (default)	rancher.io/local-path	Delete	WaitForFirstConsumer
false			

Four columns, all of which matter:

PROVISIONER — the CSI driver. `rancher.io/local-path` here; `ebs.csi.aws.com`, `pd.csi.storage.gke.io` elsewhere.

RECLAIMPOLICY — what happens to the PV when its PVC is deleted. **Delete destroys the underlying volume and its data.** Retain keeps it for manual recovery. Most default classes use Delete, which means deleting a PVC deletes your data with no confirmation. For anything you care about, use a class with Retain, or make sure your backups are real.

VOLUMEBINDINGMODE — Immediate provisions as soon as the PVC is created; WaitForFirstConsumer waits until a pod using it is scheduled. WaitForFirstConsumer **is almost always what you want** in a multi-zone cluster: with Immediate, a volume can be created in zone A while the pod is later scheduled to zone B, and the pod is then permanently Pending because the volume cannot follow it.

ALLOWVOLUMEEXPANSION — whether you can grow a PVC later by editing `spec.resources.requests.storage`. `false` here. If your class supports it, expansion is online for most drivers; **shrinking is never supported**.

Note (default): a PVC with no `storageClassName` uses the default class. A PVC with `storageClassName: ""` explicitly requests *no* class, meaning static binding only — a subtle distinction that produces permanently `Pending` PVCs.

Access modes

Mode	Meaning
<code>ReadWriteOnce (RWO)</code>	Read-write by one node
<code>ReadOnlyMany (ROX)</code>	Read-only by many nodes
<code>ReadWriteMany (RWX)</code>	Read-write by many nodes
<code>ReadWriteOncePod (RWOP)</code>	Read-write by exactly one pod

Two clarifications worth having.

RWO is per node, not per pod. Two pods on the *same* node can both mount an RWO volume. If you need a hard single-writer guarantee — and for most databases you do — `ReadWriteOncePod` is the mode that actually provides it.

RWX is not widely available. Block storage (EBS, GCP PD, Azure Disk) is RWO only. RWX needs a filesystem: NFS, EFS, Azure Files, CephFS. Writing `ReadWriteMany` against a block storage class gives you a PVC that never binds. If your design requires many pods writing one volume, reconsider — object storage is usually the better answer.

The lifecycle

```
PVC created (Pending)
  → StorageClass provisions a PV, or an existing PV matches
  → PVC Bound to PV
  → pod schedules, volume attaches to the node, mounts into the container
  → pod deleted → volume unmounts and detaches; PVC and PV survive
  → PVC deleted → reclaim policy: Delete (destroy) or Retain (keep, status Released)
```

A `Released` PV is **not automatically reusable** — it still references the old claim. Making it available again means clearing `spec.claimRef` by hand. This surprises people expecting `Retain` to mean "reuse".

Volume snapshots

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: pingd-data-snapshot
spec:
  volumeSnapshotClassName: csi-snapshotter
  source:
    persistentVolumeClaimName: pingd-data
```

Then restore by creating a PVC with `dataSource` pointing at the snapshot.

Requires the snapshot CRDs and a CSI driver that supports it. And the caveat that matters: **a snapshot of a running database is crash-consistent, not application-consistent** — the same warning as `tar`-ing a live data directory in the Docker book. For a database, use its own backup tooling, or quiesce it first.

Stateful workloads, honestly

Kubernetes can run databases. The question is whether it should.

Reasons to use a managed database: backups, point-in-time recovery, failover, version upgrades, and connection scaling are solved problems you are buying rather than building. The operational surface you avoid is enormous, and the failure modes are someone else's pager.

If you do run it in-cluster:

- Use a **mature operator** — CloudNativePG, Zalando's postgres-operator, Percona, Strimzi for Kafka. The operator is where the backup, failover and upgrade logic lives, and that logic is the actual product. A hand-written StatefulSet gives you a database that runs and no story for what happens when it does not.
- Use `ReadWriteOncePod` where supported.
- Use fast local storage (local PVs or NVMe-backed classes) — network storage latency is often the limiting factor for database performance.
- Set a **PodDisruptionBudget** (Chapter 25) so a node drain cannot take your quorum.
- Test restores. Untested backups are a hypothesis.

The middle path many teams settle on: stateless workloads in Kubernetes, stateful ones managed. That is not a failure of nerve; it is an accurate assessment of where the operational risk lies.

Try it

Look at what your cluster provides:

```
kubectl get storageclass
```

```
standard (default)  rancher.io/local-path  Delete  WaitForFirstConsumer  false
```

Note `WaitForFirstConsumer` — create a PVC and watch it stay `Pending` on purpose:

```
kubectl apply -f examples/manifests/15-pvc.yaml && kubectl get pvc pingd-data
```

NAME	STATUS	VOLUME	CAPACITY	STORAGECLASS
pingd-data	Pending			standard

That is not an error. `kubectl describe pvc pingd-data` says so:

```
kubectl describe pvc pingd-data | grep -A3 Events
```

```
WaitForFirstConsumer waiting for first consumer to be created before binding
```

Now create a pod that uses it, and watch it bind:

```
kubectl apply -f examples/manifests/15-pod-with-pvc.yaml && sleep 15 && kubectl get pvc pingd-data
```

Write something, then prove it survives the pod:

```
kubectl exec pvc-demo -- sh -c 'echo "written at $(date -u +%T)" > /data/note.txt; cat /data/note.txt'
```

```
kubectl delete pod pvc-demo && kubectl apply -f examples/manifests/15-pod-with-pvc.yaml && sleep 12
```

```
kubectl exec pvc-demo -- cat /data/note.txt
```

Same content — the pod was destroyed and recreated; the data was not.

See where it actually lives on the node: 

```
kubectl get pv -o custom-columns='NAME:.metadata.name,CLAIM:.spec.claimRef.name,RECLAIM:.spec.persistentVolumeReclaimPolicy,PATH:.spec.local.path'
```

Note `RECLAIM: Delete`. Clean up — **destructive: with this reclaim policy, deleting the PVC destroys the data permanently:**

```
kubectl delete pod pvc-demo && kubectl delete pvc pingd-data
```

```
kubectl get pv
```

pvc-f4790ebf-...	100Mi	RWO	Delete	Released	default/pingd-data
------------------	-------	-----	--------	----------	--------------------

It passes through `Released` first — reclamation is asynchronous — and the provisioner removes it shortly after. Watch it disappear:

```
sleep 30 && kubectl get pv
```

That transient `Released` state is worth recognising: a PV stuck in `Released` for a long time means the provisioner is not reclaiming it, which on a `Retain` class is expected and on a `Delete` class means something is wrong.

Takeaways

- `emptyDir` for pod-local scratch — it counts against ephemeral storage, and `medium: Memory` counts against the **memory limit**.
- `hostPath` **is a privilege escalation risk**. Restrict it by policy.
- Pods reference PVCs, never PVs. That indirection is what makes manifests portable.
- `reclaimPolicy: Delete` **destroys your data when the PVC is deleted**, with no confirmation. Most default classes use it.
- `WaitForFirstConsumer` **avoids provisioning a volume in the wrong zone**. Prefer it.
- RWO is **per node**, not per pod — use `ReadWriteOncePod` for a real single-writer guarantee. RWX needs a file-based backend and is unavailable on block storage.
- A `Retained` PV is `Released`, not reusable — clear `claimRef` to reuse it.
- Volume snapshots are crash-consistent, not application-consistent.
- Prefer managed databases. If in-cluster, use a mature operator, a PDB, and tested restores.

Previous: Chapter 14 — NetworkPolicy · Next: Chapter 16 — The debugging mindset

Chapter 16 — The debugging mindset

The Docker book had four layers to localise a problem. Kubernetes has five, and the extra one — the control plane — can itself be the fault. That is the whole difficulty: a pod that will not serve traffic might be failing for reasons in the image, the scheduler, RBAC, an admission webhook, a probe, a NetworkPolicy, a taint, or a Service selector, and the symptom looks identical in most of those cases.

The way through is the same as before: **localise the layer before investigating**.

The five layers

Layer	Question	First command
1. Object	Does the desired state say what I think?	<code>kubectl get -o yaml</code> , <code>kubectl describe</code>
2. Scheduling	Was it placed on a node?	<code>kubectl get pod -o wide</code> , events
3. Container	Did the image pull and the process start?	<code>kubectl logs</code> , container statuses
4. Network	Can it be reached, and can it reach?	endpoints, DNS, policy
5. Cluster	Is the platform itself healthy?	node status, control plane pods

Most wasted time is spent in the wrong layer. "The API can't reach the database" is a layer-4 hypothesis, and about a third of the time the truth is layer 3 (the database pod is crash-looping) or layer 1 (the Service selector does not match).

The triage sequence

Four commands, ninety seconds, before forming any theory.

1. What state is it in?

```
kubectl get pods -o wide
```

The `STATUS` and `READY` columns are different facts. `Running` with `0/1 ready` means the process started and readiness fails — a completely different problem from `Pending`.

2. What does the object say happened?

```
kubectl describe pod POD
```

This is the highest-value single command in Kubernetes debugging. It gives you the container statuses, the resolved configuration, the conditions, **and the events** — and the events are usually the answer outright.

3. What did the application say?

```
kubectl logs POD
```

```
kubectl logs POD --previous
```

4. What has the cluster been doing?

```
kubectl get events --sort-by=.lastTimestamp | tail -20
```

Remember from Chapter 4 that events expire after about an hour and are unsorted by default.

Read the status columns precisely

The `STATUS` column is a composite that the API does not store as a single field, and knowing what each value means saves a lot of guessing:

STATUS	Layer	Meaning
Pending	2	Not scheduled, or scheduled and still pulling/initialising
ContainerCreating	3	Scheduled; pulling image, mounting volumes
Running + 1/1	—	Working
Running + 0/1	3/4	Process up, readiness failing
ImagePullBackOff / ErrImagePull	3	Cannot fetch the image
CreateContainerConfigError	1	Missing ConfigMap/Secret reference
CrashLoopBackOff / Error	3	Container keeps exiting
Terminating	—	Being deleted; stuck here means a finalizer or a grace period
Evicted	5	Node pressure removed it (Chapter 20)
OOMKilled	3	Memory limit — but see Chapter 8, it may say Error

Two of these deserve care.

Pending splits into two very different problems. Look for a node assignment:

```
kubectl get pod POD -o jsonpath='{.spec.nodeName}{"\n"}'
```

Empty means the **scheduler** could not place it — a `FailedScheduling` event will say why. Non-empty means it *is* scheduled and the kubelet is working on it — image pull, volume mount, init containers. Same status, opposite investigations.

Running never means "working". It means containers exist. `READY` is what determines traffic.

Events are the narration

Events explain what controllers decided, and their messages are unusually specific. A real one from this cluster:

```
Warning FailedScheduling 8s default-scheduler
  0/3 nodes are available: 1 node(s) had intolerated taint(s),
  2 Insufficient cpu, 2 Insufficient memory.
  preemption: 0/3 nodes are available: 3 Preemption is not helpful for scheduling.
```


That single message tells you the cluster size, that one node was excluded by a taint, that the other two lacked both CPU and memory, and that preemption would not help. There is very little left to guess.

Scoped to one object:

```
kubectl events --for pod/POD
```

```
kubectl get events --field-selector involvedObject.name=POD,type=Warning
```

kubectl describe **reading order**

describe output is long. Read it in this order:

1. **Status:** and **Conditions:** — the summary and what is unsatisfied.
2. **Containers:** → **State / Last State** — including **Reason** and **Exit Code**. **Last State** is the run that failed.
3. **Events:** at the bottom — usually the actual answer.
4. **Node:**, **IP:**, **Mounts:**, **Environment:** — to confirm what it actually got versus what you meant.

That fourth point catches an unreasonable share of problems: the gap between the manifest you edited and the object that is running. A different namespace, an un-applied change, a `kustomize` overlay you forgot, an HPA overriding replicas.

Ask what changed

Same discipline as the Docker book, and it wins as often:

```
kubectl rollout history deployment/pingd
```

```
kubectl get events --sort-by=.lastTimestamp -A | tail -30
```

```
kubectl get pods -A --sort-by=.metadata.creationTimestamp | tail -10
```

- **Was it ever working?** Never-worked points at layers 1 and 2; stopped-working at 3, 4 and 5.
- **Does the previous revision work?** `kubectl rollout undo` is both a fix and a bisection.
- **Is it one pod or all of them?** One pod means a node or a scheduling problem; all pods means the image, config or a dependency.
- **Is it one namespace or the cluster?** Cluster-wide points at layer 5 — CoreDNS, a webhook, the control plane.

That third question is worth making a habit. Compare a failing pod's node against a healthy one:

```
kubectl get pods -o wide -l app=pingd
```

If every failure is on one node, you have a node problem (Chapter 21), not an application problem.

Reduce the system

Strip away layers until the failure disappears:

```
kubectl run debug --rm -it --restart=Never --image=nicolaka/netshoot -- bash
```

A fresh pod in the same namespace tests DNS, policy and Service routing *without* your application. If it can reach the Service and your pod cannot, the problem is inside your pod.

```
kubectl port-forward deploy/pingd 8080:8000
```

Bypasses Service, kube-proxy, Ingress and DNS entirely — straight to a pod. If this works and your Service does not, you have localised to layer 4 and specifically to the Service.

```
kubectl run direct --rm -it --restart=Never --image=curlimages/curl -- curl -sS
http://POD_IP:8000/
```

Pod IP directly, bypassing the Service but not the network. Between these three you can usually pin the failure to a single hop.

Rules of thumb

Empty logs mean the process never wrote to stdout — it died before starting, it logs to a file, or output is buffered. Everything the Docker book said about `PYTHONUNBUFFERED` applies.

A Service with no ready endpoints is the single most common networking cause. Check it before anything else (Chapter 19).

If it worked before the deploy, it is the deploy. Roll back first, investigate after. The cluster is not a debugging environment when users are affected.

Check the namespace. A shocking proportion of "the resource doesn't exist" is `kubectl` pointed at `default`.

Check the context. The rest is `kubectl` pointed at the wrong cluster.

When one pod is broken and its siblings are fine, compare them. Same image, same config, different outcome means the difference is the node, the scheduling, or local state:

```
diff <(kubectl get pod GOOD -o yaml) <(kubectl get pod BAD -o yaml) | head -40
```

What the rest of Part IV covers

Symptom	Chapter
Pending, ImagePullBackOff, CrashLoopBackOff, config errors	17
I need a shell, or there is no shell	18
Cannot reach a Service; DNS; policy	19
OOM, throttling, evictions, preemption	20
NotReady nodes, taints, control plane	21
I have an error message and want the answer	22

Takeaways

- Five layers: object, scheduling, container, network, cluster. Localise before investigating.
- Triage: `get pods -o wide` → `describe` → `logs` (and `--previous`) → `events`.
- `describe` **is the highest-value command** because it includes events.
- Pending with **no** `nodeName` is a scheduler problem; with a `nodeName` it is a kubelet problem. Same word, opposite investigations.
- Running ≠ working. READY decides traffic.
- `FailedScheduling` messages are unusually specific — read them literally.
- Reduce the system: a fresh netshoot pod, `port-forward` to bypass Services, direct pod IP.
- One pod broken and siblings fine means node or scheduling; all broken means image or config; cluster-wide means layer 5.
- Check the namespace, then the context.

Previous: Chapter 15 — Storage · Next: Chapter 17 — Pods that won't run

Chapter 17 — Pods that won't run

Five failure states cover almost everything. Each has a distinctive signature, and each is measured here against a real cluster.

Pending — the scheduler could not place it

First, split the two cases:

```
kubectl get pod POD -o jsonpath='{.spec.nodeName}{"\n"}'
```

Empty → the scheduler could not find a node. Non-empty → it is scheduled and the kubelet is still working (see `ContainerCreating` below).

For the unscheduled case, the event says exactly why:

```
kubectl describe pod POD | grep -A5 Events
```

Measured, for a pod requesting 500 CPUs and 900Gi:

```
Warning FailedScheduling 8s default-scheduler
  0/3 nodes are available: 1 node(s) had untoleration taint(s),
  2 Insufficient cpu, 2 Insufficient memory.
  preemption: 0/3 nodes are available: 3 Preemption is not helpful for scheduling.
```

Read it as arithmetic: 3 nodes, minus 1 for a taint, leaves 2, and both lacked capacity.

The common causes, in the order they appear in that message:

`Insufficient cpu` / `Insufficient memory`. Remember Chapter 8: this is about **requests**, not usage. The node can be idle and still full.

```
kubectl describe node NODE | sed -n '/Allocated resources/,/Events/p'
```

Either lower the request or add capacity. A request larger than any single node's allocatable can never schedule — check that first, because it is a manifest bug, not a capacity problem.

`untoleration taint`. Control-plane nodes are tainted by default; dedicated node pools usually are too.

```
kubectl get nodes -o custom-
columns='NAME:.metadata.name,TAINTS:.spec.taints[*].key'
```

Add a toleration, or target different nodes.

`node(s) didn't match Pod's node affinity/selector`. Your `nodeSelector` or affinity matches nothing. Verify the labels actually exist:

```
kubectl get nodes --show-labels
```

pod has unbound immediate PersistentVolumeClaims. A PVC that will not bind (Chapter 15). Note that with WaitForFirstConsumer a Pending PVC is *normal* and not the cause.

didn't match pod topology spread constraints. Chapter 25.

No events at all on a Pending pod means the scheduler never processed it — it is down, or the pod names a non-existent schedulerName.

ContainerCreating — **scheduled, but not started**

Scheduled (it has a nodeName) and stuck. The kubelet is doing something and failing. describe tells you which:

- **Volume mount failures** — a PVC that will not attach, a node that cannot reach the storage backend, or a multi-attach error where an RWO volume is still mounted on another node. That last one is common after an ungraceful node failure and often needs manual intervention.
- **Image pull in progress** — normal for a large image on first pull; check the events for progress.
- **CNI failures** — failed to setup network for sandbox. The CNI plugin on that node is broken, or IP addresses are exhausted.
- **Secret/ConfigMap for a projected volume missing.**

Stuck here for more than a minute or two with no progress in the events is a node-level problem (Chapter 21).

ImagePullBackOff / ErrImagePull

Measured:

```
kubectl run f-pull --image=pingd:does-not-exist --restart=Never
```

STATUS	REASON
Pending	ErrImagePull

Normal	BackOff	10s	kubelet	Back-off pulling image "pingd:does-not-exist"
Warning	Failed	10s	kubelet	Error: ImagePullBackOff

ErrImagePull is the first failure; ImagePullBackOff is the retry backoff. The full reason is in the events:

```
kubectl describe pod POD | grep -A10 Events
```

Causes, and how to tell them apart:

Typo, or the tag does not exist. Verify from outside the cluster:

```
docker manifest inspect IMAGE:TAG
```

Private registry without credentials — pull access denied or authentication required:

```
spec:
  imagePullSecrets:
  - name: regcred
```

```
kubectl create secret docker-registry regcred \
  --docker-server=ghcr.io --docker-username=USER --docker-password="$TOKEN"
```

The Secret must be **in the same namespace as the pod**. Copying a working Deployment to a new namespace and forgetting the pull secret is a very common cause.

Rate limiting — `toomanyrequests`. Authenticate to Docker Hub even for public images.

Wrong architecture — `no matching manifest for linux/arm64`. Chapter 10 of the Docker book.

The image exists locally but the cluster cannot see it. This is the one that catches people on local clusters: a `kind` or `minikube` node has its own image store. Your laptop's `docker images` is irrelevant.

```
kind load docker-image pingd:latest --name k8sbook
```

And there is a subtlety that compounds it: `imagePullPolicy` **defaults to** `Always` **when the tag is** `:latest`, and `IfNotPresent` otherwise. So a locally-loaded `:latest` image is still pulled from the registry and fails. That is why this book's Deployment sets `imagePullPolicy: IfNotPresent` explicitly — without it, the loaded image is ignored.

`CreateContainerConfigError`

Measured, for a pod referencing a ConfigMap that does not exist:

STATUS	REASON	MSG
Pending	CreateContainerConfigError	configmap "nope-missing" not found

The message is exact. Causes: a missing ConfigMap or Secret, a missing **key** within one, or a name typo. Note this fails at *container creation*, not at `kubectl apply` — the Deployment was accepted happily (Chapter 7).

Mark genuinely optional references as such:

```
envFrom:
- configMapRef:
  name: optional-config
  optional: true
```

The sibling error `CreateContainerError` usually means a bad command or working directory, and `RunContainerError` a runtime-level failure.

`CrashLoopBackOff` — **and what it actually looked like**

The container starts, exits, and the kubelet restarts it with exponential backoff.

Measured on this cluster with a container that exits 1 immediately, sampling every 2 seconds:

```
t= 2s ContainerCreating restarts=0
t= 4s Error restarts=1
t= 18s Error restarts=2
t= 40s Error restarts=3
t= 86s Error restarts=4
```

Two things worth noting.

The gaps are the backoff: 14 s, 22 s, 46 s — roughly the documented 10/20/40-second doubling, capped at 5 minutes. That progression *is* the diagnosis.

The STATUS column said Error throughout, never CrashLoopBackOff. On Kubernetes v1.37 with this failure shape, the string every tutorial describes did not appear. `CrashLoopBackOff` is still extremely common and you will see it plenty — but do not treat its absence as evidence that a container is not crash-looping. **The reliable signals are a climbing restart count and widening gaps**, not a particular status string.

```
kubectl get pod POD -o
jsonpath='{.restarts={.status.containerStatuses[0].restartCount}
lastExit={.status.containerStatuses[0].lastState.terminated.exitCode}}{"\n"}'
```

Finding the actual error

The standard advice is `kubectl logs --previous`. It is right, and it does not always work:

```
kubectl logs f-crash --previous
```

```
unable to retrieve container logs for containerd://e62b046af8bb...
```

That happens when the previous container has already been cleaned up. In the same run, plain `kubectl logs` **did** have the error, because the most recent attempt had run and exited:

```
kubectl logs f-crash
```

```
FATAL: cannot reach database
```

So: try both, and do not conclude there are no logs from one failing.

If both are empty, the container is dying before it writes anything. Then:

Check the exit code. 137 = SIGKILL (OOM or an ignored SIGTERM — Chapter 8). 1 = the application chose to exit. 127 = command not found. 126 = not executable. All the Docker book's exit codes apply unchanged.

Replace the command with a sleep to hold the environment open and look around:

```
kubectl run probe --image=pingd:latest --restart=Never --command -- sleep 3600
```

```
kubectl exec -it probe -- sh
```

Or use `kubectl debug` to copy the pod with a different command (Chapter 18):

```
kubectl debug POD --copy-to=probe --container=api -- sleep 3600
```

Common causes

- **A missing required environment variable**, and the app fails fast (which is correct — Chapter 7).
- **A dependency unreachable at startup**. Add retries; `initContainers` help at startup only.
- **A liveness probe failing** — check `restartCount` against probe events. A container restarted by liveness looks identical to one that crashed, except the events say `Unhealthy`.
- **Wrong command or entrypoint**, especially after switching to a distroless base.
- **Permission denied** on a mounted volume, because the image runs as non-root and `fsGroup` was not set (Chapter 15).
- **OOM on startup** — a JVM sizing its heap from the node rather than the limit (Chapter 8).

Terminating forever

A pod stuck `Terminating` is usually one of:

A long grace period plus a process ignoring SIGTERM. It will resolve at `terminationGracePeriodSeconds`. If it always takes exactly that long, PID 1 is not handling signals (Chapter 9).

A finalizer. Something registered cleanup that has not completed:

```
kubectl get pod POD -o jsonpath='{.metadata.finalizers}{"\n"}'
```

A gone node. Pods on a `NotReady` node stay `Terminating` because the kubelet that should confirm deletion is unreachable (Chapter 21).

Force deletion removes the API object without confirming the container stopped:

```
kubectl delete pod POD --force --grace-period=0
```

Use this carefully. For a `StatefulSet` with attached storage, the container may still be running on an unreachable node, and forcing deletion lets a replacement start — two writers on one volume. That is how you corrupt a database.

Try it

Reproduce each state and read its signature.

```
kubectl run f-pull --image=pingd:does-not-exist --restart=Never
```

```
sleep 12 && kubectl get pod f-pull && kubectl describe pod f-pull | grep -A5 Events
```



```
kubectl run f-cfg --image=busybox:1.37 --restart=Never --overrides='{ "spec":
{"containers":[{"name":"f-cfg","image":"busybox:1.37","command":
["sleep","300"],"envFrom":[{"configMapRef":{"name":"nope-missing"}}]}]}'
```

```
sleep 10 && kubectl get pod f-cfg -o custom-
columns='STATUS:.status.phase,REASON:.status.containerStatuses[0].state.waiting.rea
son,MSG:.status.containerStatuses[0].state.waiting.message'
```

Now a crash loop, sampled so you can see the backoff widening:

```
kubectl run f-crash --image=busybox:1.37 --restart=Always --command -- sh -c 'echo
"FATAL: cannot reach database" >&2; exit 1'
```

```
for i in $(seq 1 30); do kubectl get pod f-crash --no-headers; sleep 4; done
```

Watch the restart count climb with widening gaps. Then find the error:

```
kubectl logs f-crash; kubectl logs f-crash --previous
```

And an unschedulable pod, for the arithmetic in the event message:

```
kubectl run f-pend --image=busybox:1.37 --restart=Never --overrides='{ "spec":
{"containers":[{"name":"f-pend","image":"busybox:1.37","command":
["sleep","300"],"resources":{"requests":{"cpu":"500","memory":"900Gi"}}}]}'
```

```
sleep 8 && kubectl describe pod f-pend | grep -A4 Events
```

Clean up:

```
kubectl delete pod f-pull f-cfg f-crash f-pend --force --grace-period=0
```

Takeaways

- Pending with no `nodeName` is the scheduler; with a `nodeName` it is the kubelet. Check which before anything else.
- `FailedScheduling` messages are arithmetic — read them literally.
- Scheduling failures are about **requests**, not usage. A request bigger than any node is a manifest bug.
- `imagePullPolicy` defaults to `Always` for `:latest`, which is why locally-loaded images are ignored on `kind/minikube`. Set `IfNotPresent`.
- `imagePullSecrets` must be in the pod's namespace.
- `CreateContainerConfigError` names the missing ConfigMap or Secret exactly, and happens at container creation, not at apply.
- **Measured on v1.37, a crash loop showed Error, not CrashLoopBackOff.** Trust the climbing restart count and widening gaps, not the status string.
- `kubectl logs --previous` can fail with "unable to retrieve"; try plain `logs` too.

- To debug a crash loop, replace the command with `sleep` or use `kubectl debug --copy-to`.
 - **Force-deleting a StatefulSet pod on an unreachable node risks two writers on one volume.**
-

Previous: Chapter 16 — The debugging mindset · Next: Chapter 18 — Getting inside

Chapter 18 — Getting inside

The Docker book had a chapter on this because production images have no shell. Kubernetes has the same problem and a better answer: **ephemeral containers**, which let you attach a debugging toolbox to a running pod without modifying it, restarting it, or shipping tools in your image.

kubectl exec, and where it stops

```
kubectl exec -it POD -- sh
```

```
kubectl exec -it POD -c CONTAINER -- sh           # multi-container pod
kubectl exec POD -- env | sort
kubectl exec POD -- cat /etc/resolv.conf
```

Same constraint as Docker: it runs a binary **that must exist in the image**. On a distroless pod:

```
kubectl exec distro -- sh -c 'echo hi'
```

```
error: Internal error occurred: ... OCI runtime exec failed:
exec: "sh": executable file not found in $PATH
```

That is the image doing its job. You need a different tool.

Ephemeral containers

kubectl debug injects a container into a **running** pod, sharing its namespaces:

```
kubectl debug POD --image=nicolaka/netshoot --target=CONTAINER -it -- bash
```

Measured against the distroless pod above:

```
kubectl debug distro --image=busybox:1.37 --target=distro -it -- sh
```

```
# ps -o pid,comm
PID  COMMAND
  1  python3      ← the target's process
 19  sh           ← us
```

```
# ls /proc/1/root/
bin boot dev etc home lib ...
```

Two things worked there that are worth naming.

`--target` **shares the process namespace**, so `ps` shows the target's processes and PID 1 is the application. Without `--target` you get a container in the pod's network and IPC namespaces but its own PID namespace — still useful for networking, useless for inspecting processes.

`/proc/1/root/` **is the target container's filesystem.** Same trick as the Docker book: the debug container has BusyBox's filesystem at `/`, and reaches the application's through `/proc`. So you can read its config files, check whether a path exists, and inspect what it actually shipped — with no shell in the target image.

The rest of `/proc` is equally available:

```
cat /proc/1/environ | tr '\0' '\n'    # the target's environment
cat /proc/1/cmdline | tr '\0' ' '    # its exact command line
ls -l /proc/1/fd/                    # open files and sockets
cat /proc/1/limits                    # ulimits in force
```

The caveats

Ephemeral containers cannot be removed. They are recorded on the pod permanently:

```
kubectl get pod distro -o jsonpath='{.spec.ephemeralContainers[*].name}'
```

```
debugger-dpl9x debugger-6c8qt
```

Two debug sessions, two containers, both there forever. They stop running but stay in the spec, and the only way to clear them is to delete the pod. Debug a few times on the same pod and the spec accumulates clutter — mildly annoying, and worth knowing before you wonder why the pod looks strange afterwards.

They have no resource requests, so they consume from the node without being accounted for. Fine occasionally; do not leave them running.

They are subject to the same policy as any container. Pod Security Standards (Chapter 23) apply, and a restricted namespace will reject a debug container asking for privileges. This is correct and occasionally inconvenient.

RBAC: creating one needs `pods/ephemeralcontainers patch`, which is separate from `pods/exec`. Granting `exec` does not grant `debug`.

`kubectl debug` in its three modes

1. Ephemeral container in an existing pod — the default, above. Non-disruptive, and what you want for a running pod.

2. Copy the pod with changes — for pods that are crashing, where there is nothing to attach to:

```
kubectl debug POD --copy-to=POD-debug --container=api -- sleep 3600
```

```
kubectl debug POD --copy-to=POD-debug --set-image=api=busybox:1.37 -it -- sh
```

This creates a **new** pod with the same volumes, environment and configuration but a different command or image. It is the cleanest answer to a `CrashLoopBackOff` where the container dies before you can look at anything (Chapter 17). Add `--share-processes` if you need a shared PID namespace in the copy.

Remember to delete the copy afterwards — it is a real pod consuming real resources.

3. A node debug pod — for the node itself:

```
kubectl debug node/k8sbook-worker -it --image=nicolaka/netshoot
```

This creates a privileged pod on that node with the **host filesystem mounted at /host** and host namespaces shared. From there:

```
chroot /host
journalctl -u kubelet --since '10 min ago'
crictl ps
df -h
dmesg -T | tail
```

This is how you investigate a node without SSH, and it is the technique Chapter 21 relies on. It also, obviously, grants host root — so it is a privileged operation that should be tightly controlled by RBAC.

Other ways in

`kubectl cp` — works both directions, and on any pod with `tar` present:

```
kubectl cp POD:/app/config.yaml ./config.yaml
kubectl cp ./fix.conf POD:/tmp/fix.conf
kubectl cp -c CONTAINER POD:/path ./local
```

The `tar` requirement is real: on a distroless image `kubectl cp` fails, and you fall back to an ephemeral container reading `/proc/1/root/`.

`kubectl port-forward` — a tunnel straight to a pod or Service, bypassing Ingress, Service routing and DNS:

```
kubectl port-forward pod/POD 8080:8000
kubectl port-forward deploy/pingd 8080:8000
kubectl port-forward svc/pingd 8080:80
```

Invaluable for localising network problems (Chapter 19): if `port-forward` works and the Service does not, the application is fine and the problem is in Service routing.

Note that `port-forward svc/...` still forwards to **one pod**, not through the Service's load balancing — it resolves the Service to a pod and connects there.

`kubectl attach` — connects to PID 1's existing stdio rather than starting a new process. Rarely what you want, but occasionally the only way to interact with an interactive process.

`kubectl proxy` — an authenticated local proxy to the API server, for exploring the API directly:

```
kubectl proxy --port=8001 &
curl -s localhost:8001/api/v1/namespaces/default/pods | head
```

A note on `kubectl exec` and security

`pods/exec` is a **privileged permission**. It bypasses everything: the container's identity, your audit trail of what the application does, and any application-level authorisation. A user with `exec` on a pod can read its Secrets from the environment, its service account token from `/var/run/secrets/...`, and use that token to act as the pod.

Treat `exec` and `debug` as production-privileged operations in your RBAC design (Chapter 23), and make sure API server audit logging captures them.

Try it

Create a pod with no shell:

```
kubectl run distro --image=gcr.io/distroless/python3-debian12:nonroot --
restart=Never --command -- python3 -c "import
http.server,socketserver;socketserver.TCPServer(('',
8080),http.server.SimpleHTTPRequestHandler).serve_forever()"
```

```
kubectl wait --for=condition=Ready pod/distro --timeout=120s
```

Confirm `exec` cannot help:

```
kubectl exec distro -- sh -c 'echo hi'
```

Now attach a toolbox sharing its process namespace:

```
kubectl debug distro --image=busybox:1.37 --target=distro -it -- sh
```

Inside, look at the target's processes and filesystem:

```
ps -o pid,comm
ls /proc/1/root/
cat /proc/1/cmdline | tr '\0' ' '; echo
```

Exit, then see that the ephemeral container is now part of the pod forever:

```
kubectl get pod distro -o jsonpath='{.spec.ephemeralContainers[*].name}{"\n"}'
```

Try the copy mode, which is what you use for a crash-looping pod:

```
kubectl debug distro --copy-to=distro-probe --container=distro -- sleep 3600
```

```
sleep 10 && kubectl get pod distro-probe
```

And a node debug pod — this gives you the host filesystem:

```
kubectl debug node/k8sbook-worker -it --image=busybox:1.37 -- sh -c
'ls /host; echo ---; cat /host/etc/hostname'
```

Clean up. **the node debugger pod is privileged — remove it:**

```
kubectl delete pod distro distro-probe --force --grace-period=0
```

```
kubectl get pods --field-selector spec.nodeName=k8sbook-worker | grep node-debugger
```

```
kubectl delete pod -l '!app' --field-selector status.phase=Running --force --grace-period=0 2>/dev/null | grep node-debugger
```

Takeaways

- `kubectl exec` needs the binary to exist in the image. Distroless images have none, by design.
- `kubectl debug --target` **attaches an ephemeral container sharing the target's process namespace** — `ps` shows the application, and `/proc/1/root/` is its filesystem.
- Without `--target`, you share network and IPC but not PID.
- **Ephemeral containers cannot be removed** and accumulate in the pod spec. They have no resource requests.
- `kubectl debug --copy-to` makes a modified copy of a pod — the answer for crash loops where there is nothing to attach to.
- `kubectl debug node/NAME` gives a privileged pod with the host at `/host`. That is host root; control it with RBAC.
- `kubectl cp` needs `tar` in the image; `port-forward` bypasses Services and DNS, which makes it a localisation tool.
- `pods/exec` and `pods/ephemeralcontainers` are privileged permissions and are separate in RBAC.

Previous: Chapter 17 — Pods that won't run · Next: Chapter 19 — Network diagnosis

Chapter 19 — Network diagnosis

"Service X cannot reach Service Y" has about eight plausible causes in Kubernetes. This chapter walks the packet from client to container and eliminates them in order.

Follow the path

```
client pod
├─ 1. DNS: does the Service name resolve?
├─ 2. Service: does it have ready endpoints?
├─ 3. Policy: is the traffic allowed?
├─ 4. kube-proxy: is the ClusterIP programmed on this node?
├─ 5. CNI: can pod reach pod?
└─ 6. Application: is it listening on the right address and port?
```

Test in that order. Each step is one command and eliminates a whole class of cause.

Step 1: does the name resolve?

```
kubectl run t --rm -it --restart=Never --image=nicolaka/netshoot -- nslookup pingd
```

Failure means DNS (Chapter 12). Check the namespace first — a bare name only resolves within the pod's own namespace, because that is the first search domain. Cross-namespace needs `pingd.othernamespace`.

If a *fresh* pod resolves it and yours does not, compare `/etc/resolv.conf`; a `hostNetwork` pod with the wrong `dnsPolicy` is the usual answer.

Step 2: does the Service have ready endpoints?

This is the most common cause and the cheapest check. Do it first in practice.

```
kubectl get endpointslice -l kubernetes.io/service-name=pingd \
-o custom-columns='NAME:.metadata.name,ADDRESSES:.endpoints[*].addresses[0],READY:.endpoints[*].conditions.ready'
```

NAME	ADDRESSES	READY
pingd-abc12	10.244.2.5,10.244.1.4,10.244.2.6	true,true,true

Three outcomes:

`<unset>` / **no addresses** — the selector matches no pods. Compare them:

```
kubectl get svc pingd -o jsonpath='{.spec.selector}{"\n"}'
```

```
kubectl get pods -l app=pingd --show-labels
```


A mismatch here is the single most common Kubernetes networking bug, and nothing warns you (Chapter 4). It usually arrives via a typo, a copied manifest, or a Deployment whose pod template labels drifted from the Service's selector.

Addresses present but `ready: false` — the pods exist and fail readiness. This is a Chapter 9 problem, not a networking one. Remember that not-ready endpoints stay *listed*, so reading only the addresses column will mislead you.

Addresses present and ready — the Service is fine; continue to step 3.

Also verify the ports line up:

```
kubectl get svc pingd -o jsonpath='{.spec.ports[0].port}
targetPort={.spec.ports[0].targetPort}{"\n"}'
```

A `targetPort` that does not match the container's actual listening port gives you a healthy Service with ready endpoints that refuses every connection.

Step 3: is a NetworkPolicy blocking it?

```
kubectl get networkpolicy -A
```

If any policy selects the destination pod, everything not explicitly allowed is denied (Chapter 14). The specific traps to check:

- The policy uses the **Service port** rather than the **pod port**.
- An egress policy on the *client* forgot to allow DNS, so step 1 fails.
- `namespaceSelector` and `podSelector` in separate list items (OR) rather than the same item (AND) — or the reverse.

```
kubectl describe networkpolicy -n NAMESPACE POLICY
```

The fastest test is to temporarily label a debug pod so a policy permits it, and see whether that changes the outcome.

Step 4: bypass the Service

Narrow it to Service routing versus everything else. Connect straight to a pod IP:

```
POD_IP=$(kubectl get pod -l app=pingd -o jsonpath='{.items[0].status.podIP}')
```

```
kubectl run t --rm -it --restart=Never --image=curlimages/curl:8.11.1 -- curl -sS
"http://$POD_IP:8000/"
```

- **Pod IP works, ClusterIP does not** → kube-proxy or the Service definition. Check kube-proxy is running on the *client's* node, and check `targetPort`.
- **Neither works** → the CNI, a policy, or the application (steps 3, 5, 6).

Then bypass the network entirely:

```
kubectl port-forward deploy/pingd 8080:8000
```

```
curl -sS localhost:8080/
```

If that works, the application is listening correctly and every remaining suspect is in the network path.

Step 5: kube-proxy and the CNI

```
kubectl get pods -n kube-system -l k8s-app=kube-proxy -o wide
```

```
kubectl logs -n kube-system -l k8s-app=kube-proxy --tail=30
```

kube-proxy failing on **one** node produces a distinctive symptom: pods on that node cannot reach any ClusterIP, but *can* reach pod IPs directly. Everywhere else works. If your failures correlate with the client's node, this is where to look.

Inspect the rules on a node (Chapter 18's node debugger):

```
kubectl debug node/k8sbook-worker -it --image=nicolaka/netshoot -- \
  chroot /host sh -c 'iptables-save -t nat | grep pingd | head'
```

For the CNI, check its DaemonSet is healthy on every node and look for IP exhaustion:

```
kubectl get pods -n kube-system -o wide | grep -E 'calico|cilium|kindnet|flannel'
```

`failed to setup network for sandbox` in a pod's events means the CNI could not allocate an address — often the node's pod CIDR is full.

Step 6: the application

Two causes that the Docker book already covered, both of which recur here unchanged.

Bound to 127.0.0.1 inside the container. Reachable only within the pod's own network namespace — not from another pod, no matter how correct the Service is. Check from an ephemeral container sharing the pod's network:

```
kubectl debug POD --image=nicolaka/netshoot -it -- ss -tlnp
```

```
127.0.0.1:8000      ← wrong, unreachable
0.0.0.0:8000       ← right
```

`localhost` resolves to `::1`. An IPv4-only server refuses connections to `localhost:8000` while accepting `127.0.0.1:8000`. This bites hardest in **probes**, which target the local pod — a healthcheck using `localhost` fails against a working server. Use `127.0.0.1`.

Ingress-specific

Chapter 13's failure is worth repeating as a diagnostic, because it looked like a routing bug and was not:

```
curl: (56) Recv failure: Connection reset by peer
```

The Ingress was correct; the **controller pod was on a node where external traffic never arrives**. Check where it is and how traffic reaches it:

```
kubectl get pod -n ingress-nginx -l app.kubernetes.io/component=controller \
  -o custom-columns='NAME:.metadata.name,NODE:.spec.nodeName,HOSTPORT:.spec.containers[0].ports[*].hostPort'
```

Then work outward:

```
kubectl get ingressclass                                # is there a controller?
kubectl describe ingress NAME                           # did it resolve the
backend?
kubectl logs -n ingress-nginx deploy/ingress-nginx-controller --tail=50
```

A 404 from the ingress controller usually means no rule matched — check the `Host` header and `pathType` semantics. A 503 means the rule matched but the backend has no endpoints, which sends you back to step 2.

Capturing traffic

When the logic checks out and it still fails, look at the wire. An ephemeral container in the pod's network namespace:

```
kubectl debug POD --image=nicolaka/netshoot -it -- tcpdump -i any -n port 8000
```

Interpretation is the same as the Docker book:

- **Nothing** — the client never sent; it resolved somewhere else.
- **SYN, no reply** — dropped. Policy, wrong address, or nothing listening on that interface.
- **SYN → RST** — refused. Nothing listening on that port.
- **Handshake then reset** — past the network; TLS or the application rejected it.

A quick reference

Symptom	Most likely
Name does not resolve	Wrong namespace; CoreDNS; <code>hostNetwork</code> <code>dnsPolicy</code>
Resolves, connection refused	<code>targetPort</code> wrong; app on <code>127.0.0.1; ::1</code> vs <code>127.0.0.1</code>
Resolves, connection times out	NetworkPolicy; CNI; wrong node
Service has no endpoints	Selector does not match pod labels
Endpoints exist but <code>ready: false</code>	Readiness probe failing (Chapter 9)
Works from some pods only	kube-proxy on the client's node; policy by namespace
Ingress 503	Backend Service has no ready endpoints
Ingress 404	No rule matched — Host header or <code>pathType</code>
Ingress connection reset	Controller not where traffic arrives

Try it

Create the classic failure — a Service whose selector matches nothing:

```
kubectl create service clusterip broken --tcp=80:8000
```

```
kubectl get svc broken && kubectl get endpointslice -l kubernetes.io/service-name=broken
```

ENDPOINTS: `<unset>`. Now compare selector to labels, which is the diagnosis:

```
kubectl get svc broken -o jsonpath='{.spec.selector}{"\n"}'; kubectl get pods -l app=pingd --show-labels --no-headers | head -1
```

```
kubectl delete svc broken
```

Now walk the working path deliberately. DNS:

```
kubectl run t --rm -it --restart=Never --image=nicolaka/netshoot --quiet --nslookup pingd
```

Endpoints:

```
kubectl get endpointslice -l kubernetes.io/service-name=pingd -o custom-columns='ADDRESSES:.endpoints[*].addresses[0],READY:.endpoints[*].conditions.ready'
```

Pod IP directly, bypassing the Service:

```
POD_IP=$(kubectl get pod -l app=pingd -o jsonpath='{.items[0].status.podIP}') &&
kubectl run t --rm -it --restart=Never --image=curlimages/curl:8.11.1 --quiet -- cu
rl -sS "http://$POD_IP:8000/"
```

Via the Service:

```
kubectl run t --rm -it --restart=Never --image=curlimages/curl:8.11.1 --quiet --
curl -sS http://pingd/
```

And bypassing the network entirely:

```
kubectl port-forward deploy/pingd 8080:8000 & sleep 3; curl -sS localhost:8080/;
kill %1
```

Finally, check what the application is actually bound to:

```
POD=$(kubectl get pod -l app=pingd -o jsonpath='{.items[0].metadata.name}') &&
kubectl debug $POD --image=nicolaka/netshoot -it --quiet -- ss -tln
```

0.0.0.0:8000 is what you want to see.

Takeaways

- Walk the path: DNS → endpoints → policy → kube-proxy → CNI → application.
- **Check endpoints first in practice.** A selector that matches no pods is the most common cause and produces no error anywhere.
- Read `conditions.ready`, not just addresses — not-ready pods stay listed.
- A `targetPort` mismatch gives you a healthy-looking Service that refuses every connection.
- Pod IP works but ClusterIP does not → kube-proxy or the Service. Neither works → CNI, policy or the app.
- `port-forward` bypasses Service, DNS and Ingress — the fastest way to exonerate the application.
- kube-proxy broken on one node: that node's pods cannot reach any ClusterIP but can reach pod IPs.
- The application bound to `127.0.0.1`, and `localhost` meaning `::1`, are still the two most common application-side causes — especially in probes.
- Ingress 503 = no ready endpoints; 404 = no rule matched; connection reset = controller not where traffic arrives.

Previous: Chapter 18 — Getting inside · Next: Chapter 20 — Resources and eviction

Chapter 20 — Resources and eviction

Chapter 8 covered how requests and limits are declared. This chapter covers what happens when they are wrong: containers killed for memory, throttled for CPU, and pods evicted by a node under pressure.

OOMKilled

A container exceeding `limits.memory` is killed by the kernel's cgroup OOM killer.

The measurement from Chapter 8 is worth restating here because it changes the procedure. On this cluster (v1.37, containerd 2.3.4, cgroup v2), a container that was definitively OOM-killed reported:

```
{"terminated":{"exitCode":137,"reason":"Error", ...}}
```

`reason: Error`, **not** `OOMKilled` — while the node's kernel log was unambiguous:

```
Memory cgroup out of memory: Killed process 160986 (python)
```

`OOMKilled` does appear on many clusters, so treat its absence as inconclusive rather than exculpatory. The reliable procedure:

1. **Exit code 137.** Always present.

```
kubectl get pod POD -o
jsonpath='{.status.containerStatuses[0].lastState.terminated.exitCode}
reason={.status.containerStatuses[0].lastState.terminated.reason}{"\n"}'
```

2. **Was it at its limit?**

```
kubectl top pod POD --containers
```

3. **The node's kernel log** — definitive:

```
kubectl debug node/NODE -it --image=busybox:1.37 -- chroot /host dmesg -T | grep -
i 'memory cgroup out of memory' | tail
```

4. **Distinguish from an ignored SIGTERM.** 137 is SIGKILL, which is also what you get when a pod exceeds its termination grace period (Chapter 9). If the kill happened during a delete or a rollout, and memory was nowhere near the limit, it is a signal-handling problem, not memory.

Fixing it

- **Raise the limit** if the workload legitimately needs it.
- **Fix the runtime's sizing.** A JVM using `-Xmx` equal to the container limit will always eventually OOM, because heap is not the JVM's only memory. Use `-XX:MaxRAMPercentage=75`. Node needs `--max-old-space-size` explicitly. Pass the real limit in with the downward API (Chapter 7).
- **Look for a leak.** Memory climbing steadily to the limit, then a restart, then climbing again, is a leak with a restart schedule.

- **Check** `initialDelaySeconds` **on liveness** if the OOM happens at startup — some runtimes peak during initialisation.

Note that a container hitting its memory limit is killed *individually*. The pod is not rescheduled; the container restarts in place with the same pod IP.

CPU throttling

CPU limits do not kill; they throttle. The symptom is latency at low reported utilisation, and it is one of the most misdiagnosed problems in Kubernetes.

The mechanism is exactly what the Docker book measured: `limits.cpu: 500m` is a quota of 50 ms per 100 ms period. Multi-threaded processes exhaust it early and then freeze for the rest of the period.

The metric to look at:

```
container_cpu_cfs_throttled_periods_total / container_cpu_cfs_periods_total
```

Anything above a few percent on a latency-sensitive service is worth attention. `kubectl top` will **not** show this — throttling keeps average utilisation low, which is why the problem hides.

Without Prometheus, read the cgroup directly via a node debug pod:

```
kubectl debug node/NODE -it --image=busybox:1.37 -- \
  chroot /host sh -c 'find /sys/fs/cgroup/kubelet.slice -name cpu.stat | head -3 |
  xargs grep -H nr_throttled'
```

Fixes: raise the limit, reduce worker/thread counts to match the quota (do not size pools from `nproc` — it reports the *node's* cores), or remove the CPU limit and rely on requests for fair sharing.

For latency-critical workloads — the SIP load generator being the case in point — CPU limits are actively harmful. Use Guaranteed QoS with the static CPU manager policy for exclusive cores instead.

Node pressure and eviction

When a node runs short of memory, disk or PIDs, the **kubelet** evicts pods to recover. This is distinct from OOM killing: the kubelet chooses victims deliberately, and the pod is *deleted* and rescheduled elsewhere rather than restarted in place.

```
kubectl get nodes -o custom-columns='NAME:.metadata.name,MEM:.status.conditions[?
(@.type=="MemoryPressure")].status,DISK:.status.conditions[?
(@.type=="DiskPressure")].status,PID:.status.conditions[?
(@.type=="PIDPressure")].status'
```

NAME	MEM	DISK	PID
k8sbook-control-plane	False	False	False
k8sbook-worker	False	False	False
k8sbook-worker2	False	False	False

Any of these `True` means the node is shedding load.

Eviction order

The kubelet ranks candidates by:

1. **QoS class** — `BestEffort` first, then `Burstable`, then `Guaranteed`.
2. **Usage above requests** — a pod far over its request is evicted before one within it.
3. **Pod priority** — lower `priorityClassName` first.

This is the practical consequence of Chapter 8: a pod with no resources block is `BestEffort` and is evicted first, every time. Setting requests is not bureaucracy; it is survival ranking.

An evicted pod:

```
kubectl get pods --field-selector status.phase=Failed
```

```
kubectl describe pod POD | grep -A3 'Status:\|Message:'
```

The message names the resource: `The node was low on resource: memory or ephemeral-storage.`

Thresholds

The kubelet's thresholds are configuration, not universal constants. On this cluster:

```
kubectl get --raw /api/v1/nodes/k8sbook-worker/proxy/configz |
jq .kubeletconfig.evictionHard
```

```
{"imagefs.available":"0%","nodefs.available":"0%","nodefs.inodesFree":"0%"}
```

kind sets these to 0%, effectively disabling eviction — sensible for a local test cluster, and a genuine difference from production, where the defaults are around `memory.available<100Mi`, `nodefs.available<10%`, `imagefs.available<15%`. So eviction is one of the few behaviours you cannot exercise realistically on kind. Check your real clusters rather than assuming defaults.

Similarly, `capacity` and `allocatable` are identical here:

```
capacity:   cpu=22 mem=32448976Ki pods=110
allocatable: cpu=22 mem=32448976Ki pods=110
```

Production nodes reserve capacity for the kubelet and the OS (`--kube-reserved`, `--system-reserved`), so `allocatable` is meaningfully lower than `capacity`. **Always schedule against allocatable**, and be aware that a managed node pool's usable capacity is 5–15% below the instance size you are paying for.

Ephemeral storage

The most common eviction cause in practice, and the least anticipated. Container logs, writable layers and `emptyDir` volumes all consume node disk. One pod writing a large log or filling an `emptyDir` triggers `DiskPressure`, and the kubelet evicts pods — **including well-behaved ones** on the same node.

Defend with requests and limits (Chapter 8), `sizeLimit` on `emptyDir`, and log rotation.

Preemption

Distinct from eviction. When a high-priority pod cannot schedule, the scheduler may **evict lower-priority pods** to make room.

```
kubectl get priorityclass
```

```
system-cluster-critical    20000000000
system-node-critical       2000001000
```

Those two ship with every cluster and are reserved for control-plane components. Define your own for workload tiers:

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000
globalDefault: false
preemptionPolicy: PreemptLowerPriority
description: "Customer-facing services"
```

Use `preemptionPolicy: Never` for pods that should get scheduling *preference* without displacing anything.

Preemption shows in the scheduler's message, as seen in Chapter 17:

```
preemption: 0/3 nodes are available: 3 Preemption is not helpful for scheduling.
```

That line means the scheduler considered preemption and concluded it would not help — usually because the pending pod's request is larger than anything it could free.

A caution: priority is cluster-wide and comparative. Once teams start assigning priorities, they all assign high ones. Govern priority classes centrally, or the mechanism stops discriminating.

Disruption budgets

Eviction also happens *voluntarily* — a node drain for maintenance or an autoscaler scale-down. A `PodDisruptionBudget` limits how much of a workload may be voluntarily disrupted at once:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: pingd
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: pingd
```

`kubectl drain` respects this and blocks rather than taking your service below two replicas. Note the asymmetry: **PDBs do not protect against involuntary disruption** — node failure, kernel OOM, or kubelet eviction under pressure ignore them entirely. Chapter 25 covers them properly.

A PDB with `minAvailable` equal to the replica count blocks drains **forever**, which turns a routine node upgrade into an incident. `maxUnavailable: 1` is usually safer.

Try it

Look at the pressure conditions and allocatable capacity:

```
kubectl get nodes -o custom-columns='NAME:.metadata.name,MEM:.status.conditions[?(@.type=="MemoryPressure")].status,DISK:.status.conditions[?(@.type=="DiskPressure")].status'
```

```
kubectl get node k8sbook-worker -o jsonpath='{.status.capacity.cpu}/{.status.capacity.memory}{"\n"}allocatable={.status.allocatable.cpu}/{.status.allocatable.memory}{"\n"}'
```

Check your kubelet's real eviction thresholds rather than assuming:

```
kubectl get --raw /api/v1/nodes/k8sbook-worker/proxy/configz | python3 -c "import json,sys; print(json.load(sys.stdin)['kubeletconfig'].get('evictionHard'))"
```

Trigger an OOM kill and read every signal:

```
kubectl apply -f examples/manifests/08-oom.yaml && sleep 40
```

```
kubectl get pod oom3 -o jsonpath='{.status.containerStatuses[0].lastState.terminated.exitCode}reason={.status.containerStatuses[0].lastState.terminated.reason}restarts={.status.containerStatuses[0].restartCount}{"\n"}'
```

Then confirm from the node that it really was memory:

```
kubectl debug node/${(kubectl get pod oom3 -o jsonpath='{.spec.nodeName}')} -it --image=busybox:1.37 --quiet -- chroot /host sh -c "dmesg -T | grep -i 'memory cgroup out of memory' | tail -2"
```

```
kubectl delete -f examples/manifests/08-oom.yaml --force --grace-period=0
```

Create a PriorityClass and see it applied:

```
kubectl apply -f examples/manifests/20-priorityclass.yaml && kubectl get priorityclass high-priority
```

```
kubectl delete -f examples/manifests/20-priorityclass.yaml
```

Clean up the node debugger pods this created:

```
kubectl get pods -o name | grep node-debugger | xargs -r kubectl delete --force --
grace-period=0
```

Takeaways

- **Exit 137 is the reliable OOM signal**, not `reason: OOMKilled` — measured `Error` here. Confirm with the node's `Memory cgroup out of memory` log.
- 137 during a delete or rollout, with memory nowhere near the limit, is an ignored `SIGTERM` instead.
- CPU throttling causes latency at *low* reported utilisation. `kubectl top` cannot show it; use `container_cpu_cfs_throttled_periods_total` or read `cpu.stat` on the node.
- Eviction ranks by **QoS class, then usage above requests, then priority**. `BestEffort` pods go first — which is what "forgetting to set resources" actually costs.
- **kind disables eviction** (`evictionHard: 0%`) and reserves nothing, so allocatable equals capacity. Production differs on both counts; check your real clusters.
- Ephemeral storage is the most common eviction trigger, and it evicts innocent neighbours.
- Preemption is the scheduler displacing lower-priority pods; eviction is the kubelet shedding load. Different mechanisms, different fixes.
- PDBs constrain **voluntary** disruption only, and a too-strict PDB blocks node drains forever.

Previous: Chapter 19 — Network diagnosis · Next: Chapter 21 — Nodes and the control plane

Chapter 21 — Nodes and the control plane

Layer 5. When the platform itself is the problem, application-level debugging goes in circles — every service looks broken and none of them is at fault.

Node conditions

```
kubectl get nodes
```

```
kubectl get node NODE -o jsonpath='{range .status.conditions[*]}{.type}={.status}{.reason}}{"\n"}{end}'
```

```
MemoryPressure=False (KubeletHasSufficientMemory)
DiskPressure=False (KubeletHasNoDiskPressure)
PIDPressure=False (KubeletHasSufficientPID)
Ready=True (KubeletReady)
```

`Ready=True` means the kubelet has reported in recently *and* considers itself healthy. The three pressure conditions are Chapter 20's subject.

NotReady

The node stopped heartbeating, or the kubelet reported itself unhealthy. What follows is a timeline worth knowing precisely, because it determines how long you have:

```
t+0s      kubelet stops reporting
t+~40s    node-monitor-grace-period expires → node marked NotReady
          → node controller adds a NoExecute taint
t+~5m     tolerationSeconds expires → pods marked for deletion, rescheduled
          elsewhere
```

Between t+40s and t+5m, the containers are still running on the unreachable node. Nothing stopped them. The kubelet is gone, so nothing manages them, but the processes are alive and may still be writing to storage and serving traffic if the network partition is partial.

This is why `kubectl delete pod --force` on a `NotReady` node is dangerous for stateful workloads (Chapter 17): you are telling the API server the pod is gone so a replacement can start, while the original may still be running with the same volume attached.

Common causes, in rough order:

- The node lost network connectivity to the API server.
- The kubelet crashed or was OOM-killed.
- The node ran out of disk and the kubelet cannot function.
- The container runtime (containerd) died.
- The VM was terminated by the cloud provider.

Investigate with a node debug pod (Chapter 18), since SSH may not be available:

```
kubectl debug node/NODE -it --image=nicolaka/netshoot -- chroot /host bash
```

```
systemctl status kubelet
journalctl -u kubelet --since '15 min ago' | tail -50
crictl ps
df -h
free -m
dmesg -T | tail -30
```

The kubelet's journal is usually explicit about why it is unhappy.

Taints and node lifecycle

```
kubectl get nodes -o custom-
columns='NAME:.metadata.name,TAINTS:.spec.taints[*].key'
```

```
k8sbook-control-plane    node-role.kubernetes.io/control-plane
k8sbook-worker           <none>
k8sbook-worker2          <none>
```

The control-plane taint is why your workloads land on workers without asking.

Taints applied automatically by the node controller are worth recognising, because they explain sudden mass rescheduling:

Taint	When
node.kubernetes.io/not-ready	Node <code>NotReady</code>
node.kubernetes.io/unreachable	Node controller cannot reach it
node.kubernetes.io/memory-pressure	Memory pressure
node.kubernetes.io/disk-pressure	Disk pressure
node.kubernetes.io/unschedulable	Cordoned

The first two are `NoExecute`, which is what triggers the 5-minute eviction above. Every pod gets a default 300-second toleration for them; lower it for workloads that must fail over faster, at the cost of more churn during transient blips:

```
tolerations:
- key: node.kubernetes.io/unreachable
  operator: Exists
  effect: NoExecute
  tolerationSeconds: 60
```

Cordon and drain

```
kubectl cordon NODE
```

```
k8sbook-worker2    Ready,SchedulingDisabled
```

Cordoning stops *new* pods scheduling there; existing pods keep running. It is the safe first step before maintenance.

```
kubectl drain NODE --ignore-daemonsets --delete-emptydir-data
```

Drain cordons **and** evicts. The flags are almost always required: DaemonSet pods cannot be rescheduled elsewhere so drain refuses without `--ignore-daemonsets`, and anything with an `emptyDir` needs explicit acknowledgement that its data will be lost.

Drain uses the **Eviction API**, so it respects PodDisruptionBudgets — a drain that hangs is usually a PDB doing its job (Chapter 25). Check:

```
kubectl get pdb -A
```

```
kubectl uncordon NODE
```

Control plane health

```
kubectl get --raw='/livez'
```

```
ok
```

```
kubectl get --raw='/readyz?verbose'
```

```
[+]ping ok
[+]log ok
[+]etcd ok
[+]etcd-readiness ok
[+]informer-sync ok
...
```

The `?verbose` form lists every check individually, which is the fastest way to see *which* subsystem is unhealthy. `[-]etcd failed` is a very different problem from a failing admission webhook.

On self-managed clusters, the components are static pods (Chapter 3):

```
kubectl get pods -n kube-system -l tier=control-plane
```

```
kubectl logs -n kube-system kube-apiserver-NODE --tail=100
```

And when the API server is down entirely, you cannot use `kubectl` at all — go to the node:

```
crictl ps -a | grep apiserver
crictl logs CONTAINER_ID
cat /etc/kubernetes/manifests/kube-apiserver.yaml
```

The kubelet watches that directory, so **fixing the manifest file fixes the API server**. That is the bootstrap escape hatch, and it is the reason static pods exist.

etcd

The only stateful component, and the one whose failure is unrecoverable without a backup.

```
kubectl -n kube-system exec etcd-k8sbook-control-plane -- etcdctl \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
endpoint health
```

Back it up:

```
etcdctl snapshot save /backup/etcd-$(date +%F).db
```

Things that actually go wrong with etcd:

Slow disk. etcd fsyncs every write. On slow storage, `etcd_disk_wal_fsync_duration_seconds` climbs and *everything* in the cluster gets slower — API calls, scheduling, controller reactions. The symptom looks like a broken cluster; the cause is a disk. Watch this metric.

Database size. The default quota is 2 GiB. Exceeding it puts etcd into a maintenance mode where it **rejects all writes** — the cluster becomes read-only, which is a spectacular failure. Causes: many large objects (Secrets, ConfigMaps), or an unbounded controller creating objects. Defragment and raise the quota if needed.

Quorum loss. With 3 members you can lose 1. Lose 2 and the cluster stops accepting writes until quorum is restored. Odd member counts, spread across failure domains.

On managed clusters (EKS/GKE/AKS) none of this is yours — which is a substantial part of what you are paying for.

Certificates

Kubernetes uses TLS everywhere, and kubeadm-issued client certificates expire after **one year**. An expired certificate presents as sudden, total authentication failure:

```
Unable to connect to the server: x509: certificate has expired or is not yet valid
```

```
kubeadm certs check-expiration
```

```
kubeadm certs renew all
```

Kubelet client certificates usually auto-rotate; the control plane's do not unless you upgrade regularly (an upgrade renews them as a side effect). A cluster that has run untouched for thirteen months is a cluster about to fail, and this is a genuinely common cause of "the cluster died and nothing changed".

Version skew

```
kubectl get nodes -o jsonpath='{range .items[*]}{.metadata.name}{ " kubelet="}
{.status.nodeInfo.kubeletVersion}{ " runtime="}
{.status.nodeInfo.containerRuntimeVersion}{ "\n"}{end}'
```

```
k8sbook-control-plane kubelet=v1.37.0 runtime=containerd://2.3.4
k8sbook-worker        kubelet=v1.37.0 runtime=containerd://2.3.4
k8sbook-worker2       kubelet=v1.37.0 runtime=containerd://2.3.4
```

The supported skew: kubelet may be up to **3 minor versions** behind the API server, never ahead. `kubectl` may be one minor version either side.

Upgrade order is control plane first, then nodes, one minor version at a time. Skipping minor versions is unsupported and will bite you on API removals.

Which brings up the thing that actually breaks upgrades: **removed APIs**. Kubernetes removes deprecated API versions on a schedule, and manifests using them fail to apply after an upgrade. Check before upgrading:

```
kubectl get --raw /metrics | grep apiserver_requested_deprecated_apis
```

Tools like `pluto` and `kubent` scan manifests and live clusters for deprecated APIs. Run one before every upgrade.

Namespace stuck Terminating

A namespace that will not delete is almost always a **finalizer** on some object inside it, or an unavailable APIService blocking enumeration.

```
kubectl get namespace NS -o jsonpath='{.status.conditions}' | python3 -m json.tool
```

The conditions name the blocking resource explicitly. Check for broken aggregated APIs, which is a common cause:

```
kubectl get apiservice | grep -v True
```

An unavailable APIService — often a metrics or webhook service whose backing pods are gone — blocks namespace deletion cluster-wide.

The `kubectl patch` to strip finalizers is widely circulated and **should be a last resort**: it deletes the namespace object while leaving the resources it contained orphaned, and any external resources those finalizers were meant to clean up (cloud load balancers, volumes) leak silently.

Try it

Check control plane health in detail:

```
kubectl get --raw='/readyz?verbose' | head -12
```



```
kubectl get --raw='/livez'
```

Look at node conditions and taints:

```
kubectl get nodes -o custom-  
columns='NAME:.metadata.name,TAINTS:.spec.taints[*].key,READY:.status.conditions[?  
(@.type=="Ready")].status'
```

Cordon a node and watch the status change, then reverse it:

```
kubectl cordon k8sbook-worker2 && kubectl get node k8sbook-worker2
```

```
k8sbook-worker2    Ready,SchedulingDisabled
```

```
kubectl uncordon k8sbook-worker2 && kubectl get node k8sbook-worker2
```

Try a drain, and see it move pods:

```
kubectl drain k8sbook-worker2 --ignore-daemonsets --delete-emptydir-data --  
timeout=120s
```

```
kubectl get pods -l app=pingd -o wide
```

Everything has moved off worker2. Bring it back:

```
kubectl uncordon k8sbook-worker2
```

Note the pods do **not** move back — the scheduler does not rebalance (Chapter 3). Force redistribution with a restart:

```
kubectl rollout restart deployment/pingd && kubectl rollout status deployment/pingd
```

```
kubectl get pods -l app=pingd -o wide
```

Get onto a node without SSH:

```
kubectl debug node/k8sbook-worker -it --image=busybox:1.37 --quiet -- chroot /host  
sh -c 'uname -a; df -h /var/lib | tail -1'
```

Check version skew and deprecated API usage:

```
kubectl get nodes -o jsonpath='{range .items[*]}{.metadata.name}{ " "  
{.status.nodeInfo.kubeletVersion}{ "\n"}{end}'
```

```
kubectl get --raw /metrics | grep -c apiserver_requested_deprecated_apis || echo  
"none requested"
```

Clean up node debugger pods:

```
kubectl get pods -o name | grep node-debugger | xargs -r kubectl delete --force --
grace-period=0
```

Takeaways

- `NotReady` at ~40 s, pods rescheduled at ~5 min. **In between, the containers are still running** on the unreachable node — which is why force-deleting stateful pods is dangerous.
- The node controller's `NoExecute` taints explain sudden mass rescheduling. Every pod tolerates them for 300 s by default.
- `cordon` stops new scheduling; `drain` also evicts and respects PDBs. A hanging drain is usually a PDB.
- **Pods do not move back after uncordon** — the scheduler never rebalances. Use `rollout restart`.
- `/readyz?verbose` names the failing control-plane subsystem.
- Static pod manifests on disk are the escape hatch when the API server is down.
- `etcd` fails in three ways: **slow disk** (cluster-wide latency), **2 GiB quota exceeded** (all writes rejected), and quorum loss. Back it up.
- **kubeadm certificates expire after one year** and present as total auth failure.
- Kubelets may be 3 minor versions behind the API server, never ahead. Scan for removed APIs before upgrading.
- A stuck `Terminating` namespace is a finalizer or a broken `APIService`. Stripping finalizers leaks real resources.

Previous: Chapter 20 — Resources and eviction · Next: Chapter 22 — Cookbook

Chapter 22 — Cookbook: symptom → cause → fix

The reference chapter. Grouped by what you actually see, most likely cause first.

Pods

Pending, **no node assigned**

```
kubectl describe pod POD | grep -A5 Events
```

FailedScheduling messages are arithmetic — read them literally:

Message fragment	Cause	Fix
Insufficient cpu / memory	Requests exceed free allocatable (not usage)	Lower requests, add nodes
untolerated taint	Node tainted	Add toleration, or target other nodes
didn't match node affinity/selector	Labels do not exist	<code>kubectl get nodes --show-labels</code>
unbound immediate PersistentVolumeClaims	PVC will not bind	Ch 15
didn't match pod topology spread constraints	Spread rules unsatisfiable	Ch 25

No events at all → the scheduler is down, or `schedulerName` names something nonexistent.

A request larger than any single node's allocatable can never schedule. Check that first — it is a manifest bug, not a capacity problem.

Pending **with a node assigned**

Not a scheduler problem. The kubelet is stuck on image pull, volume mount, or init containers. `describe` says which. Stuck >2 minutes with no event progress → node problem (Ch 21).

ImagePullBackOff / ErrImagePull

```
kubectl describe pod POD | grep -A10 Events
```

Message	Fix
not found / manifest unknown	Typo or missing tag. Verify with <code>docker manifest inspect</code>
pull access denied / authentication required	<code>imagePullSecrets</code> — must be in the pod's namespace
toomanyrequests	Authenticate to Docker Hub, or mirror
no matching manifest for linux/amd64	Wrong architecture

Local image on kind/minikube not being used? `imagePullPolicy` defaults to `Always` for `:latest`. Load the image *and* set `imagePullPolicy: IfNotPresent`.

```
kind load docker-image IMAGE:TAG --name CLUSTER
```

CreateContainerConfigError

```
kubectl get pod POD -o
jsonpath='{.status.containerStatuses[0].state.waiting.message}'{"\n"}
```

```
configmap "nope-missing" not found
```

Missing ConfigMap/Secret, or a missing key within one. Note it fails at container creation, not at `apply`. Use `optional: true` if genuinely optional.

CrashLoopBackOff — or Error with a climbing restart count

Measured on v1.37, the STATUS column showed Error, never CrashLoopBackOff. Do not rely on the string. The reliable signals:

```
kubectl get pod POD -o
jsonpath='{restarts={.status.containerStatuses[0].restartCount}
lastExit={.status.containerStatuses[0].lastState.terminated.exitCode}}{"\n"}
```

A climbing restart count with widening gaps (≈10s, 20s, 40s, capped at 5 min) is the backoff.

Find the error — **try both**, `--previous` can fail with "unable to retrieve container logs":

```
kubectl logs POD --previous; kubectl logs POD
```

Exit codes: 1 app error · 126 not executable · 127 not found · 137 SIGKILL (OOM or ignored SIGTERM) · 143 SIGTERM.

Both logs empty → hold the environment open:

```
kubectl debug POD --copy-to=probe --container=CONTAINER -- sleep 3600
```

Causes: missing required env var; unreachable dependency at startup; **liveness probe failing** (events say `Unhealthy`); wrong command after a base-image change; permission denied on a volume (`fsGroup`); OOM at startup.

Running **but 0/1 READY**

Readiness failing. Not a networking problem.

```
kubectl describe pod POD | grep -A5 Events    # look for Unhealthy
```

Check the probe uses `127.0.0.1`, **not** `localhost` (which is `::1`), a path that exists, and `timeoutSeconds` above the default 1 s.

Terminating **forever**

Cause	Check
Long grace period + ignored SIGTERM	Always exactly <code>terminationGracePeriodSeconds</code> ? PID 1 issue
Finalizer	<code>kubectl get pod POD -o jsonpath='{.metadata.finalizers}'</code>
Node gone	<code>kubectl get nodes</code>

```
kubectl delete pod POD --force --grace-period=0
```

For StatefulSet pods on an unreachable node this risks two writers on one volume.

Evicted

```
kubectl describe pod POD | grep -i message
```

The node was low on resource: `ephemeral-storage` (most common) or `memory`. BestEffort pods go first — **set requests** (Ch 8, 20).

Exit 137

```
kubectl get pod POD -o
jsonpath='{.status.containerStatuses[0].lastState.terminated}{"\n"}'
```

`reason: OOMKilled` confirms memory — but its **absence does not rule it out** (measured `Error` here). Confirm on the node:

```
kubectl debug node/NODE -it --image=busybox:1.37 -- chroot /host dmesg -T | grep -i 'memory cgroup out of memory'
```

137 during a delete/rollout with memory well below the limit → ignored SIGTERM instead.

Networking

Service unreachable — check this first

```
kubectl get endpointslice -l kubernetes.io/service-name=SVC \
-o custom-columns='ADDRESSES:.endpoints[*].addresses[0],READY:.endpoints[*].conditions.ready'
```

Result	Cause
<unset> / empty	Selector does not match pod labels — the most common bug
Addresses, ready: false	Readiness failing (Ch 9)
Addresses, ready	Service is fine — look elsewhere

```
kubectl get svc SVC -o jsonpath='{.spec.selector}{"\n"}'; kubectl get pods --show-labels
```

Also verify `targetPort` matches the container's real port — a mismatch gives a healthy Service that refuses everything.

Name does not resolve

- Bare names only resolve **in the pod's own namespace**. Use `svc.namespace`.
- `hostNetwork: true` needs `dnsPolicy: ClusterFirstWithHostNet`, or no Service resolves.
- CoreDNS down: `kubectl get endpointslice -n kube-system -l kubernetes.io/service-name=kube-dns`

External DNS is slow

`ndots:5` costs 4 queries per external name (3 NXDOMAIN). Fix with a trailing dot (`api.example.com.`), `ndots: 2` via `dnsConfig`, or `NodeLocal DNSCache`. (Ch 12)

Intermittent ~5-second DNS failures

The A/AAAA conntrack race. Use `single-request-reopen`, `NodeLocal DNSCache`, or a different CNI.

Connection refused

App bound to `127.0.0.1` inside the container, or `localhost` → `::1` against an IPv4-only listener.

```
kubectl debug POD --image=nicolaka/netshoot -it -- ss -tln
```

Want `0.0.0.0:PORT`, not `127.0.0.1:PORT`.

Connection times out

NetworkPolicy (Ch 14), CNI, or wrong address.

```
kubectl get networkpolicy -A
```

Policy traps: uses the **Service** port instead of the **pod** port; egress policy forgot DNS; namespaceSelector+podSelector in separate list items (OR) instead of the same item (AND).

Verify your CNI enforces policy at all — Flannel does not.

Works from some pods, not others

kube-proxy broken on the client's node → that node cannot reach any ClusterIP but *can* reach pod IPs.

```
kubectl get pods -n kube-system -l k8s-app=kube-proxy -o wide
```

Ingress 404 / 503 / connection reset

Symptom	Cause
404	No rule matched — <code>Host</code> header, or <code>pathType: Prefix</code> is segment-based
503	Backend Service has no ready endpoints
Connection reset	Controller pod is not where external traffic arrives
Nothing happens at all	No controller — <code>kubectl get ingressclass</code>

A misspelled ingress annotation is **silently ignored**. No error, no event.

Configuration

ConfigMap change had no effect

Env vars freeze at container start. Volume-mounted files update in ~1 minute; `subPath` **mounts never update**.

```
kubectl rollout restart deployment/NAME
```

Better: a config checksum annotation on the pod template (Ch 7).

Secret value is wrong

```
kubectl get secret NAME -o jsonpath='{.data.KEY}' | base64 -d
```

`stringData` takes plaintext; `data` requires you to encode. Mixing them up is common.

Storage

PVC Pending

With `WaitForFirstConsumer` this is **normal** until a pod uses it.

```
kubectl describe pvc NAME | grep -A3 Events
```

Otherwise: no default `StorageClass`, an access mode the backend cannot provide (RWX on block storage), or `storageClassName: ""` (explicitly no class → static binding only).

Pod stuck `ContainerCreating` with a volume error

`Multi-Attach error` → an RWO volume still attached to another node, typically after an ungraceful node failure. Wait for the attach to time out, or detach manually.

Data disappeared after deleting a PVC

`reclaimPolicy: Delete` on most default `StorageClasses` destroys the volume. Use a `Retain` class for anything that matters — and have backups.

Cluster

Node NotReady

```
kubectl debug node/NODE -it --image=nicolaka/netshoot -- chroot /host bash
systemctl status kubelet; journalctl -u kubelet --since '15 min ago' | tail -50;
df -h
```

Timeline: `NotReady` at ~40 s, pods rescheduled at ~5 min. **In between, containers keep running on the node.**

kubectl hangs or auth fails

```
kubectl get --raw='/readyz?verbose'
```

```
Unable to connect: x509: certificate has expired
```

→ `kubeadm certs check-expiration` / `kubeadm certs renew all`. `kubeadm` certs expire after **one year**.

Wrong context is the other classic: `kubectl config current-context`.

Everything is slow

`etcd` disk latency. Watch `etcd_disk_wal_fsync_duration_seconds`. A slow disk presents as cluster-wide API slowness with no obvious culprit.

All writes rejected

`etcd` exceeded its 2 GiB quota and entered maintenance mode. Defragment, raise the quota, and find what is creating objects.

Drain hangs

A `PodDisruptionBudget` is blocking it — which is the PDB working.

```
kubectl get pdb -A
```

A PDB with `minAvailable` equal to replica count blocks drains **forever**.

Pods did not move back after uncordon

They never do. **The scheduler does not rebalance.**

```
kubectl rollout restart deployment/NAME
```

Namespace stuck Terminating

```
kubectl get namespace NS -o jsonpath='{.status.conditions}' | python3 -m json.tool
kubectl get apiservice | grep -v True
```

A broken `APIService` blocks namespace deletion cluster-wide. **Stripping finalizers leaks real resources** — last resort.

Upgrade broke manifests

Removed API versions.

```
kubectl get --raw /metrics | grep apiserver_requested_deprecated_apis
```

Run `pluto` or `kubent` before every upgrade. Never skip minor versions.

Deployments

Rollout stuck

```
kubectl rollout status deployment/NAME --timeout=60s
kubectl get pods -l app=NAME
```

New pods not becoming ready → readiness probe or a broken image. **This is readiness doing its job** — with `maxUnavailable: 0` the old pods keep serving.

`progressDeadlineSeconds` stops the rollout but does **not** roll back automatically.

```
kubectl rollout undo deployment/NAME
```

Change had no effect

```
kubectl get deployment NAME -o jsonpath='{gen={.metadata.generation}
observed={.status.observedGeneration}}{"\n"}'
```

Differ → no controller processed it. Equal → it was applied; check you edited the right object, namespace, or cluster.

`replicas` keeps changing back

An HPA (or GitOps controller) owns the field. **Remove** `replicas` **from your manifest**.

Cannot change the selector

```
spec.selector: Invalid value: ...: field is immutable
```

Delete and recreate the Deployment. Never put a version in a selector.

Pods created and destroyed continuously

Two Deployments with **overlapping selectors** adopting each other's pods. Selectors must be disjoint.

Takeaways

- Recurring root causes across this whole chapter are few: **selector mismatch**, missing resource requests, probes using `localhost`, env-var config that never refreshes, `imagePullPolicy: Always` on `:latest`, and requests-vs-usage confusion.
 - Two status strings are unreliable on current versions: `CrashLoopBackOff` (may show `Error`) and `OOMKilled` (may show `Error`). Use restart counts and exit code 137.
 - When nothing here matches, go back to Chapter 16 and localise the layer.
-

Previous: Chapter 21 — Nodes and the control plane · Next: Chapter 23 — Security

Chapter 23 — Security

Everything the Docker book said about container hardening still applies — non-root, dropped capabilities, read-only root filesystems, no privileged containers. Kubernetes adds three things on top: an identity system (ServiceAccounts), an authorisation system (RBAC), and a mechanism to enforce pod-level hardening cluster-wide (Pod Security Standards).

The defaults

Two defaults are good and one is not.

RBAC's default is genuinely restrictive. What can the default ServiceAccount do?

```
kubectl auth can-i --list --as=system:serviceaccount:default:default
```

Resources	Verbs
selfsubjectreviews.authentication.k8s.io	[create]
selfsubjectaccessreviews.authorization.k8s.io	[create]
clustertrustbundles.certificates.k8s.io	[get list watch]

Essentially nothing — it can ask about its own permissions and read public trust bundles.

```
kubectl auth can-i get secrets --as=system:serviceaccount:default:default
```

```
no
```

That is the right default, and it means an application that never talks to the API server is not exposed by RBAC at all.

But the token is mounted anyway. Measured on a `pingd` pod:

```
kubectl exec POD -- ls /var/run/secrets/kubernetes.io/serviceaccount/
```

```
ca.crt namespace token
```

Every pod gets a credential it almost certainly does not need. An attacker with code execution gets a valid API token for free. Turn it off unless the workload uses the API:

```
spec:
  automountServiceAccountToken: false
```

Or on the ServiceAccount, which covers everything using it:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: pingd
automountServiceAccountToken: false
```

This is the cheapest security improvement available in Kubernetes, and it is almost never set.

ServiceAccounts

Every pod runs as a ServiceAccount — `default` in its namespace if you do not specify one. Give each workload its own:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: pingd
automountServiceAccountToken: false
---
apiVersion: apps/v1
kind: Deployment
spec:
  template:
    spec:
      serviceAccountName: pingd
```

Two reasons beyond least privilege: the audit log shows *which workload* made a call, and you can bind cloud IAM to a specific ServiceAccount (IRSA on EKS, Workload Identity on GKE) rather than to the node — which is what stops every pod on a node sharing the node's cloud permissions.

Modern tokens are **short-lived and audience-bound**, projected into the pod and rotated automatically. The long-lived `Secret`-based tokens of older Kubernetes are no longer created by default; if you find one, treat it as a credential to rotate.

RBAC

Four object types, and the pattern is regular:

- **Role** — permissions **within one namespace**.
- **ClusterRole** — permissions cluster-wide, or on cluster-scoped resources.
- **RoleBinding** — grants a Role *or* a ClusterRole, **within one namespace**.
- **ClusterRoleBinding** — grants a ClusterRole cluster-wide.

```

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: pingd-reader
rules:
  - apiGroups: [""]
    resources: ["configmaps"]
    resourceNames: ["pingd-config"]      # narrow to one object
    verbs: ["get", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  namespace: default
  name: pingd-reader
subjects:
  - kind: ServiceAccount
    name: pingd
    namespace: default
roleRef:
  kind: Role
  name: pingd-reader
  apiGroup: rbac.authorization.k8s.io

```

The combination worth knowing: a **RoleBinding** referencing a **ClusterRole** grants that **ClusterRole's permissions only inside the binding's namespace**. That is how you reuse the built-in `view/edit/admin` ClusterRoles per namespace instead of writing your own:

```
kubectl create rolebinding team-a-edit --clusterrole=edit --group=team-a -n team-a
```

RBAC is **purely additive** — there are no deny rules. Permissions are the union of every binding that applies. You cannot subtract, only avoid granting.

The permissions that are really cluster-admin

Several innocuous-looking grants are equivalent to full control. Know them before you write a Role:

- `secrets`: `list` in a namespace reads *every* secret in it. `get` with `resourceNames` is narrower; `list` is not restrictable by name.
- `pods/exec`, `pods/attach`, `pods/portforward`, `pods/ephemeralcontainers` — run code inside any pod, therefore assume its identity and read its secrets. Separate permissions; granting `exec` does not grant `debug`.
- `create pods` — create a pod with `hostPath: /`, or with another ServiceAccount, and you own the node. Pod-creating permission is node-level power unless constrained by Pod Security Standards.
- `escalate / bind` — create roles more powerful than your own. Never grant these.
- `impersonate` — become anyone.
- `nodes/proxy` — reach the kubelet API directly, bypassing the API server's authorisation.

Audit what you have:

```
kubectl get clusterrolebindings -o json | python3 -c "
import json,sys
for b in json.load(sys.stdin)['items']:
    if b['roleRef']['name'] == 'cluster-admin':
        print(b['metadata']['name'], '->', [s.get('name') for s in
b.get('subjects') or []])
"
```

```
kubectl auth can-i --list --as=system:serviceaccount:NS:SA
```

```
kubectl auth can-i create pods --as=jane@example.com -n production
```

`kubectl auth can-i` is the fastest way to answer "can this thing do that", and it works for any subject.

Pod Security Standards

The successor to PodSecurityPolicy (removed in v1.25). Three profiles, enforced by a built-in admission controller, applied with **namespace labels**:

```
kubectl label namespace prod pod-security.kubernetes.io/enforce=restricted
kubectl label namespace prod pod-security.kubernetes.io/warn=restricted
kubectl label namespace prod pod-security.kubernetes.io/audit=restricted
```

- `privileged` — no restrictions.
- `baseline` — blocks the obviously dangerous: privileged containers, host namespaces, most `hostPath`, adding capabilities beyond a small set.
- `restricted` — the hardened profile: non-root, no privilege escalation, all capabilities dropped, `seccomp RuntimeDefault`, restricted volume types.

The three modes matter: `warn` and `audit` let you see what *would* break before you `enforce`. Roll out with `warn` first, always.

Measured — creating a default `busybox` pod in a `restricted` namespace:

```
Error from server (Forbidden): pods "bad" is forbidden:
violates PodSecurity "restricted:latest":
  allowPrivilegeEscalation != false
  unrestricted capabilities (must set securityContext.capabilities.drop=["ALL"])
  runAsNonRoot != true
  seccompProfile (must set securityContext.seccompProfile.type to "RuntimeDefault")
```

Four itemised violations with the exact field for each. This is unusually good error reporting, and it is effectively a checklist for writing a compliant pod.

A compliant pod — which is what this book's `pingd` Deployment already is:

```
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    fsGroup: 10001
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: api
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
```

Important limitation: PSS applies to *Pods*, and most pods are created by controllers. A Deployment whose template violates the policy is **accepted** — measured:

```
deployment.apps/baddep      0/1   0   0
replicaset.apps/baddep-...  1     0   0
```

The Deployment exists with zero pods, and the real error is in the **ReplicaSet's** events:

```
Warning FailedCreate replicaset-controller
Error creating: pods "baddep-..." is forbidden: violates PodSecurity
"restricted:latest": ...
```

Helpfully, `enforce` alone also emits a `Warning`: on `stderr` when you create the Deployment, so you are not entirely without feedback. Setting `warn` explicitly is still worth doing — it covers dry-runs and clients that discard warnings — but the common claim that you get *no* signal until the ReplicaSet fails is not quite right on current versions.

For anything PSS cannot express — required labels, allowed registries, mandatory resource limits — you need a policy engine (Chapter 24).

securityContext

The Docker book's hardening list, in Kubernetes form:


```
spec:
  securityContext:                # pod level
    runAsNonRoot: true
    runAsUser: 10001
    runAsGroup: 10001
    fsGroup: 10001                # group ownership of mounted volumes
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: api
      securityContext:            # container level, overrides pod level
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        privileged: false
        capabilities:
          drop: ["ALL"]
```

Notes that save time:

- `runAsNonRoot: true` **requires a numeric UID in the image**, because the kubelet cannot resolve a username to check it. This is why the Docker book insisted on `USER 10001`.
- `fsGroup` sets group ownership on mounted volumes — the fix for "permission denied" on a PVC when running non-root.
- `readOnlyRootFilesystem` needs writable paths supplied as `emptyDir` volumes (Chapter 15).
- Container-level settings override pod-level; some fields exist at only one level.

The four layers of a secure cluster

1. **Authentication** — who you are. Certificates, OIDC, cloud IAM. Kubernetes has no user objects; identity comes from outside.
2. **Authorization** — RBAC. Least privilege, per workload.
3. **Admission** — what may be created. PSS plus a policy engine (Chapter 24).
4. **Runtime** — what a running container may do. `securityContext`, `NetworkPolicy` (Chapter 14), and the kernel-level isolation from the Docker book.

Weakness at any layer undermines the rest. `create pods` without admission control is cluster-admin; admission control without RBAC is bypassable by editing the policy.

Audit logging

```
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
  - level: RequestResponse
    resources:
      - group: ""
        resources: ["secrets"]
  - level: Metadata
    resources:
      - group: ""
        resources: ["pods/exec", "pods/attach", "pods/portforward"]
```

Log secret access and every `exec` at minimum. Without audit logging you cannot answer "who read that" or "who ran what in production", which is precisely the question after an incident. Managed clusters expose this through their cloud logging; enable it before you need it.

Try it

Check the default ServiceAccount's permissions:

```
kubectl auth can-i --list --as=system:serviceaccount:default:default
```

```
kubectl auth can-i get secrets --as=system:serviceaccount:default:default
```

Confirm the token is mounted whether you need it or not:

```
POD=$(kubectl get pod -l app=pingd -o jsonpath='{.items[0].metadata.name}') &&
kubectl exec $POD -- ls /var/run/secrets/kubernetes.io/serviceaccount/
```

Now see Pod Security Standards reject a non-compliant pod:

```
kubectl create namespace pss-test && kubectl label namespace pss-test pod-security.kubernetes.io/enforce=restricted
```

```
kubectl run bad -n pss-test --image=busybox:1.37 --restart=Never --command --
sleep 60
```

Read the four itemised violations. Now a compliant pod in the same namespace:

```
kubectl apply -n pss-test -f examples/manifests/23-compliant-pod.yaml && kubectl
get pod -n pss-test good
```

And confirm the *controller* case — a violating Deployment is accepted, then fails silently:

```
kubectl create deployment baddep -n pss-test --image=busybox:1.37 -- sleep 300
```

```
sleep 8 && kubectl get deploy,rs,pods -n pss-test | head -6
```

```
kubectl describe rs -n pss-test -l app=baddep | grep -A5 Events | tail -4
```

The Deployment exists with zero pods, and the reason is only in the ReplicaSet's events.

Clean up:

```
kubectl delete namespace pss-test
```

Takeaways

- RBAC's default is restrictive — but **the ServiceAccount token is mounted into every pod anyway**. Set `automountServiceAccountToken: false` unless the workload uses the API.
- Give each workload its own ServiceAccount: least privilege, audit attribution, and cloud IAM binding.
- A **RoleBinding referencing a ClusterRole** grants it within one namespace — use it to reuse `view/edit/admin`.
- RBAC is additive; there are no deny rules.
- `secrets: list, pods/exec, create pods, escalate, bind, impersonate` and `nodes/proxy` are effectively cluster-admin. Audit them.
- **Pod Security Standards** enforce hardening by namespace label. Use `warn` before `enforce`. Violations are itemised with the exact fields.
- PSS applies to pods, so a violating **Deployment is accepted** and fails at the ReplicaSet — enable `warn` for feedback at apply time.
- `runAsNonRoot` requires a numeric UID in the image; `fsGroup` fixes volume permissions.
- Four layers: authentication, authorization, admission, runtime. Weakness anywhere undermines the rest.

Previous: Chapter 22 — Cookbook · Next: Chapter 24 — Admission control and supply chain

Chapter 24 — Admission control and supply chain

Admission control is the API server's last checkpoint: after authentication and authorisation, before persistence. It is where "no pod may run as root", "every workload must have resource limits", and "only signed images from our registry" become enforceable rather than aspirational.

The request path

```
request → authentication → authorization → MUTATING admission
        → schema validation → VALIDATING admission → etcd
```

Two things follow from that ordering.

Mutating runs before validating. A mutating webhook can add a sidecar, inject default resource limits, or set a securityContext, and the validating stage then checks the *mutated* object. That is how service meshes inject proxies and how policy engines can both fix and reject.

Both run before anything is stored. A rejected request never reaches etcd, so admission is a genuine gate rather than a cleanup process.

Built-in controllers handle a lot already — `NamespaceLifecycle`, `LimitRanger`, `ResourceQuota`, `ServiceAccount`, `PodSecurity` (Chapter 23), `DefaultStorageClass`. What you add on top is for policy the built-ins cannot express.

Policy engines

Writing raw admission webhooks means running an HTTPS server, managing its certificates, and being in the critical path of every API write. Use a policy engine instead.

Kyverno — policies are Kubernetes resources in YAML, no new language:

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-resource-limits
spec:
  validationFailureAction: Enforce
  rules:
    - name: check-limits
      match:
        any:
          - resources:
              kinds: ["Pod"]
      validate:
        message: "Every container must set memory and CPU limits."
        pattern:
          spec:
            containers:
              - resources:
                  limits:
                    memory: "?*"
                    cpu: "?*"

```

Kyverno also **mutates** and **generates** — inject default labels, or create a NetworkPolicy and a ResourceQuota automatically whenever a namespace appears. That generation capability is genuinely useful for multi-tenancy (Chapter 28).

OPA Gatekeeper — policies in Rego, more expressive and a steeper learning curve. Better when policies need real logic; heavier for simple rules.

Validating Admission Policy — built into Kubernetes (stable since v1.30), using CEL expressions, **with no webhook to run**:

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata:
  name: require-limits
spec:
  failurePolicy: Fail
  matchConstraints:
    resourceRules:
      - apiGroups: ["apps"]
        apiVersions: ["v1"]
        operations: ["CREATE", "UPDATE"]
        resources: ["deployments"]
  validations:
    - expression: "object.spec.template.spec.containers.all(c, has(c.resources.limits))"
      message: "All containers must have resource limits."
```

This is the option to reach for first now. No extra component, no certificate rotation, no availability risk, and it covers a large fraction of what people install Gatekeeper for. Complex policies still want Kyverno or Gatekeeper.

Webhook failure modes

If you do run webhooks, understand the risk you are taking on.

```
failurePolicy: Fail      # reject requests when the webhook is unreachable
failurePolicy: Ignore    # allow them through
```

`Fail` is the secure choice and it makes your webhook a **hard dependency of the API server**. A webhook that is down with `failurePolicy: Fail` and a broad `matchConstraints` can block all pod creation cluster-wide — including the pods that would restore the webhook. That is a genuine, recurring way to brick a cluster.

Protections, all of which are worth applying:

```
namespaceSelector:
  matchExpressions:
    - key: kubernetes.io/metadata.name
      operator: NotIn
      values: ["kube-system"]
timeoutSeconds: 5
```

- **Exclude** `kube-system` so recovery is always possible.

- **Short timeouts** — the default is 10 s and it is added to every matching API call.
- **Scope narrowly** — match only the resources and operations you actually care about.
- **Run the webhook itself highly available**, with a PDB.

Supply chain

The Docker book covered building trustworthy images: SBOMs, scanning, provenance and signing. Kubernetes is where you *enforce* that only such images run.

Restrict registries

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: restrict-registries
spec:
  validationFailureAction: Enforce
  rules:
    - name: allowed-registries
      match:
        any:
          - resources:
              kinds: ["Pod"]
      validate:
        message: "Images must come from ghcr.io/our-org/."
        pattern:
          spec:
            containers:
              - image: "ghcr.io/our-org/*"
```

Simple and effective — it blocks the typosquatting and random-Docker-Hub-image problems in one rule.

Require digests, not tags

```
validate:
  message: "Images must be referenced by digest."
  pattern:
    spec:
      containers:
        - image: ".*@sha256:.*"
```

This is the enforcement half of the Docker book's tagging discipline. A tag is mutable; a digest is not. Requiring digests at admission means the image that passed your pipeline is the image that runs.

Verify signatures

```

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: verify-signatures
spec:
  validationFailureAction: Enforce
  rules:
    - name: verify
      match:
        any:
          - resources:
              kinds: ["Pod"]
      verifyImages:
        - imageReferences: ["ghcr.io/our-org/*"]
          attestors:
            - entries:
                - keyless:
                    subject: "https://github.com/our-org/*"
                    issuer: "https://token.actions.githubusercontent.com"

```

This is what makes signing meaningful. The Docker book's point stands: **signing without verification is ceremony**. Admission control is where verification happens, and until you have this policy, `cosign sign` in your pipeline achieves nothing.

Sigstore Policy Controller and Connaisseur do the same job if you prefer a dedicated tool.

Scan continuously, not just at build

An image that was clean when built accumulates CVEs. Scan what is *running*:

```
trivy k8s --report summary cluster
```

Operators like Trivy Operator scan workloads continuously and publish results as custom resources you can alert on. This closes the gap the Docker book identified: a weekly rebuild handles images you own; continuous scanning of running workloads catches everything else, including third-party images.

A layered policy set

A defensible starting point, roughly in order of value:

1. **Pod Security Standards** — `restricted` where possible, `baseline` elsewhere (Chapter 23).
2. **Require resource requests and limits** — otherwise BestEffort pods and no scheduling accounting (Chapter 8).
3. **Restrict registries** to your own plus an explicit allowlist.
4. **Require digests** for production namespaces.
5. **Verify signatures** for your own images.
6. **Require standard labels** (owner, team, cost centre) — dull, and it makes everything else attributable.
7. **Block** `:latest`, `hostPath`, `host networking`, and `host ports` outside a defined allowlist.

Roll each out in audit mode first. A policy engine switched straight to enforce across an existing cluster will block deployments during an incident, and the resulting distrust sets the whole effort back further than the delay would have.

Try it

Kyverno and Gatekeeper need installing; the built-in policy does not. Create a `ValidatingAdmissionPolicy` requiring resource limits on Deployments:

```
kubectl apply -f examples/manifests/24-validating-policy.yaml
```

Now try a Deployment with no limits:

```
kubectl create deployment nolimits --image=busybox:1.37 -- sleep 300
```

It is rejected at admission, by the API server, with your message — no webhook involved.

Confirm a compliant one is accepted:

```
kubectl apply -f examples/manifests/24-with-limits.yaml && kubectl get deploy  
withlimits
```

Inspect what admission plugins your API server runs:

```
kubectl get --raw='/livez?verbose' | grep -i admission
```

And see the webhooks currently registered — on a fresh cluster this is where the ingress controller's admission webhook shows up:

```
kubectl get validatingwebhookconfigurations,mutatingwebhookconfigurations
```

Check their failure policies, because this is the cluster-bricking risk:

```
kubectl get validatingwebhookconfigurations -o custom-  
columns='NAME:.metadata.name,POLICY:.webhooks[*].failurePolicy,TIMEOUT:.webhooks[*]  
.timeoutSeconds'
```

Clean up:

```
kubectl delete -f examples/manifests/24-validating-policy.yaml -f examples/  
manifests/24-with-limits.yaml --ignore-not-found
```

Takeaways

- Admission runs **after** authz and **before** etcd. Mutating first, then validating — which is how sidecars get injected and defaults applied.
- **Validating Admission Policy (CEL) is built in** and needs no webhook. Try it before installing a policy engine.

- Kyverno for YAML-native policy plus mutation and generation; Gatekeeper/Rego for complex logic.
 - `failurePolicy: Fail` **makes a webhook a hard dependency of the API server**. Exclude `kube-system`, use short timeouts, scope narrowly, and run it HA — or you can block all pod creation, including the pods that would fix it.
 - Enforce supply chain at admission: allowed registries, **digests not tags**, and signature verification. **Signing without verification is ceremony**.
 - Scan running workloads continuously, not only at build time.
 - Roll every policy out in audit mode first.
-

Previous: Chapter 23 — Security · Next: Chapter 25 — Scaling and disruption

Chapter 25 — Scaling and disruption

Two related problems: adding capacity when load rises, and not losing availability when the cluster moves things around underneath you.

Horizontal Pod Autoscaler

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: pingd
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: pingd
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

The control loop: every 15 seconds, compute

```
desiredReplicas = ceil(currentReplicas × currentMetric / targetMetric)
```

`averageUtilization: 70` means **70% of the pod's CPU request, not of a core and not of the node**. A pod requesting 100m and using 70m is at 100% utilisation. This trips people up constantly: an HPA appears to scale far too eagerly, and the cause is a request that is much lower than actual usage.

So the HPA is only as good as your requests. Get Chapter 8 right first.

Requirements and traps

metrics-server must be installed. Without it the HPA reports `<unknown>` and does nothing.

```
kubectl top pods
```

If that fails, the HPA cannot work.

Remove replicas from your Deployment manifest. This is the trap worth repeating from Chapter 6: if `replicas: 3` is in the manifest and an HPA has scaled to 12, the next `kubectl apply` or GitOps sync sets it back to 3, the HPA scales up again, and you have two controllers fighting over one field. Under server-side apply you may instead get a conflict error, which is better but still needs fixing. Delete the field.

Scaling has asymmetric behaviour by design. Scale-up is immediate; scale-down waits a stabilisation window (300 s by default) to avoid flapping. Tune it explicitly:

```

behavior:
  scaleDown:
    stabilizationWindowSeconds: 300
    policies:
      - type: Percent
        value: 50
        periodSeconds: 60
  scaleUp:
    stabilizationWindowSeconds: 0
    policies:
      - type: Percent
        value: 100
        periodSeconds: 30

```

CPU is a poor proxy for most services. A latency-bound service waiting on a database uses little CPU while being completely saturated. Scale on what actually indicates load — queue depth, in-flight requests, or requests per second — via custom or external metrics:

```

- type: Pods
  pods:
    metric:
      name: http_inflight_requests
    target:
      type: AverageValue
      averageValue: "30"

```

This needs a metrics adapter (Prometheus Adapter, or KEDA). **KEDA** is worth knowing: it scales on queue length, Kafka lag, cloud queue depth and dozens of other sources, and supports scale-to-zero, which the HPA does not.

Vertical Pod Autoscaler

Adjusts requests and limits rather than replica count.

```

apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: pingd
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: pingd
  updatePolicy:
    updateMode: "Off" # recommend only

```

`updateMode: "Off"` **is the mode to use.** It computes recommendations and changes nothing, which makes VPA an excellent free advisor for the sizing problem in Chapter 8:

```
kubectl describe vpa pingd | grep -A10 'Recommendation'
```

In `Auto` mode, VPA historically had to **evict pods** to change their resources, which is disruptive and interacts badly with PDBs. In-place resizing is stabilising in recent versions and changes this materially, but check what your cluster actually supports before enabling it.

Do not run VPA in `Auto` mode and an HPA on CPU against the same workload. They fight: VPA raises the request, which lowers utilisation, which makes the HPA scale down, which raises per-pod load, which makes VPA raise the request again. VPA on memory plus HPA on CPU is a supported combination.

Cluster Autoscaler and Karpenter

Pod autoscaling is useless if there is nowhere to put the pods.

Cluster Autoscaler watches for `Pending` pods that cannot schedule and adds nodes to a node group; it removes nodes that have been underutilised for a period. It works within predefined node groups, so instance types are fixed.

Karpenter provisions nodes directly from a set of constraints, choosing instance types to fit the pending pods, and consolidates aggressively. It is generally faster and cheaper, and it is much more willing to move your pods around — which makes PDBs and graceful shutdown non-optional.

Things that block scale-down, and therefore cost money:

- Pods with no controller (bare pods) — nothing can recreate them.
- Pods with local storage (`emptyDir`) unless annotated as safe to evict.
- Pods a PDB will not permit to be evicted.
- `kube-system` pods without a PDB.

```
metadata:
  annotations:
    cluster-autoscaler.kubernetes.io/safe-to-evict: "true"
```

A node stuck at low utilisation for weeks is usually one of those five, and the autoscaler's logs say which.

PodDisruptionBudgets

The mechanism that stops voluntary disruption from taking your service down.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: pingd
spec:
  maxUnavailable: 1
  selector:
    matchLabels:
      app: pingd
```

Voluntary disruption — node drains, autoscaler consolidation, `kubectl delete node` — goes through the Eviction API and respects PDBs. **Involuntary disruption** — node failure, kernel OOM, kubelet eviction under pressure — does **not**. A PDB is not a durability guarantee.

Chapter 21 measured a drain moving pods off a node. With a PDB, that drain blocks rather than violating your minimum.

The rules that matter:

Prefer `maxUnavailable: 1` **over** `minAvailable: N`. A PDB with `minAvailable` equal to the replica count can never be satisfied, so **drains block forever** and node upgrades become incidents. `maxUnavailable` scales with the Deployment automatically.

A single-replica Deployment with a PDB cannot be drained at all without violating it. If you have single-replica workloads, either accept the disruption or run two.

Every workload that matters needs one, including infrastructure. CoreDNS without a PDB can lose all replicas to a drain (Chapter 12).

```
kubectl get pdb -A
```

```
kubectl get pdb pingd -o jsonpath='{.status.disruptionsAllowed}
current={.status.currentHealthy} desired={.status.desiredHealthy}}{"\n"}'
```

`disruptionsAllowed: 0` means the next drain will block. That is worth alerting on — it is usually the first sign that a workload is degraded.

Topology spread

Replicas on one node means one node failure is an outage. Chapter 21's measurement showed all three `pingd` pods landing on a single worker after a drain, and **staying there** because the scheduler never rebalances.

```
spec:
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: kubernetes.io/hostname
      whenUnsatisfiable: ScheduleAnyway
      labelSelector:
        matchLabels:
          app: pingd
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule
      labelSelector:
        matchLabels:
          app: pingd
```

- `maxSkew` — the permitted difference in pod count between topology domains.
- `whenUnsatisfiable: DoNotSchedule` — hard requirement; the pod stays `Pending` if it cannot be satisfied.
- `ScheduleAnyway` — a preference.

Use `ScheduleAnyway` for hostname spreading (you would rather run unbalanced than not at all) and `DoNotSchedule` for zone spreading if your availability model depends on it. This is the modern replacement for pod anti-affinity, which is more expensive to evaluate and harder to read.

Remember it applies **at scheduling time only**. After a drain, redistribution needs a `kubectl rollout restart` — which is a good argument for doing one after any significant node event.

Try it

Check metrics-server, without which the HPA is inert:

```
kubectl top pods -l app=pingd 2>&1 | head -3
```

Create an HPA:

```
kubectl autoscale deployment pingd --min=3 --max=10 --cpu-percent=70
```

```
kubectl get hpa pingd
```

<unknown> in the TARGETS column means no metrics-server. Note what happens to the manifest conflict — apply the original Deployment, which specifies `replicas: 3`:

```
kubectl apply -f examples/manifests/01-deployment.yaml && kubectl get hpa,deploy pingd
```

Now a PodDisruptionBudget, and see it block a drain:

```
kubectl apply -f examples/manifests/25-pdb.yaml && kubectl get pdb pingd
```

```
kubectl get pdb pingd -o jsonpath='{.status.disruptionsAllowed}
healthy={.status.currentHealthy}{"\n"}'
```

Try draining the node holding most replicas:

```
kubectl drain k8sbook-worker --ignore-daemonsets --delete-emptydir-data --
timeout=60s
```

With `maxUnavailable: 1` it evicts one at a time and waits for replacements; with a stricter budget it would block. Restore:

```
kubectl uncordon k8sbook-worker
```

Add topology spread and force redistribution:

```
kubectl patch deployment pingd --type=merge -p '{"spec":{"template":{"spec":
{"topologySpreadConstraints":[{"maxSkew":1,"topologyKey":"kubernetes.io/
hostname","whenUnsatisfiable":"ScheduleAnyway","labelSelector":{"matchLabels":
{"app":"pingd"}}}]}}}'
```

```
kubectl rollout status deployment/pingd && kubectl get pods -l app=pingd -o wide --
no-headers | awk '{print $1, $7}'
```

Clean up:

```
kubectl delete hpa pingd; kubectl delete -f examples/manifests/25-pdb.yaml
```

Takeaways

- HPA targets a percentage of the **CPU request**, not a core. Bad requests produce nonsense scaling.
- **Remove replicas from manifests managed by an HPA**, or the two fight.
- Scale-up is immediate; scale-down waits 300 s by default. Tune with `behavior`.
- CPU is a poor load signal for latency-bound services — scale on queue depth or in-flight requests, via Prometheus Adapter or KEDA (which also does scale-to-zero).
- **VPA in `updateMode: "Off"` is a free sizing advisor**. Do not combine `Auto` VPA with a CPU-based HPA.
- Cluster Autoscaler works within node groups; Karpenter provisions to fit and moves pods more aggressively — so PDBs and graceful shutdown matter more with it.
- **PDBs constrain voluntary disruption only**. Node failure ignores them.
- Prefer `maxUnavailable: 1`. `minAvailable` equal to replica count **blocks drains forever**.
- Alert on `disruptionsAllowed: 0`.
- Topology spread applies at scheduling time only; use `rollout restart` to rebalance after node events.

Previous: Chapter 24 — Admission control and supply chain · Next: Chapter 26 — Observability

Chapter 26 — Observability

A cluster produces four kinds of signal: metrics, logs, events and traces. Kubernetes ships almost none of the infrastructure to collect them — it produces the data and expects you to bring the stack.

The gap you notice first

```
kubectl top pods
```

```
error: Metrics API not available
```

That is a default cluster. `kubectl top` needs **metrics-server**, which is not installed by default anywhere except some managed offerings. Neither is anything that stores logs, keeps events beyond an hour, or traces requests.

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

metrics-server provides *only* current CPU and memory for pods and nodes — enough for `kubectl top` and the HPA (Chapter 25), and nothing else. It stores no history. It is a prerequisite, not a monitoring system.

Metrics

Prometheus is the de facto standard, usually via kube-prometheus-stack, which bundles Prometheus, Alertmanager, Grafana, node-exporter and kube-state-metrics with dashboards and alert rules already written.

Three distinct sources, and knowing which is which saves confusion:

- **kubelet / cAdvisor** — actual container resource usage: `container_cpu_usage_seconds_total`, `container_memory_working_set_bytes`, `container_cpu_cfs_throttled_periods_total`.
- **kube-state-metrics** — the *state of API objects*: `kube_pod_status_phase`, `kube_deployment_status_replicas_available`, `kube_pod_container_status_restarts_total`. This is object state, not resource usage, and it is what most alerting is built on.
- **Your application** — request rate, error rate, latency. Nothing else can tell you these.

Scraping is declarative with the Prometheus Operator:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: pingd
spec:
  selector:
    matchLabels:
      app: pingd
  endpoints:
    - port: metrics
      interval: 30s
```


The trap: `ServiceMonitor` **selects a Service, and that Service must expose the metrics port**. A `ServiceMonitor` whose selector matches nothing produces no error — exactly the silent failure of Chapter 19, in a different guise. Check `Status → Targets` in Prometheus.

Logs

```
kubectl logs -f --tail=100 deploy/pingd
kubectl logs POD -c CONTAINER --previous
kubectl logs -l app=pingd --prefix --tail=20
```

`kubectl logs` reads files on the node written by the container runtime. Those files are rotated by the kubelet (10 MB per file, 5 files by default) and **deleted when the pod is deleted**. So logs from a pod that no longer exists are gone, which is precisely when you want them — after a crash, a rollout, or an eviction.

Shipping them elsewhere is not optional in production. The standard pattern is a **DaemonSet collector** — Fluent Bit, Vector, Promtail, Filebeat — reading `/var/log/containers/` on each node and forwarding to Loki, Elasticsearch/OpenSearch, or a managed service.

Everything the Docker book said applies unchanged: **log to stdout**, use **structured JSON**, set `PYTHONUNBUFFERED=1` or the equivalent, and never log secrets. Kubernetes adds one thing worth doing — enrich each line with pod, namespace, node and container, which collectors do automatically from the file path and the API.

Events

Events are the cluster's own narration (Chapter 16), and they are **deleted after about an hour**:

```
kubectl get events --sort-by=.lastTimestamp -A | tail -20
```

That retention makes them useless for post-incident analysis unless you export them. Ship them like logs — most collectors have a Kubernetes events input, and `kubernetes-event-exporter` is purpose-built.

Events worth alerting on directly: `Failed`, `FailedScheduling`, `Unhealthy`, `OOMKilling`, `Evicted`, `BackOff`, `FailedMount`.

Traces

OpenTelemetry, as in the Docker book. What Kubernetes adds is the ability to correlate a span with the pod that produced it — inject identity via the downward API (Chapter 7):

```
env:
- name: OTEL_RESOURCE_ATTRIBUTES
  value: "k8s.pod.name=$(POD_NAME),k8s.namespace.name=$(POD_NAMESPACE),k8s.node.name=$(NODE_NAME)"
```

The OpenTelemetry Operator can auto-instrument workloads by annotation, without changing images — worth knowing before you plan an instrumentation project.

What to alert on

Symptoms, not causes. "CPU is high" is not an incident; "the error rate is 5%" is.

Workload health

```
# Pods restarting repeatedly
increase(kube_pod_container_status_restarts_total[15m]) > 3
```

```
# Deployment below desired replicas for 15 minutes
kube_deployment_status_replicas_available < kube_deployment_spec_replicas
```

```
# Pods stuck Pending
kube_pod_status_phase{phase="Pending"} > 0
```

Resource pressure — the two that hide

```
# CPU throttling: latency with low utilisation (Chapter 20)
rate(container_cpu_cfs_throttled_periods_total[5m])
/ rate(container_cpu_cfs_periods_total[5m]) > 0.25
```

```
# Approaching the memory limit before it OOMs
container_memory_working_set_bytes
/ container_spec_memory_limit_bytes > 0.9
```

Both matter because `kubectl top` cannot show throttling at all, and an OOM kill may report `reason: Error` rather than `OOMKilled` (Chapter 20).

Cluster health

```
kube_node_status_condition{condition="Ready",status="true"} == 0
```

```
# A PDB that will block the next drain (Chapter 25)
kube_poddisruptionbudget_status_pod_disruptions_allowed == 0
```

```
# Certificate expiry – the one-year cliff from Chapter 21
apiserver_client_certificate_expiration_seconds_bucket
```

```
# etcd disk latency – presents as whole-cluster slowness
histogram_quantile(0.99, rate(etcd_disk_wal_fsync_duration_seconds_bucket[5m])) > 0.5
```

Do not alert on node CPU utilisation, pod count, or memory usage without reference to limits. They generate noise and train people to ignore alerts.

Dashboards worth having

- **Cluster capacity** — allocatable vs requests vs actual usage, per node. This single view answers most "why won't it schedule" questions.

- **Per-workload** — replicas desired/available, restarts, CPU and memory against requests and limits, **throttling percentage**.
- **Golden signals per service** — rate, errors, duration, saturation.
- **Control plane** — API latency, etcd fsync, scheduler queue depth.

A minimal stack

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring --create-namespace
```

```
helm repo add grafana https://grafana.github.io/helm-charts
helm install loki grafana/loki-stack -n monitoring
```

That is metrics, alerting, dashboards and logs in two commands. Add metrics-server separately — kube-prometheus-stack does not include it, and the HPA needs it.

Give the monitoring stack **resource requests and a PDB**. Monitoring that is BestEffort gets evicted exactly when the cluster is under pressure, which is when you need it (Chapter 20).

Cost visibility

Worth a mention because it is invisible by default and material in practice. OpenCost and Kubecost attribute node cost to namespaces, workloads and labels using requests and usage. The usual finding is that requests are set far above real usage, and that the gap is most of the bill. Chapter 25's VPA in recommendation mode is the cheap way to act on it.

Try it

Confirm the gap:

```
kubectl top pods
```

```
error: Metrics API not available
```

Install metrics-server. On kind, the kubelet's serving certificate is self-signed, so it needs `--kubelet-insecure-tls` — a local-cluster concession, not a production setting:

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

```
kubectl patch deployment metrics-server -n kube-system --type=json -p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-tls"}]'
```

```
kubectl rollout status deployment/metrics-server -n kube-system --timeout=180s
```

```
kubectl top nodes; kubectl top pods -l app=pingd --containers
```

Now compare the two numbers Chapter 8 insisted were different — requests (what the scheduler sees) versus usage (what is happening):

```
kubectl describe node k8sbook-worker | sed -n '/Allocated resources/,/Events/p' | head -8
```

```
kubectl top node k8sbook-worker
```

Read the raw per-container metrics the kubelet exposes — this is the endpoint metrics-server scrapes:

```
kubectl get --raw "/api/v1/nodes/k8sbook-worker/proxy/metrics/resource" | grep -m3 '^container'
```

```
container_cpu_usage_seconds_total{container="kube-proxy",namespace="kube-system",...}
container_memory_working_set_bytes{container="kindnet-cni",namespace="kube-system",...}
```

Note what is **not** there: the `/metrics/resource` endpoint carries CPU and memory only. The throttling counters (`container_cpu_cfs_throttled_periods_total`) come from the kubelet's cAdvisor endpoint (`/metrics/cadvisor`) and are only emitted for containers with CPU limits that have actually been throttled — which is why you need Prometheus scraping cAdvisor to alert on throttling, and why it stays invisible until you do.

And see how short event retention is:

```
kubectl get events -A --sort-by=.lastTimestamp -o custom-columns='AGE:.lastTimestamp,REASON:.reason,OBJECT:.involvedObject.name' | head -5
```

Nothing older than about an hour.

Takeaways

- A default cluster has **no metrics, no log storage, no event retention, no tracing**. `kubectl top` fails until you install metrics-server.
- metrics-server serves current CPU/memory only — a prerequisite for `top` and the HPA, not a monitoring system.
- Three metric sources: kubelet/cAdvisor (usage), kube-state-metrics (object state), your app (golden signals). You need all three.
- A `ServiceMonitor` whose selector matches nothing fails **silently**. Check Prometheus targets.
- **Pod logs are deleted with the pod**. Ship them off-node or lose them exactly when it matters.
- **Events expire after ~1 hour**. Export them or you cannot do post-incident analysis.

- Alert on symptoms. The two hidden ones worth explicit rules: **CPU throttling ratio** and **memory approaching the limit**.
 - Give your monitoring stack requests and a PDB, or it is evicted when you need it most.
-

Previous: Chapter 25 — Scaling and disruption · Next: Chapter 27 — Deployment strategies and GitOps

Chapter 27 — Deployment strategies and GitOps

Chapter 6 covered the mechanics of a rolling update. This chapter covers choosing a strategy, and the question of what drives the cluster in the first place.

Rolling update

The default, and the right answer for most services:

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

The defaults are 25%/25%, which **dips below capacity during every rollout**. Set `maxUnavailable: 0` for anything serving traffic.

What makes it safe is readiness gating (Chapter 9): a new pod that never becomes ready stops the rollout. Chapter 9 measured this — a broken readiness path left three old pods serving while the new one sat 0/1, and the rollout stalled rather than taking the service down.

The limitations are worth naming:

- **Two versions run simultaneously.** Your API and database schema must tolerate that.
- **No automatic rollback.** `progressDeadlineSeconds` stops a stalled rollout; it does not reverse it.
- **No traffic control.** You cannot send 5% of requests to the new version — the split is by replica count, and only in whole pods.

Recreate

```
spec:
  strategy:
    type: Recreate
```

Terminate everything, then start the new version. Downtime by design.

Legitimate when two versions genuinely cannot coexist: a singleton holding an exclusive lock, a `ReadWriteOnce` volume, or a schema migration that is not backwards compatible. Choosing it deliberately is fine; choosing it because it is simpler is not.

Blue/green

Two full environments, traffic switched at once — implemented with plain Kubernetes by changing a Service selector:

```
spec:
  selector:
    app: pingd
    version: blue      # change to `green` to switch
```

Instant switch, instant rollback, no version mixing. Costs double the resources during the transition, and the switch is atomic, so a bad version reaches 100% of traffic immediately — you find out fast, which cuts both ways.

Note that in-flight connections to blue are not drained by a selector change; they simply stop receiving *new* connections.

Canary

Route a small fraction of traffic to the new version, watch, then proceed or abort.

With replicas only — crude but free:

```
9 pods labelled version=stable + 1 pod labelled version=canary
```

Both matched by one Service, so roughly 10% of connections hit the canary. Granularity is limited by replica count, and it is per *connection*, not per request (Chapter 11).

With a Gateway API HTTPRoute — proper weighting (Chapter 13):

```
backendRefs:
  - name: pingd-stable
    port: 80
    weight: 90
  - name: pingd-canary
    port: 80
    weight: 10
```

With a progressive delivery controller — Argo Rollouts or Flagger — the weights step automatically while metrics are checked at each stage:

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
spec:
  strategy:
    canary:
      steps:
        - setWeight: 10
        - pause: {duration: 5m}
        - analysis:
            templates:
              - templateName: success-rate
        - setWeight: 50
        - pause: {duration: 5m}
        - setWeight: 100
```

The analysis step is the point: it queries Prometheus for error rate or latency and **rolls back automatically** if the canary is worse. This is the only approach in this chapter that gives you automated rollback on real signal, and it is why progressive delivery is worth the extra component for high-traffic services.

Choosing

Situation	Strategy
Ordinary stateless service	Rolling, <code>maxUnavailable: 0</code>
Versions cannot coexist	Recreate, accepting downtime
Need instant rollback, can afford 2×	Blue/green
High traffic, want automated safety	Canary with analysis
Database schema change	Expand/contract, independent of the above

That last row matters more than the strategy choice. **Any strategy that runs two versions at once requires backwards-compatible schema changes:** add columns, deploy code that writes both, migrate, then remove — across several releases. Getting this wrong makes every rollout strategy equally broken, and it is where most deployment incidents actually originate.

GitOps

The deployment question underneath all of the above: what puts manifests into the cluster?

Push — CI runs `kubectl apply`. Simple, and it means your CI system holds cluster credentials, and nothing detects or corrects drift.

Pull (GitOps) — a controller *in* the cluster watches a git repository and reconciles. Credentials never leave the cluster, drift is corrected continuously, and git is the audit log.

The second is Chapter 2's model applied to deployment: git holds desired state, a controller converges towards it, continuously.

Argo CD — a UI-forward implementation with strong visualisation of sync status and diffs:


```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: pingd
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/org/manifests
    targetRevision: main
    path: apps/pingd/overlays/production
  destination:
    server: https://kubernetes.default.svc
    namespace: production
  syncPolicy:
    automated:
      prune: true
      selfHeal: true

```

`selfHeal: true` reverts manual changes; `prune: true` deletes resources removed from git. Both are the point of GitOps and both will surprise someone the first time.

Flux — a set of controllers, no UI, more composable, integrates naturally with Helm and Kustomize.

What GitOps actually gives you

- **The cluster's state is reviewable** — every change is a pull request.
- **Drift is corrected**, so `kubectl edit` in production stops being a permanent invisible change (Chapter 2's imperative/declarative conflict, solved).
- **Rollback is `git revert`**. This resolves the tension in Chapter 6: `kubectl rollout undo` desynchronises the cluster from git, whereas a revert fixes both.
- **CI does not need cluster credentials** — it builds and pushes images, and updates a tag in git.

What it does not solve

- **Secrets**. Plain Secrets cannot go in git. Use Sealed Secrets, SOPS, or the External Secrets Operator (Chapter 7).
- **The image tag update**. Something must write the new tag into git — a CI step, Argo CD Image Updater, or Flux's image automation. This is the most commonly under-designed part of a GitOps setup.
- **Ordering across applications**. Sync waves and health checks help; complex inter-app dependencies are still awkward.
- **Emergency changes**. `selfHeal` will revert your 3am `kubectl edit` within minutes. Have a documented break-glass procedure — usually pausing sync — and expect to need it.

Repository layout

The pattern that scales:

```
manifests/
  base/
    pingd/                # kustomize base
  overlays/
    staging/
      kustomization.yaml   # patches: replicas, image tag, resources
    production/
      kustomization.yaml
```

Keep **application code and manifests in separate repositories** once more than one team is involved. A single repo is simpler at first and becomes painful: every image tag update creates a commit in the application repo, triggering CI, which builds an image, which updates a tag. Separating them breaks that loop.

Chapter 29 covers Kustomize and Helm in detail.

Deploying the running example

Whatever drives it, the pipeline shape from the Docker book holds:

```
build image → test → scan → push with an immutable tag
→ update the tag in the manifests repo
→ GitOps controller syncs
→ wait for rollout, verify health
```

```
kubectl rollout status deployment/pingd --timeout=300s
```

That command belongs in any push-based pipeline — it blocks until ready and exits non-zero on failure, which is what turns "the API accepted my manifest" into "the new version is actually serving".

Try it

Watch a rolling update maintain capacity, using the running example:

```
kubectl set image deployment/pingd api=pingd:latest --record=false && kubectl set
env deployment/pingd PINGD_VERSION=2.0.0
```

```
for i in $(seq 1 10); do kubectl get deployment pingd -o
jsonpath='{.status.readyReplicas} updated={.status.updatedReplicas}
total={.status.replicas}{"\n"}'; sleep 2; done
```

ready never drops below 3 because maxUnavailable: 0.

Now simulate blue/green with a Service selector. Create a second Deployment labelled green:

```
kubectl apply -f examples/manifests/27-bluegreen.yaml && kubectl rollout status
deployment/pingd-green --timeout=180s
```

Note the two sets of pod IPs:

```
kubectl get pods -l app=pingd -o jsonpath='blue: {.items[*].status.podIP}{"\n"}';
kubectl get pods -l app=pingd-green -o jsonpath='green: {.items[*].status.podIP}{"\n"}'
```

```
blue: 10.244.2.27 10.244.2.29 10.244.2.26
green: 10.244.1.47 10.244.2.32 10.244.1.48
```

Point the Service at green and watch the endpoints move wholesale:

```
kubectl patch service pingd -p '{"spec":{"selector":{"app":"pingd-green"}}}'
```

```
sleep 6 && kubectl get endpointslice -l kubernetes.io/service-name=pingd -o
jsonpath='{.items[*].endpoints[*].addresses[0]}{"\n"}'
```

```
10.244.1.48 10.244.1.47 10.244.2.32
```

Entirely green IPs, in one atomic change. Switch back instantly — this is the blue/green rollback:

```
kubectl patch service pingd -p '{"spec":{"selector":{"app":"pingd"}}}'
```

Clean up:

```
kubectl delete -f examples/manifests/27-bluegreen.yaml && kubectl rollout undo
deployment/pingd
```

Takeaways

- Rolling with `maxUnavailable: 0` is the default answer. Readiness gating is what makes it safe; without a readiness probe a broken version rolls out completely.
- Rolling updates **do not roll back automatically**. `progressDeadlineSeconds` only stops.
- Blue/green gives instant switch and rollback at double the cost; canary gives graduated exposure but needs a router that can weight traffic.
- **Only progressive delivery with metric analysis (Argo Rollouts, Flagger) gives automated rollback on real signal.**
- Any strategy running two versions at once requires **backwards-compatible schema changes**. This causes more incidents than the strategy choice does.
- GitOps is Chapter 2's reconciliation applied to deployment: git is desired state, drift is corrected, rollback is `git revert`, and CI never holds cluster credentials.
- GitOps does not solve secrets, the image-tag update loop, cross-app ordering, or emergency changes — `selfHeal` will revert your manual fix.
- Separate application and manifest repositories once more than one team is involved.

Chapter 28 — Multi-tenancy

Several teams, one cluster. It is the usual reason to adopt Kubernetes and the source of its sharpest limitation: **namespaces are an organisational boundary, not a security one**, and a great deal of multi-tenancy design is working around that.

What a namespace gives you

- **Name scoping** — two teams can both have a `pingd` Service.
- **An RBAC target** — permissions granted per namespace (Chapter 23).
- **A quota target** — ResourceQuota and LimitRange.
- **A policy target** — NetworkPolicy, Pod Security Standards.

That is genuinely a lot, and it covers most *cooperative* multi-tenancy: teams within one organisation who are not trying to attack each other.

What it does not give you

Kernel isolation. Pods from different namespaces run on the same nodes, sharing a kernel. A container escape crosses every namespace on that node. This is the Docker book's Chapter 1 point, unchanged.

Network isolation. Every pod can reach every other pod in every namespace **by default**. Namespaces do nothing about this — you need NetworkPolicy (Chapter 14), and you need it in every namespace.

Resource isolation. Without quotas, one namespace can consume the whole cluster. Even with quotas, noisy neighbours share node CPU, memory bandwidth, disk and network.

Control plane isolation. All tenants share one API server and one etcd. A tenant creating objects in a hot loop degrades the cluster for everyone.

Cluster-scoped resources. CRDs, ClusterRoles, StorageClasses, PriorityClasses, webhooks — all global. One tenant installing an operator affects everyone.

So the honest position: **namespaces are appropriate for teams that trust each other. Hostile or untrusted tenants need separate clusters**, or virtual control planes (vCluster, Capsule), or sandboxed runtimes (gVisor, Kata) — and usually a combination.

ResourceQuota

```

apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-a
  namespace: team-a
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    limits.cpu: "20"
    limits.memory: 40Gi
    persistentvolumeclaims: "10"
    requests.storage: 100Gi
    pods: "50"
    services.loadbalancers: "2"
    count/deployments.apps: "20"

```

Quota on `services.loadbalancers` is worth setting: each one is a cloud load balancer with a bill (Chapter 11), and it is the cheapest way for a team to accidentally spend money.

The interaction that breaks existing workloads: once a quota on `requests.cpu` exists in a namespace, **every pod must specify that request** or creation is rejected. Adding a quota to a namespace full of pods that never set requests will block all future deployments there.

Always pair it with a LimitRange so existing manifests keep working:

```

apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
  namespace: team-a
spec:
  limits:
    - type: Container
      default:                                     # applied as limits if unset
        cpu: 500m
        memory: 512Mi
      defaultRequest:                             # applied as requests if unset
        cpu: 50m
        memory: 128Mi
      max:
        cpu: "4"
        memory: 8Gi
      min:
        cpu: 10m
        memory: 16Mi

```

This also eliminates BestEffort pods across the namespace, which is Chapter 20's eviction ranking problem solved by default.

Check usage:

```
kubectl describe resourcequota -n team-a
```

Quota is also **scoped**, which is useful for separating long-running from batch workloads:

```
spec:
  scopeSelector:
    matchExpressions:
      - operator: In
        scopeName: PriorityClass
        values: ["high-priority"]
```

A namespace template

What every tenant namespace should get, ideally generated automatically (Kyverno's generation rules, from Chapter 24, do this well):

1. **ResourceQuota** — capacity ceiling.
2. **LimitRange** — defaults so quota does not break things.
3. **NetworkPolicy** — default-deny ingress, plus DNS egress.
4. **Pod Security Standards labels** — `enforce=baseline` or `restricted`.
5. **RBAC** — a RoleBinding granting the team `edit` in their namespace only.
6. **Ownership labels** — team, cost centre, contact.

```
kubectl create rolebinding team-a-edit --clusterrole=edit --group=team-a -n team-a
```

Note `edit` rather than `admin`: `admin` includes the ability to create RoleBindings, which lets a tenant grant themselves anything else the cluster permits within that namespace.

What tenants must not have

- `cluster-admin`, obviously.
- **ClusterRole creation** or `escalate/bind` — self-escalation.
- **CRD creation** — `cluster-global`, and CRDs change API behaviour for everyone.
- **Node access** — `nodes/proxy`, or the ability to create privileged pods, `hostPath` mounts, or host networking. Pod Security Standards is what enforces this.
- **Webhook registration** — a tenant webhook with `failurePolicy: Fail` can break the cluster (Chapter 24).

The pattern to internalise: `create pods` **is node-level power unless admission control constrains it**. RBAC alone does not make a tenant safe.

Stronger isolation

When cooperative multi-tenancy is not enough, in increasing order of separation:

Node isolation. Dedicate nodes per tenant with taints and node affinity. Removes the shared kernel problem between tenants, at the cost of poorer bin-packing.

```
kubectl taint nodes NODE tenant=team-a:NoSchedule
kubectl label nodes NODE tenant=team-a
```

Sandboxed runtimes. gVisor or Kata Containers via RuntimeClass, so a container escape does not reach the host kernel:

```
apiVersion: node.k8s.io/v1
kind: RuntimeClass
metadata:
  name: gvisor
handler: runsc
---
spec:
  runtimeClassName: gvisor
```

This is the real answer for running untrusted code — customer-submitted workloads, CI for arbitrary pull requests, anything you did not write.

Virtual clusters. vCluster runs a tenant control plane (API server, scheduler) inside a namespace of a host cluster. Tenants get their own CRDs, ClusterRoles and API server, while pods actually run on the host's nodes. A genuinely good middle ground: much stronger isolation than namespaces, far cheaper than separate clusters.

Separate clusters. Complete isolation, complete duplication of cost and operational effort. The right answer for hostile tenants, hard regulatory boundaries, or when blast radius must be absolute.

Cost attribution

Multi-tenancy raises "who is spending what". Requests are the basis — a namespace's cost is roughly its share of node cost weighted by requests, which is why over-requesting is expensive even when nothing is used.

OpenCost and Kubecost do this properly (Chapter 26). Enforce ownership labels by policy so the attribution has something to attribute to:

```
validate:
  message: "All workloads must carry a team label."
  pattern:
    metadata:
      labels:
        team: "?*"
```

Try it

Create a tenant namespace with the full template:

```
kubectl apply -f examples/manifests/28-tenant.yaml
```

```
kubectl describe resourcequota -n team-a
```

See the LimitRange supply defaults to a pod that specifies nothing:

```
kubectl run defaulted -n team-a --image=busybox:1.37 --restart=Never --command --
sleep 300
```

```
sleep 5 && kubectl get pod defaulted -n team-a -o
jsonpath='{requests={.spec.containers[0].resources.requests}
limits={.spec.containers[0].resources.limits}}{"\n"}'
```

Requests and limits it never asked for, and therefore **not** BestEffort:

```
kubectl get pod defaulted -n team-a -o jsonpath='{qos={.status.qosClass}}{"\n"}'
```

Now hit the quota. Ask for more than the namespace permits:

```
kubectl run toobig -n team-a --image=busybox:1.37 --restart=Never --
overrides='{spec:{containers:[{"name":"toobig","image":"busybox:
1.37","command":["sleep","300"],"resources":{"requests":
{"cpu":"50","memory":"100Gi"},"limits":{"cpu":"50","memory":"100Gi"}}}]}}'
```

```
Error from server (Forbidden): pods "toobig" is forbidden:
[maximum cpu usage per Container is 2, but limit is 50,
maximum memory usage per Container is 4Gi, but limit is 100Gi]
```

Rejected at admission with exact numbers — and note *which* object caught it: the **LimitRange's** `max`, not the `ResourceQuota`, because a per-container ceiling is checked before the namespace total. The two work together, and reading the message tells you which limit you hit.

Check that a tenant cannot escalate:

```
kubectl auth can-i create clusterrolebindings --as=system:serviceaccount:team-
a:default
kubectl auth can-i get secrets -n default --as=system:serviceaccount:team-a:default
```

Clean up:

```
kubectl delete namespace team-a
```

Takeaways

- Namespaces scope names, RBAC, quotas and policy. They provide **no kernel, network, resource or control-plane isolation**.
- Suitable for **cooperative** tenants. Hostile or untrusted workloads need node isolation, sandboxed runtimes, virtual clusters or separate clusters.
- A ResourceQuota forces every pod to specify the quotaed resource**. Always pair it with a `LimitRange`, which also eliminates `BestEffort` pods.
- Quota `services.loadbalancers` — each one costs money.
- Grant tenants `edit`, not `admin`; `admin` lets them create `RoleBindings`.
- Never grant CRD creation, webhook registration, `escalate/bind`, or node access.

- `create pods` **is node-level power unless Pod Security Standards constrains it.**
- vCluster is a strong middle ground between namespaces and separate clusters.
- Attribute cost by requests, and enforce ownership labels so there is something to attribute to.

Previous: Chapter 27 — Deployment strategies and GitOps · Next: Chapter 29 — Packaging: Helm and Kustomize

Chapter 29 — Packaging: Helm and Kustomize

Raw manifests stop scaling the moment you need the same application in staging and production with different replica counts, image tags and resource limits. Copying YAML and editing three values is how drift starts.

Two tools solve this differently, and the difference is worth understanding before choosing.

Kustomize — overlays, no templating

Built into `kubectl`. You write plain, valid YAML and describe **patches** on top of it.

```
manifests/
  base/
    kustomization.yaml
    deployment.yaml
    service.yaml
  overlays/
    staging/kustomization.yaml
    production/kustomization.yaml
```

```
# base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
commonLabels:
  app.kubernetes.io/name: pingd
```

```
# overlays/production/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: production
resources:
  - ../../base
replicas:
  - name: pingd
    count: 10
images:
  - name: pingd
    newName: ghcr.io/org/pingd
    newTag: 1.4.2
patches:
  - target:
      kind: Deployment
      name: pingd
    patch: |-
      - op: replace
        path: /spec/template/spec/containers/0/resources/limits/memory
        value: 1Gi
```

```
kubectl apply -k overlays/production
```

```
kubectl kustomize overlays/production          # render without applying
```

The strength: the base is a real, valid manifest. You can `kubectl apply -f base/deployment.yaml` and it works. Your editor validates it, `kubectl explain` applies, and diffs are readable.

The weakness: only what Kustomize can express. Conditionals, loops and computed values are absent by design. "Deploy an Ingress only in production" means a separate resource in the production overlay, not an `{{ if }}`.

Two features worth knowing:

`configMapGenerator` appends a content hash to the ConfigMap name, so changing config changes the name, which changes the pod template, which **triggers a rollout automatically** — solving Chapter 7's "config changed but nothing restarted" problem cleanly:

```
configMapGenerator:
- name: pingd-config
  literals:
  - LOG_LEVEL=INFO
```

`components` are reusable, optional overlay fragments — the closest Kustomize gets to conditionals.

Helm — templating and releases

```
pingd/
Chart.yaml
values.yaml
templates/
  deployment.yaml
  service.yaml
  _helpers.tpl
```

```
# templates/deployment.yaml
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    metadata:
      annotations:
        checksum/config: {{
include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
    spec:
      containers:
        - name: api
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}"
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
{{- if .Values.ingress.enabled }}
# ... an Ingress, only when enabled
{{- end }}
```

```
helm install pingd ./pingd -f values-production.yaml
helm upgrade pingd ./pingd --set image.tag=1.4.3
helm rollback pingd 3
helm template pingd ./pingd -f values-production.yaml    # render locally
helm diff upgrade pingd ./pingd                          # plugin, very worth
installing
```

The strengths: real conditionals and loops; a **package** others can install with one command; release history and `helm rollback`; and a huge ecosystem — most third-party software ships a chart.

The weaknesses, stated plainly:

- **Templates are not YAML.** They are Go text templates that produce YAML, so indentation bugs are common, editors cannot validate them, and errors surface as unhelpful parse failures. `nindent` exists because of this.
- **Values files become sprawling.** A mature chart has hundreds of options, most unused, and understanding one requires reading the templates.
- **Debugging is indirect.** `helm template` is essential; without it you are guessing.
- **Release state lives in cluster Secrets**, which can drift from git and complicates GitOps.

Choosing

Kustomize for your own applications, especially with GitOps. The output is auditable, diffs are meaningful, and you never debug a template.

Helm for distributing software to others, and for consuming third-party software — which is most of the ecosystem.

Both together is common and works well: consume third-party charts with Helm, manage your own manifests with Kustomize. Both Argo CD and Flux support rendering a Helm chart and then applying Kustomize patches on top, which lets you fix a chart's shortcomings without forking it.

A pragmatic middle path some teams take: use `helm template` to render third-party charts to plain YAML, commit that, and manage everything with Kustomize. You lose `helm upgrade` and gain a fully auditable, diffable repository.

Practices that avoid pain

Pin chart versions. `helm install foo/bar` without `--version` takes whatever is current — the same mutable-tag problem as `:latest`.

```
helm install monitoring prometheus-community/kube-prometheus-stack --version 66.2.1
```

Render and review before upgrading. `helm diff upgrade` shows what will change; a chart version bump can rewrite far more than you expect, including CRDs.

Never `kubectl edit` a Helm-managed resource. The next `helm upgrade` reverts it, and your change is invisible in the meantime. Same principle as Chapter 2's imperative/declarative conflict.

CRDs are a Helm weak spot. `helm install` creates CRDs from `crds/`, but `helm upgrade` does **not** update them, and `helm uninstall` does not remove them. CRD upgrades are a manual step, and getting this wrong breaks operators in confusing ways. Read the chart's upgrade notes.

Keep values files small and per-environment. A single values file with every option set is unreadable; rely on chart defaults and override only what differs.

Do not template what you can patch. If a value differs in one environment, a Kustomize patch is clearer than a new chart parameter that must be threaded through templates.

The image tag question

Both tools face Chapter 27's unsolved problem: **something must update the image tag** for a new build.

- **CI writes to the manifests repo** — explicit, auditable, and the commit is the deployment record. Most common with GitOps.
- **Argo CD Image Updater / Flux image automation** — a controller watches the registry and commits the update. Less pipeline code, more moving parts.
- `helm upgrade --set image.tag=...` **from CI** — simple, and it puts cluster credentials in CI and leaves git out of date.

The first is the one to prefer if you have GitOps at all.

Try it

Kustomize needs nothing installed. Build a base and two overlays:

```
mkdir -p /tmp/kdemo/base /tmp/kdemo/overlays/production
```

```
cp examples/manifests/01-deployment.yaml examples/manifests/02-service.yaml /tmp/kdemo/base/
```

```
cat > /tmp/kdemo/base/kustomization.yaml <<'EOF'
resources:
- 01-deployment.yaml
- 02-service.yaml
EOF
```

```
cat > /tmp/kdemo/overlays/production/kustomization.yaml <<'EOF'
resources:
- ../../base
namePrefix: prod-
replicas:
- name: pingd
  count: 5
images:
- name: pingd
  newTag: "1.4.2"
EOF
```

Render it without applying anything:

```
kubectl kustomize /tmp/kdemo/overlays/production | grep -E 'name:|replicas:|image:' | head -12
```

Five replicas, a `prod-` prefix and a pinned tag, from a base that was never modified.

Compare against the base:

```
diff <(kubectl kustomize /tmp/kdemo/base) <(kubectl kustomize /tmp/kdemo/overlays/production) | head -20
```

That diff is the whole argument for overlays: you can see precisely what an environment changes.

Now see what a Helm chart renders to, without installing it:

```
helm template demo oci://registry-1.docker.io/bitnamicharts/nginx --version 18.2.5 2>/dev/null | head -30 || echo "(helm not installed - skip)"
```

Clean up:

```
rm -rf /tmp/kdemo
```

Takeaways

- **Kustomize patches valid YAML; Helm templates text.** That difference determines everything else about how each fails.
- Kustomize's base is a real manifest you can apply and validate. Its limits are deliberate — no conditionals or loops.
- `configMapGenerator` hashes content into the name, which **triggers a rollout on config change** — the clean fix for Chapter 7's problem.
- Helm gives conditionals, packaging, release history and the ecosystem. It costs you non-YAML templates, sprawling values files, and indirect debugging.
- Use **Kustomize for your own apps, Helm for third-party software**. Combining them is normal.
- **Pin chart versions**; `helm diff upgrade` before upgrading.
- **Helm does not upgrade CRDs.** That is a manual step and a common source of broken operators.
- Never `kubectl edit` a Helm- or Kustomize-managed resource; the next apply reverts it.
- Something still has to update the image tag — prefer CI committing to the manifests repo.

Previous: Chapter 28 — Multi-tenancy · Next: Chapter 30 — Operators and CRDs

Chapter 30 — Operators and CRDs

Chapter 2 described the control loop: observe desired state, observe actual state, close the gap. Custom resources let you use that machinery for **your own** concepts, and operators are the controllers that do it.

This is the mechanism behind most of the Kubernetes ecosystem — cert-manager, Prometheus Operator, database operators, Argo CD. It is also the point at which people start building things they should not.

Custom Resource Definitions

A CRD teaches the API server a new type:

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: pingdservices.example.com
spec:
  group: example.com
  names:
    kind: PingdService
    plural: pingdservices
    singular: pingdservice
    shortNames: ["pds"]
  scope: Namespaced
  versions:
    - name: v1alpha1
      served: true
      storage: true
      subresources:
        status: {}
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              required: ["replicas"]
              properties:
                replicas:
                  type: integer
                  minimum: 1
                  maximum: 100
                version:
                  type: string
            status:
              type: object
              properties:
                readyReplicas: {type: integer}
  additionalPrinterColumns:
    - name: Replicas
      type: integer
      jsonPath: .spec.replicas
```

Once applied, that type behaves like any built-in:

```
kubectl get pingdservices
kubectl explain pingdservice.spec
kubectl get pds -o yaml
```

RBAC applies, admission applies, `kubectl explain` works from your schema, and it is stored in etcd. **You have extended the API, not built something alongside it** — that is the whole appeal.

Details that matter:

- **A schema is effectively mandatory.** Without validation you get a typed object with untyped contents, and errors surface at runtime instead of at apply.
- `subresources: status: {}` separates spec from status, so a controller updating status cannot accidentally modify spec, and RBAC can be granted separately. Always enable it.
- `additionalPrinterColumns` makes `kubectl get` readable — cheap and consistently appreciated.
- **Versioning is real work.** Once `v1alpha1` has users, moving to `v1beta1` needs a conversion strategy, possibly a conversion webhook. Design the schema as though changing it is expensive, because it is.
- **CRDs are cluster-scoped.** Installing one affects every namespace, which is why tenants must not be allowed to create them (Chapter 28).

What an operator is

A controller that reconciles your custom resource. The loop is exactly Chapter 2's:

```
watch PingdService objects
  → read spec (desired)
  → read actual (Deployments, Services, external systems)
  → take one step to close the gap
  → update status
  → repeat
```

The genuine value is **encoding operational knowledge as software**. A database operator does not merely create a StatefulSet; it handles failover, backup scheduling, point-in-time recovery, version upgrades, connection routing and replica scaling. That accumulated knowledge *is* the product, and it is why Chapter 15 recommended an operator over a hand-written StatefulSet for databases.

Building one

Kubebuilder / controller-runtime (Go) — the reference approach, used by most serious operators:


```
func (r *PingdServiceReconciler) Reconcile(ctx context.Context, req ctrl.Request)
(ctrl.Result, error) {
    var svc examplev1alpha1.PingdService
    if err := r.Get(ctx, req.NamespacedName, &svc); err != nil {
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }

    desired := buildDeployment(&svc)
    if err := ctrl.SetControllerReference(&svc, desired, r.Scheme); err != nil {
        return ctrl.Result{}, err
    }
    if err := r.Patch(ctx, desired, client.Apply, client.ForceOwnership,
        client.FieldOwner("pingd-operator")); err != nil {
        return ctrl.Result{}, err
    }

    svc.Status.ReadyReplicas = observed.Status.ReadyReplicas
    return ctrl.Result{}, r.Status().Update(ctx, &svc)
}
```

Operator SDK wraps Kubebuilder and adds Ansible and Helm-based operators — useful when the logic is "install this chart with these values".

Metacontroller / kopf (Python) / shell-operator — lighter options when the logic is simple.

The rules that make a reconciler correct

These are where most bespoke operators go wrong:

Be idempotent. `Reconcile` will be called repeatedly for the same object, on a resync timer, after restarts, and on unrelated events. Running it twice must be identical to running it once.

Be level-triggered, not edge-triggered. Read current state; never assume you saw the previous event. Chapter 2's point, and the thing that makes controllers robust.

Set ownerReferences. This is what makes garbage collection delete your children when the parent goes (Chapter 2). Forgetting it leaks resources permanently.

Report through `status.conditions`. Use the standard shape — `type`, `status`, `reason`, `message`, `observedGeneration`. Every debugging tool and every human knows how to read it.

Never block. A `Reconcile` that waits on a slow external call stalls the queue for every object. Return and requeue instead:

```
return ctrl.Result{RequeueAfter: 30 * time.Second}, nil
```

Use finalizers only when you must, and make them robust. A finalizer whose controller is gone blocks deletion forever, which is Chapter 21's stuck-namespace problem.

Handle conflicts. Optimistic concurrency (Chapter 2) means updates fail if something else wrote first. Retry on conflict; do not treat it as an error.

Should you build one?

Usually not. The honest test:

Build an operator when:

- You are operating stateful, complex software where the operational knowledge is real — failover, backups, upgrades.
- You need to reconcile external systems (cloud resources, DNS, certificates) with cluster state.
- The same runbook is executed often enough that automating it saves genuine time.
- You are distributing software others will run.

Do not build one when:

- A Helm chart or Kustomize overlay would do. This covers most cases.
- You want to abstract a Deployment behind a friendlier name. You have added a component, a schema, RBAC, a release process and a failure mode, and saved twenty lines of YAML.
- Nobody will maintain it. An unmaintained operator is worse than no operator: it holds finalizers, blocks upgrades, and eventually breaks against a newer API.

An operator is a **distributed system you now own**. It runs continuously, has privileged access, and can damage the cluster if it misbehaves — a buggy reconcile loop creating objects in a hot cycle is an effective denial of service against your own API server, and it happens.

Before building, check [OperatorHub](#) and the CNCF landscape. The mature operators for Postgres, Kafka, Redis, Elasticsearch and certificates represent many person-years each.

Consuming operators

Which is what most people should do.

Evaluate before adopting: is it maintained, does it have a CRD versioning story, what RBAC does it demand (many want cluster-admin), how does it upgrade, and what happens if it stops running?

That last question matters most. A well-built operator's absence should be *inert* — existing workloads keep running, and only changes stop. An operator whose absence breaks running workloads is a single point of failure you have installed voluntarily.

Watch out for CRD upgrades. As Chapter 29 noted, Helm does **not** upgrade CRDs on `helm upgrade`. Operator upgrades frequently require applying new CRDs manually first, and skipping it produces confusing failures where new fields are silently dropped.

Uninstalling is harder than installing. Removing an operator leaves its CRDs, and deleting a CRD **deletes every object of that type** — which for a database operator means deleting your databases. Read the uninstall documentation before you need it.

Try it

Create a CRD and use it as a first-class API object — no controller needed to see the machinery:

```
kubectl apply -f examples/manifests/30-crd.yaml
```

```
kubectl api-resources | grep example.com
```

Your type is now in the API. It has documentation generated from your schema:

```
kubectl explain pingdservice.spec
```

Create one:

```
kubectl apply -f examples/manifests/30-cr.yaml && kubectl get pds
```

The custom printer columns show up in `kubectl get`. Now watch the schema reject invalid input — this is validation you got for free:

```
kubectl apply -f - <<'EOF'
apiVersion: example.com/v1alpha1
kind: PingdService
metadata:
  name: invalid
spec:
  replicas: 500
EOF
```

Rejected, because the schema caps `replicas` at 100. And a missing required field:

```
kubectl apply -f - <<'EOF'
apiVersion: example.com/v1alpha1
kind: PingdService
metadata:
  name: incomplete
spec:
  version: "1.0"
EOF
```

Nothing is running a controller, so nothing happens beyond storage — which is precisely the point: **a CRD is an API, and an operator is what gives it behaviour.**

Clean up — **deleting a CRD deletes every object of that type:**

```
kubectl delete -f examples/manifests/30-crd.yaml
```

Takeaways

- A CRD extends the API server: RBAC, admission, `kubectl explain` and etcd storage all apply to your type.
- Always define a **schema**, enable the **status subresource**, and add `additionalPrinterColumns`. Treat versioning as expensive from day one.
- CRDs are **cluster-scoped** — never let tenants create them.

- An operator is a controller for your CRD. The value is **encoded operational knowledge**, not abstraction.
 - Reconcilers must be **idempotent, level-triggered, non-blocking**, set `ownerReferences`, report via `status.conditions`, and retry on conflict.
 - **Most teams should not build one.** If Helm or Kustomize would do, use those.
 - An operator is a privileged distributed system you now own and must maintain.
 - When adopting one, ask what happens if it stops — its absence should be inert.
 - **Helm does not upgrade CRDs, and deleting a CRD deletes all its objects.**
-

Previous: Chapter 29 — Packaging: Helm and Kustomize · Next: Chapter 31 — The ecosystem

Chapter 31 — The ecosystem

Kubernetes is deliberately incomplete. It defines interfaces — for containers, networking, storage, and increasingly for gateways and policy — and expects implementations to be plugged in. That is why "install Kubernetes" is never the end of the work, and why the landscape has several hundred projects in it.

This chapter is a map, with opinions.

The interfaces

CRI (Container Runtime Interface) — how the kubelet talks to a runtime. **containerd** is the default nearly everywhere; **CRI-O** is Kubernetes-only and the default on OpenShift. Dockershim was removed in v1.24; images are unaffected because they are OCI images either way (Docker book, Chapter 31).

CNI (Container Network Interface) — pod networking. Your choice determines whether NetworkPolicy works at all (Chapter 14):

CNI	Notes
Cilium	eBPF-based; can replace kube-proxy; L7 policy, DNS-based egress, Hubble observability. The current default recommendation.
Calico	Mature, widely deployed, good policy support, optional eBPF mode.
Flannel	Simple, and does not enforce NetworkPolicy .
AWS VPC CNI / Azure CNI	Pods get real VPC addresses. Native integration; watch IP exhaustion.

CSI (Container Storage Interface) — storage drivers, per Chapter 15.

Gateway API — the typed replacement for Ingress (Chapter 13).

Distributions

Managed control planes — EKS, GKE, AKS, DigitalOcean, Linode. The control plane, etcd backups, and control-plane upgrades are someone else's problem. **This is the right default for most organisations**; the differentiating value of running your own control plane is almost always negative.

GKE Autopilot and EKS Auto Mode go further and manage nodes too, charging per pod. For teams without platform engineers this is often the correct answer.

Lightweight — k3s (a single binary, production-capable, excellent for edge and small clusters), k0s, MicroK8s. k3s in particular deserves consideration where a full cluster is overkill but you want the real API.

Local — kind (used throughout this book), minikube, k3d, Docker Desktop's built-in cluster. kind is the best fit for CI because it is fast and disposable.

Enterprise — OpenShift (an opinionated platform with its own build and routing systems), Rancher, Tanzu. You are buying support, integration and a security posture.

Service mesh

Sidecar or per-node proxies giving you mTLS, retries, timeouts, circuit breaking, traffic splitting and detailed L7 telemetry.

Istio — the most capable and the most complex. Ambient mode removes per-pod sidecars, which materially reduces the resource overhead and operational weight that made classic Istio hard to justify.

Linkerd — deliberately simpler, Rust micro-proxy, very low overhead. If you want mTLS and golden-signal metrics without a project, this is the one.

Cilium Service Mesh — mesh features in the CNI, no sidecars.

Should you? A mesh solves real problems: mTLS between services, retries and timeouts without touching application code, and L7 metrics for everything. It also adds a proxy to every request path, meaningful resource overhead, and a substantial new failure domain — mesh misconfiguration causes outages that are genuinely hard to diagnose.

Adopt one when you have a **specific** need: a compliance requirement for mTLS, or enough services that per-application retry logic is unmanageable. "Microservices best practice" is not a reason. Note also that the Gateway API (Chapter 13) now covers traffic splitting without a mesh, which removes one of the historically common justifications.

What you will actually install

A realistic list for a production cluster, roughly in order of adoption:

Need	Common choice
Metrics, alerting, dashboards	kube-prometheus-stack
Basic resource metrics / HPA	metrics-server (separate!)
Log aggregation	Loki, or a managed service
Ingress	ingress-nginx, or a Gateway API implementation
Certificates	cert-manager — install on day one
Secrets	External Secrets Operator, or Sealed Secrets
Deployment	Argo CD or Flux
Policy	Kyverno, or built-in ValidatingAdmissionPolicy
Node autoscaling	Karpenter (AWS) or Cluster Autoscaler
Backup	Velero
Cost	OpenCost / Kubecost
Image scanning	Trivy Operator

Two notes. **cert-manager** is the least controversial thing on that list — automatic certificate issuance and renewal, and everything else assumes it. And **Velero** is the one people install after they needed it: cluster-level backup and restore of resources plus volume snapshots.

Choosing from the landscape

The CNCF landscape has hundreds of entries and is not a shortlist. A practical filter:

Prefer graduated or incubating CNCF projects. Sandbox projects may be excellent and may be abandoned in a year.

Check the maintenance signal, not the star count: recent releases, issue response, more than one active maintainer, and a security policy.

Understand the business model. Open-core projects have features you will eventually want behind a paid tier. That is legitimate; know it before standardising.

Prefer fewer components. Every addition is something to upgrade, monitor, secure and debug at 3am. The built-in `ValidatingAdmissionPolicy` (Chapter 24) instead of a policy engine, or Gateway API instead of a mesh, are real wins when they suffice.

Ask what happens when it stops. As Chapter 30 put it: a good component's absence should be inert.

Where things are heading

Stated with appropriate uncertainty:

Gateway API replacing Ingress for new deployments. Core is stable; adoption is broad.

eBPF everywhere. Cilium's approach — networking, policy, observability and mesh in the kernel without sidecars — is clearly the direction.

Sidecar-less meshes. Istio ambient and Cilium remove the per-pod proxy that made meshes expensive.

In-place pod resizing, which makes VPA usable without evictions (Chapter 25).

Supply chain enforcement becoming mandatory rather than good practice (Chapter 24).

WebAssembly as an additional workload type — smaller, faster to start, sandboxed by design. Real, interesting, still maturing, and a complement rather than a replacement.

The honest summary

Kubernetes is a platform for building platforms. The API and the control loop are excellent and stable. Everything above them is an ecosystem of varying quality that you must assemble, operate and keep current.

That assembly work is why "platform engineering" exists as a job, and why Chapter 1 argued against adopting Kubernetes without a present constraint. If you do have one, the ecosystem is a genuine asset — nearly every problem has a mature, well-understood solution. The skill is choosing few of them.

Try it

See the interfaces in use on your own cluster:

```
kubectl get nodes -o jsonpath='{range .items[*]}{.metadata.name}{ " runtime="}{.status.nodeInfo.containerRuntimeVersion}{ "\n"}{end}'
```

```
k8sbook-control-plane runtime=containerd://2.3.4
```

Which CNI is running:

```
kubectl get pods -n kube-system -o custom-columns='NAME:.metadata.name,IMAGE:.spec.containers[0].image' | grep -Ei 'cilium|calico|flannel|kindnet'
```

What API groups your cluster serves — this is the extension surface:

```
kubectl api-versions | sort | head -25
```

And which of those came from CRDs rather than core Kubernetes:

```
kubectl get crd -o custom-columns='NAME:.metadata.name,GROUP:.spec.group' --no-headers | head
```

On this book's cluster that shows the ingress controller's CRDs — everything else is built in.

Check what a managed cluster would hide from you:

```
kubectl get pods -n kube-system -l tier=control-plane
```

On EKS/GKE/AKS this returns nothing, because the control plane is not yours to see.

Takeaways

- Kubernetes defines interfaces (CRI, CNI, CSI, Gateway API) and expects implementations. "Install Kubernetes" is never the whole job.
- **Your CNI choice determines whether NetworkPolicy works.** Flannel does not enforce it.
- **Managed control planes are the right default.** Running your own rarely differentiates anything.
- k3s is a strong middle path; kind is the best local and CI cluster.
- A service mesh solves real problems and adds a proxy to every request and a large failure domain. Adopt for a specific need — Gateway API now covers traffic splitting without one.
- cert-manager on day one. metrics-server is **separate** from kube-prometheus-stack.
- Filter the landscape by maintenance signal, business model, and **what happens when it stops**. Prefer fewer components.
- Direction of travel: Gateway API, eBPF, sidecar-less meshes, in-place resizing, mandatory supply chain.

Previous: [Chapter 30 — Operators and CRDs](#) · Next: [Chapter 32 — Anti-patterns](#)

Chapter 32 — Anti-patterns

The catalogue, each with the reason and the alternative. Most appear earlier in the book; collecting them makes the review checklist.

Ordered roughly by how much damage they cause.

Workloads

No resource requests

Why it's wrong. The pod is `BestEffort` and **evicted first** under node pressure. It is also invisible to the scheduler's accounting, so it contributes to overcommit without being counted, and an HPA targeting CPU utilisation cannot work at all.

Fix. Set requests on everything; use a `LimitRange` to make it the namespace default. (*Ch 8, 20, 28*)

Liveness probe that checks dependencies

```
livenessProbe:
  httpGet: { path: /health-including-database }  #
```

Why it's wrong. A database blip fails liveness on every replica of every service at once, Kubernetes restarts your entire fleet, and the reconnection storm delays the recovery that was already happening.

Fix. Liveness checks only itself. Dependencies belong in **readiness**. (*Ch 9*)

`localhost` in a probe

Why it's wrong. Inside a container `localhost` resolves to `:::1`. An IPv4-only server refuses the connection, the probe fails, and the container is restarted while working perfectly.

Fix. `127.0.0.1`. (*Ch 9, 19*)

No readiness probe

Why it's wrong. Pods receive traffic the moment the process starts, before they can serve. Worse, **rollouts have nothing to gate on**, so a completely broken version rolls out to 100%.

Fix. A readiness probe is what makes rolling updates safe. (*Ch 6, 9*)

`timeoutSeconds` left at the default

Why it's wrong. The default is **1 second**. A service that occasionally takes 1.2 s to answer under load gets restarted, under load, making the load worse.

Fix. Set it deliberately, with `failureThreshold: 3` at minimum. (Ch 9)

Bare pods

Why it's wrong. Nothing watches them. If it dies it stays dead; if its node dies it is gone.

Fix. Deployment, StatefulSet, Job or DaemonSet. Bare pods are for debugging. (Ch 5)

StatefulSet for a stateless service

Why it's wrong. Slower rollouts, slower scaling, and ordering guarantees you are paying for and not using. A stuck pod blocks the entire rollout.

Fix. Deployment, unless you need stable identity, per-pod storage or ordered startup. (Ch 10)

A version label in the Deployment selector

```
selector:
  matchLabels:
    app: pingd
    version: "1.4.2"    #
```

Why it's wrong. `spec.selector` is **immutable**. Every release now requires deleting and recreating the Deployment.

Fix. Keep the selector minimal and permanent; put changing labels in the pod template. (Ch 6)

Overlapping selectors between Deployments

Why it's wrong. Each ReplicaSet adopts any pod matching its selector, so two Deployments continuously delete each other's pods. The symptom is endless pod churn with no obvious cause.

Fix. Selectors must be disjoint. (Ch 6)

Jobs without `ttlSecondsAfterFinished`

Why it's wrong. Completed pods are retained forever, and clusters accumulate thousands.

Fix. Set it on every Job. Add `activeDeadlineSeconds` for anything that could hang. (Ch 10)

CronJob with default `concurrencyPolicy`

Why it's wrong. The default is `Allow`. A run that takes longer than the interval overlaps with the next one — two copies of your nightly job writing to the same table.

Fix. `Forbid` or `Replace`, plus an explicit `timeZone`. (Ch 10)

Configuration

Config in env vars that you expect to update

Why it's wrong. **Environment variables freeze at container start.** Changing a ConfigMap has no effect until the pod restarts — possibly days later, during an unrelated incident, producing a version nobody deployed.

Fix. Mount as a volume (updates in ~1 min), or add a config checksum annotation to force a rollout. `subPath` mounts **never** update. (*Ch 7*)

Treating Secrets as secret

Why it's wrong. Base64 is an encoding. Anyone with `get secrets` in the namespace reads them, and `etcd` stores them unencrypted by default.

Fix. Encryption at rest, tight RBAC (`list` on Secrets reads them all), and an external secret store. (*Ch 7, 23*)

Secrets in git

Why it's wrong. Permanent, and a `git revert` does not unpublish them.

Fix. Sealed Secrets, SOPS, or the External Secrets Operator. Rotate anything committed. (*Ch 7, 27*)

Networking

Assuming namespaces isolate the network

Why it's wrong. Every pod can reach every pod in every namespace by default.

Fix. NetworkPolicy — in every namespace. (*Ch 14, 28*)

NetworkPolicy on a CNI that ignores it

Why it's wrong. The objects apply cleanly and do nothing. You believe you are protected and are not. Flannel does not enforce policy.

Fix. **Test enforcement** before relying on it. (*Ch 14*)

Egress policy without DNS

Why it's wrong. Name resolution silently breaks for the selected pods, and nobody connects the failure to the policy they just applied.

Fix. Always allow UDP **and** TCP port 53 to kube-dns. (*Ch 14*)

NetworkPolicy using the Service port

Why it's wrong. Policy is evaluated after DNAT, so it sees the **pod** port. Writing the Service port blocks everything.

Fix. Use the container's port. (*Ch 14*)

An app bound to 127.0.0.1

Why it's wrong. Reachable only inside its own pod. Probably the most common containerisation bug there is — everything looks correct and nothing can reach it.

Fix. Bind 0.0.0.0. (*Ch 19*)

A LoadBalancer Service per microservice

Why it's wrong. Each one is a real cloud load balancer with a monthly bill and an IP. Twenty services is twenty load balancers.

Fix. One Ingress or Gateway in front of many Services. (*Ch 11, 13*)

Relying on ClusterIP to balance gRPC

Why it's wrong. Load balancing is **per connection**. HTTP/2 and gRPC use one long-lived connection, which pins to a single pod forever.

Fix. Headless Service with client-side balancing, or a proxy that understands HTTP/2. (*Ch 11*)

Deployment**:latest, or any mutable tag**

Why it's wrong. You cannot tell what is running, cannot roll back, and replicas created at different times run different code. On kind/minikube it also makes `imagePullPolicy` default to `Always`, so your locally-loaded image is ignored.

Fix. Immutable tags, ideally digests, enforced at admission. (*Ch 17, 24; Docker book Ch 15*)

Default rollout strategy for a service under load

Why it's wrong. The default is 25%/25%, which **dips below capacity on every rollout**.

Fix. `maxSurge: 1, maxUnavailable: 0`. (*Ch 6, 27*)

replicas in a manifest managed by an HPA

Why it's wrong. Every `apply` or GitOps sync resets the count the HPA just calculated, and the two fight — or you get a server-side apply conflict.

Fix. Remove the field. (*Ch 6, 25*)

kubectl edit in production

Why it's wrong. Invisible to git, reverted by the next apply or by GitOps `selfHeal`, and absent from any review or audit trail.

Fix. Change the manifest. Keep a documented break-glass procedure for genuine emergencies. (*Ch 2, 27*)

kubectl rollout undo as the rollback procedure

Why it's wrong. It desynchronises the cluster from git; the next sync re-applies the broken version.

Fix. Emergency stop only, immediately followed by a git revert. (*Ch 6, 27*)

Expecting progressDeadlineSeconds to roll back

Why it's wrong. It stops a stalled rollout. It does **not** reverse it.

Fix. Argo Rollouts or Flagger for automated rollback on metrics. (*Ch 6, 27*)

Cluster**No PodDisruptionBudget**

Why it's wrong. A node drain can take every replica at once. This applies to infrastructure too — CoreDNS without a PDB is a cluster-wide outage waiting for a node upgrade.

Fix. `maxUnavailable: 1` on anything that matters. (*Ch 12, 25*)

A PDB that can never be satisfied

```
spec:
  minAvailable: 3           # with replicas: 3
```

Why it's wrong. Drains **block forever**, turning routine node maintenance into an incident.

Fix. `maxUnavailable: 1`, which scales with the Deployment. Alert on `disruptionsAllowed: 0`. (*Ch 25*)

Expecting the scheduler to rebalance

Why it's wrong. It never does. After a drain, all your replicas can sit on one node indefinitely — measured in Chapter 21.

Fix. `kubectl rollout restart` after node events; `topologySpreadConstraints` for placement. (Ch 3, 21, 25)

Webhook with `failurePolicy: Fail` and broad scope

Why it's wrong. When the webhook is down it blocks all pod creation cluster-wide — including the pods that would restore it.

Fix. Exclude `kube-system`, short timeouts, narrow matching, run it HA. (Ch 24)

Force-deleting a StatefulSet pod on an unreachable node

Why it's wrong. The container may still be running with the volume attached. Deleting the API object lets a replacement start — two writers, one volume, corrupted data.

Fix. Confirm the node is truly gone first. (Ch 17, 21)

Never upgrading

Why it's wrong. `kubeadm` certificates expire after **one year** and present as total authentication failure. Support windows are months, not years, and skipping minor versions is unsupported.

Fix. Regular upgrades; scan for removed APIs with `pluto/kubent` first. (Ch 21)

No etcd backup

Why it's wrong. `etcd` is the only state. Losing it loses the cluster's definition of itself.

Fix. Scheduled snapshots, **tested restores**, or a managed control plane. (Ch 21)

Security**Mounting the ServiceAccount token everywhere**

Why it's wrong. Every pod carries an API credential it does not need, free for anyone who achieves code execution.

Fix. `automountServiceAccountToken: false` unless the workload uses the API. The cheapest security win available. (Ch 23)

Every workload using the default ServiceAccount

Why it's wrong. No least privilege, and audit logs cannot attribute API calls to a workload.

Fix. One ServiceAccount per workload. (*Ch 23*)

Granting cluster-admin because something failed

Why it's wrong. It works, so it stays, forever.

Fix. `kubectl auth can-i` to find the missing permission and grant that. (*Ch 23*)

Letting tenants create pods without Pod Security Standards

Why it's wrong. `create pods` plus `hostPath: /` is host root. **RBAC alone does not make a tenant safe.**

Fix. PSS baseline minimum, restricted where possible. (*Ch 23, 28*)

Signing images without verifying them

Why it's wrong. Ceremony. A signature nothing checks provides nothing.

Fix. Verify at admission. (*Ch 24*)

Namespaces as a security boundary for untrusted tenants

Why it's wrong. Shared kernel, shared nodes, shared control plane, shared cluster-scoped resources.

Fix. Separate clusters, vCluster, or sandboxed runtimes for anything untrusted. (*Ch 28*)

Operations**No metrics-server, then wondering why the HPA does nothing**

Why it's wrong. The HPA reports `<unknown>` and never scales. It is not installed by default.

Fix. Install it — separately from kube-prometheus-stack. (*Ch 25, 26*)

Relying on `kubectl logs` after the fact

Why it's wrong. Pod logs are **deleted with the pod** — exactly when you want them.

Fix. Ship logs off-node. Export events too; they expire in about an hour. (*Ch 26*)

Monitoring stack with no requests and no PDB

Why it's wrong. It is `BestEffort`, so it is evicted first — precisely when the cluster is under pressure and you need it.

Fix. Requests and a PDB on your observability stack. (*Ch 20, 26*)

Alerting on CPU utilisation

Why it's wrong. Noise. A pod at 90% CPU serving fine is not an incident, and the alert that fires constantly is the alert nobody reads.

Fix. Alert on symptoms — error rate, latency, restarts, **CPU throttling ratio**, memory against limit. (*Ch 26*)

Building an operator instead of writing a Helm chart

Why it's wrong. You have added a privileged distributed system, a schema, a release process and a failure mode, to save some YAML.

Fix. Operators are for encoded operational knowledge. Use packaging tools for packaging. (*Ch 30*)

Adopting Kubernetes with no present constraint

Why it's wrong. A large permanent operational burden to solve problems you do not have, with engineering time going into the platform rather than the product.

Fix. Adopt for a specific constraint. A managed container service is often the better answer. (*Ch 1*)

The review checklist

For reviewing a workload manifest:

- [] Resource **requests** on every container; limits on memory
- [] Readiness probe, and a liveness probe that checks **only itself**
- [] Probes use `127.0.0.1`, a named port, and `timeoutSeconds` set deliberately
- [] `startupProbe` for slow starters
- [] Immutable image tag or digest; `imagePullPolicy` correct for your cluster
- [] `maxSurge: 1`, `maxUnavailable: 0`
- [] Minimal, permanent `selector` with no version label
- [] `securityContext`: non-root numeric UID, `allowPrivilegeEscalation: false`, `capabilities: drop: [ALL]`, `readOnlyRootFilesystem`, `seccompProfile: RuntimeDefault`
- [] Dedicated `ServiceAccount`, `automountServiceAccountToken: false` if unused
- [] `terminationGracePeriodSeconds` longer than the longest request; `preStop` sleep if rollouts drop connections

- [] PodDisruptionBudget with `maxUnavailable`
 - [] `topologySpreadConstraints` across nodes
 - [] Config from a mounted ConfigMap, or a checksum annotation to force rollouts
 - [] No `replicas` if an HPA owns it
 - [] Ownership labels for cost attribution
 - [] NetworkPolicy exists for the namespace, DNS allowed in egress
-

Previous: Chapter 31 — The ecosystem · Next: Appendix A — kubectl cheatsheet

Appendix A — kubectl cheatsheet

marks anything that deletes data or is hard to undo.

Triage sequence

```
kubectl get pods -o wide           # STATUS and READY are different
facts
kubectl describe pod POD          # includes EVENTS – usually the
answer
kubectl logs POD --previous; kubectl logs POD # try both
kubectl get events --sort-by=.lastTimestamp | tail -20
```

Getting and formatting

```
kubectl get pods -o wide
kubectl get pods -A
kubectl get pods -l app=pingd --show-labels
kubectl get pods --field-selector status.phase=Running
kubectl get pods --sort-by=.metadata.creationTimestamp
kubectl get pod POD -o yaml
kubectl get pods -o name           # feeds other commands

kubectl get pods -o custom-columns='NAME:.metadata.name,NODE:.spec.nodeName,QOS:.status.qosClass'
kubectl get pod POD -o jsonpath='{.spec.nodeName}{"\n"}'
kubectl get pods -o jsonpath='{range .items[*]}{.metadata.name}{ " "}{.status.podIP}{
"\n"}{end}'
```

Diagnostic one-liners

```
# why did it die
kubectl get pod POD -o jsonpath='{exit={.status.containerStatuses[0].lastState.terminated.exitCode} reason={.status.containerStatuses[0].lastState.terminated.reason} restarts={.status.containerStatuses[0].restartCount}}{"\n"}'

# scheduled or not (Pending triage)
kubectl get pod POD -o jsonpath='{.spec.nodeName}}{"\n"}'

# has my change been seen
kubectl get deploy NAME -o jsonpath='{gen={.metadata.generation} observed={.status.observedGeneration}}{"\n"}'

# controller opinion
kubectl get deploy NAME -o
jsonpath='{range .status.conditions[*]}{.type}={.status} {.reason}}{"\n"}{end}'

# service backends – check conditions, not just addresses
kubectl get endpointslice -l kubernetes.io/service-name=SVC \
  -o custom-columns='ADDR:.endpoints[*].addresses[0],READY:.endpoints[*].conditions
  .ready'

# node pressure
kubectl get nodes -o custom-columns='NAME:.metadata.name,MEM:.status.conditions[?
  (@.type=="MemoryPressure")].status,DISK:.status.conditions[?
  (@.type=="DiskPressure")].status,READY:.status.conditions[?
  (@.type=="Ready")].status'

# requests vs allocatable (what the scheduler sees)
kubectl describe node NODE | sed -n '/Allocated resources/,/Events/p'
```

Logs

```
kubectl logs -f --tail=100 --timestamps deploy/pingd
kubectl logs POD -c CONTAINER
kubectl logs POD --previous # the instance that crashed
kubectl logs -l app=pingd --prefix --tail=20
kubectl logs POD --since=10m
```

Getting inside

```

kubectl exec -it POD -- sh
kubectl exec POD -- env | sort

# no shell in the image (distroless):
kubectl debug POD --image=nicolaka/netshoot --target=CONTAINER -it -- bash
# then: ps -o pid,comm | ls /proc/1/root/ | cat /proc/1/environ | tr '\0' '\n'

# crash-looping (nothing to attach to):
kubectl debug POD --copy-to=probe --container=api -- sleep 3600

# the node itself ( privileged, host fs at /host):
kubectl debug node/NODE -it --image=nicolaka/netshoot -- chroot /host bash

kubectl cp POD:/path/file ./file # needs tar in the image
kubectl port-forward deploy/pingd 8080:8000 # bypasses Service, DNS, Ingress

```

Networking checks

```

kubectl run t --rm -it --restart=Never --image=nicolaka/netshoot -- bash
kubectl run t --rm -it --restart=Never --image=nicolaka/netshoot -- nslookup pingd
kubectl run t --rm -it --restart=Never --image=curlimages/curl:8.11.1 -- curl -sS h
ttp://pingd/

kubectl exec POD -- cat /etc/resolv.conf # 127.0.0.1? ndots:5?
kubectl debug POD --image=nicolaka/netshoot -it -- ss -tln # 0.0.0.0 or
127.0.0.1?
kubectl get networkpolicy -A

```

Workload management

```

kubectl apply -f manifest.yaml
kubectl apply -k overlays/production
kubectl diff -f manifest.yaml # before applying
kubectl apply -f manifest.yaml --dry-run=server

kubectl rollout status deployment/NAME --timeout=300s # exits non-zero on failure
kubectl rollout history deployment/NAME
kubectl rollout undo deployment/NAME # desyncs from git
kubectl rollout restart deployment/NAME # graceful restart / rebalance
kubectl rollout pause|resume deployment/NAME

kubectl scale deployment NAME --replicas=5
kubectl set image deployment/NAME api=IMAGE:TAG
kubectl set env deployment/NAME KEY=value
kubectl annotate deployment/NAME kubernetes.io/change-cause="Deploy 1.4.2
(a1b2c3d)"

```

Nodes

```

kubectl top nodes; kubectl top pods --containers      # needs metrics-server
kubectl cordon NODE
kubectl drain NODE --ignore-daemonsets --delete-emptydir-data
kubectl uncordon NODE                                # note: pods do NOT move back
kubectl taint nodes NODE key=value:NoSchedule
kubectl taint nodes NODE key-                        # remove

```

API discovery

```

kubectl api-resources
kubectl api-resources --namespaced=false
kubectl api-versions
kubectl explain deployment.spec.strategy
kubectl explain pod.spec.containers.resources --recursive

```

RBAC

```

kubectl auth can-i create deployments -n production
kubectl auth can-i --list --as=system:serviceaccount:NS:SA
kubectl auth can-i get secrets --as=system:serviceaccount:default:default
kubectl auth whoami

kubectl create rolebinding team-a-edit --clusterrole=edit --group=team-a -n team-a

```

Contexts and namespaces

```

kubectl config get-contexts
kubectl config current-context                      # check before every production
command
kubectl config use-context CLUSTER
kubectl config set-context --current --namespace=myapp

```

Cleanup

```

kubectl delete -f manifest.yaml
kubectl delete pod POD --force --grace-period=0    # risky for StatefulSets
kubectl delete pods --field-selector status.phase=Failed
kubectl delete namespace NS                        # deletes everything in it

```

Status reference

STATUS	Meaning
Pending + no nodeName	Scheduler could not place it
Pending + nodeName	Kubelet working: image pull, volumes, init
Running + 0/1	Readiness failing
ImagePullBackOff	Cannot fetch image
CreateContainerConfigError	Missing ConfigMap/Secret
CrashLoopBackOff or Error	Container keeps exiting — trust restart count
Terminating (stuck)	Finalizer, grace period, or gone node
Evicted	Node pressure

Exit codes

0 clean · 1 app error · 126 not executable · 127 not found · 137 **SIGKILL — OOM or ignored SIGTERM** · 139 SIGSEGV · 143 SIGTERM

Signals are 128 + N. **Exit 137 is a more reliable OOM signal than** reason: OOMKilled.

cgroup / node inspection

```
kubectl debug node/NODE -it --image=busybox:1.37 -- chroot /host sh
# then:
journalctl -u kubelet --since '15 min ago' | tail -50
crictl ps; crictl logs CONTAINER_ID
dmesg -T | grep -i 'memory cgroup out of memory'
df -h; free -m
cat /etc/kubernetes/manifests/kube-apiserver.yaml # static pod escape hatch
```

Control plane

```
kubectl get --raw='/livez'
kubectl get --raw='/readyz?verbose'
kubectl get pods -n kube-system -l tier=control-plane
kubectl get --raw "/api/v1/nodes/NODE/proxy/metrics/resource" | grep '^container'
kubeadm certs check-expiration # one-year cliff
```

[Back to contents](#) · [Next: Appendix B — Glossary](#)

Appendix B — Glossary

Admission controller — Code that inspects API requests after authorisation and before persistence. Mutating runs first, then validating. *(Ch 24)*

Allocatable — A node's capacity minus reservations for the kubelet and OS. What the scheduler actually uses. On kind it equals capacity; on production nodes it is 5–15% lower. *(Ch 20)*

Annotation — Non-identifying metadata, not queryable. Contrast **label**. *(Ch 4)*

BestEffort — The QoS class for pods with no requests or limits. **Evicted first**. *(Ch 8)*

Burstable — QoS class where some requests are set but do not equal limits. *(Ch 8)*

cgroup — The kernel mechanism enforcing resource limits. Memory limits kill; CPU limits throttle. *(Ch 8, 20)*

CNI — Container Network Interface. Gives pods IPs and routes pod-to-pod traffic without NAT. **Determines whether NetworkPolicy is enforced at all**. *(Ch 3, 14, 31)*

ConfigMap — Non-secret configuration. Consumed as env vars (**frozen at container start**) or files (**updated in ~1 min**). *(Ch 7)*

Container Runtime Interface (CRI) — How the kubelet talks to containerd or CRI-O. *(Ch 31)*

Control loop — Observe desired, observe actual, take one step to close the gap, repeat. **Level-triggered**, so it survives missed events and restarts. The core idea. *(Ch 2)*

CoreDNS — Cluster DNS, running as an ordinary Deployment — and therefore a cluster-wide failure domain. *(Ch 12)*

CRD — CustomResourceDefinition. Teaches the API server a new type. Cluster-scoped. *(Ch 30)*

CSI — Container Storage Interface. Storage drivers. *(Ch 15)*

DaemonSet — One pod per node, automatically. Infrastructure DaemonSets need tolerations: `[{operator: Exists}]` or they skip tainted nodes. *(Ch 10)*

Deployment — Manages ReplicaSets to run a stateless service with rollouts and rollbacks. `spec.selector` is **immutable**. *(Ch 6)*

Downward API — Exposes pod metadata and **the container's own resource limits** to the container. *(Ch 7)*

EndpointSlice — The actual list of a Service's backends. Not-ready pods stay listed with `conditions.ready: false`. *(Ch 11)*

etcd — The only stateful component; every object is a key. Fails via slow disk (cluster-wide latency), a 2 GiB quota (all writes rejected), or quorum loss. *(Ch 3, 21)*

Eviction — The kubelet deleting pods to recover node resources. Ranks by QoS, then usage above requests, then priority. Distinct from **preemption**. *(Ch 20)*

Ephemeral container — A debugging container injected into a running pod. **Cannot be removed**. *(Ch 18)*

Finalizer — A key blocking deletion until a controller does cleanup. A stale one blocks deletion forever. *(Ch 21)*

Gateway API — The typed, role-separated successor to Ingress. Core is stable. *(Ch 13)*

Guaranteed — QoS class where requests equal limits for CPU and memory. Evicted last; unlocks exclusive CPU pinning. *(Ch 8)*

HPA — HorizontalPodAutoscaler. Targets a percentage of the **CPU request**, not a core. Needs metrics-server. Remove replicas from manifests it manages. *(Ch 25)*

Ingress — HTTP routing to Services. Does nothing without a controller. Everything beyond host/path/TLS is **controller-specific annotations, silently ignored when misspelled**. *(Ch 13)*

Init container — Runs to completion before app containers, in order. May use a different image. *(Ch 5)*

Job — Runs pods until N complete. Always set `ttlSecondsAfterFinished`. *(Ch 10)*

kube-proxy — Implements Service ClusterIPs as node-level rules. Broken on one node: that node cannot reach any ClusterIP but *can* reach pod IPs. *(Ch 3, 19)*

kubelet — The node agent. Creates containers, **runs probes locally**, reports status, evicts under pressure, runs static pods. *(Ch 3)*

Label / selector — Key-value identification, and the queries over it. **The coupling between nearly every Kubernetes object**. A selector matching nothing is never an error. *(Ch 4)*

LimitRange — Namespace defaults, minimums and maximums for container resources. Pair with ResourceQuota so existing manifests keep working. *(Ch 8, 28)*

Liveness probe — "Is it wedged?" Failing **restarts** the container. **Must not check dependencies**. *(Ch 9)*

Namespace — Scopes names, RBAC, quotas and policy. **Not a security, network or kernel boundary**. *(Ch 4, 28)*

NetworkPolicy — Additive allow-lists. Selecting a pod with no rules denies that direction. Uses the **pod** port, not the Service port. Requires DNS to be allowed in egress. *(Ch 14)*

Operator — A controller reconciling a custom resource. Value is **encoded operational knowledge**. *(Ch 30)*

ownerReferences — What drives cascading deletion. Deleting the middle of a chain does not stick. *(Ch 2)*

PDB — PodDisruptionBudget. Constrains **voluntary** disruption only (drains, autoscaler consolidation) — never node failure. `minAvailable == replicas` blocks drains forever. *(Ch 25)*

Pod — Containers sharing a network namespace, IPC and volumes, co-scheduled and co-terminated. **Not** a shared PID namespace by default. The unit of scheduling. *(Ch 5)*

Pod Security Standards — `privileged/baseline/restricted`, applied by namespace label with `enforce/warn/audit` modes. *(Ch 23)*

Preemption — The scheduler evicting lower-priority pods to make room. Distinct from **eviction**. *(Ch 20)*

PV / PVC — PersistentVolume (actual storage) and PersistentVolumeClaim (the request). Pods reference PVCs. *(Ch 15)*

QoS class — Derived, not declared: Guaranteed, Burstable, BestEffort. Determines eviction order. *(Ch 8)*

Readiness probe — "Can it serve now?" Failing marks it **not-ready in EndpointSlices**. **Gates rollouts** — without one, a broken version rolls out completely. (Ch 9)

ReplicaSet — Keeps N pods alive. Managed by a Deployment; old ones are retained at zero replicas so rollback is instant. (Ch 6)

Requests vs limits — **Requests are for the scheduler; limits are for the kernel**. Scheduling never considers actual usage. (Ch 8)

ResourceQuota — Caps a namespace's total consumption. **Once it exists, every pod must specify the quotaed resource**. (Ch 28)

Secret — Base64-**encoded**, not encrypted; unencrypted in etcd by default. `list` on Secrets reads them all. (Ch 7, 23)

Service — Stable name and virtual IP over a changing set of pods, matched **by label selector**. Balances **per connection**, which is why gRPC pins to one pod. (Ch 11)

ServiceAccount — A pod's identity. Its token is **mounted into every pod by default** even when unused. (Ch 23)

StatefulSet — Stable ordinal names, stable per-pod DNS and PVCs, ordered operations. PVCs survive deletion deliberately. (Ch 10)

Static pod — A pod started by the kubelet from a manifest on disk, owned by the **Node**. How the control plane bootstraps, and the escape hatch when the API server is down. (Ch 3)

Taint / toleration — Taints repel pods from a node; tolerations permit them. `NoExecute` taints applied by the node controller drive the ~5-minute failover. (Ch 5, 21)

Topology spread constraints — Distribute replicas across nodes or zones. Applies **at scheduling time only**. (Ch 25)

Validating Admission Policy — CEL-based admission built into Kubernetes, **no webhook required**. (Ch 24)

VPA — VerticalPodAutoscaler. In `updateMode: "Off"` it is a free sizing advisor. Do not combine Auto mode with a CPU-based HPA. (Ch 25)

[Back to contents](#) · [Previous: Appendix A](#) · [Next: Appendix C — Further reading](#)

Appendix C — Further reading

Primary documentation

- **Kubernetes docs** — <https://kubernetes.io/docs/>. The Concepts section rewards reading in order rather than dipping into. The API reference is generated and authoritative.
- `kubectl explain` — the documentation for *your* cluster version, always correct, always available offline. Use it before searching.
- **Kubernetes Enhancement Proposals** — <https://github.com/kubernetes/enhancements>. Where features are designed. The KEP for a feature explains *why* it works the way it does, which the user docs rarely do.
- **API deprecation guide** — <https://kubernetes.io/docs/reference/using-api/deprecation-guide/>. Read before every upgrade (Ch 21).
- **Gateway API** — <https://gateway-api.sigs.k8s.io/>.

Understanding the internals

- `kubernetes/community` **design proposals** — <https://github.com/kubernetes/community>. The original architecture documents.
- **Kelsey Hightower, *Kubernetes The Hard Way*** — <https://github.com/kelseyhightower/kubernetes-the-hard-way>. Build a cluster component by component with no tooling. The single best way to make Chapter 3 concrete.
- **Brendan Burns et al., *Kubernetes: Up and Running*** (O'Reilly) — solid general introduction.
- **Marko Lukša, *Kubernetes in Action*** (Manning) — still the best explanatory book, particularly on the object model.
- **Jeff Geerling / Learnk8s resources** — good practical material on sizing and capacity.

Operations and production

- **Google SRE Book** — <https://sre.google/books/>. Not Kubernetes-specific, and the right frame for what to alert on (Ch 26).
- **Learnk8s Production Best Practices checklist** — <https://learnk8s.io/production-best-practices>. Overlaps heavily with Chapter 32's checklist and is worth cross-referencing.
- **Kubernetes Failure Stories** — <https://k8s.af/>. Real post-mortems. More instructive than any best-practice list, and a good antidote to overconfidence.
- **CIS Kubernetes Benchmark** — the hardening checklist your auditor has read.
- **NSA/CISA Kubernetes Hardening Guidance** — free, thorough, and reasonable.

Security

- **Pod Security Standards** — <https://kubernetes.io/docs/concepts/security/pod-security-standards/>.
- **RBAC docs** — <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>.

- `kubectl-who-can` (Aqua) — reverse RBAC lookup: who can perform this action?
- `rbac-lookup`, `rakless` — audit bindings and access matrices.
- **kube-bench** — automates the CIS benchmark.
- **kube-hunter**, **Trivy Operator** — cluster vulnerability scanning.
- **Sigstore / cosign** — <https://docs.sigstore.dev/>, for Chapter 24's verification.

Networking

- **Cilium docs** — <https://docs.cilium.io/>. Among the best technical writing in the ecosystem, and useful even if you run a different CNI.
- **Calico's networking guides** — good on BGP, IPAM and policy.
- **"A Guide to the Kubernetes Networking Model"** (Julia Evans' and others' write-ups) — for the mental model.
- **NetworkPolicy editor** — <https://editor.networkpolicy.io/>. Visualises what a policy actually does, which catches the AND/OR list-item trap from Chapter 14.

Tools referenced in this book

Tool	For	Where
kind	Local/CI clusters	kind.sigs.k8s.io
k9s	Terminal UI	k9scli.io
kubectx / kubens	Context and namespace switching	github.com/ahmetb/kubectx
kube-ps1 / starship	Current context in your prompt	—
stern	Multi-pod log tailing	github.com/stern/stern
netshoot	Debugging toolbox image	github.com/nicolaka/netshoot
kustomize	Overlays (built into kubectl)	kustomize.io
helm	Packaging	helm.sh
helm diff	Preview upgrades — install this	github.com/databus23/helm-diff
Argo CD / Flux	GitOps	argo-cd.readthedocs.io / fluxcd.io
Argo Rollouts / Flagger	Progressive delivery with rollback	—
Kyverno	Policy as YAML	kyverno.io
cert-manager	Certificates — install on day one	cert-manager.io
External Secrets Operator	Secrets from Vault/cloud	external-secrets.io
Velero	Backup and restore	velero.io
metrics-server	kubectl top and HPA	github.com/kubernetes-sigs/metrics-server
kube-prometheus-stack	Metrics, alerts, dashboards	github.com/prometheus-operator
OpenCost / Kubecost	Cost attribution	opencost.io
pluto / kubent	Deprecated API detection	—
vCluster	Virtual clusters for tenancy	vcluster.com
polaris / kubescape	Manifest linting against best practice	—

polaris deserves a specific mention: it checks manifests against a large fraction of Chapter 32's checklist automatically, and belongs in CI.

Certification, if useful

- **CKA** — cluster operations. Practical, hands-on, and genuinely a good forcing function for Parts I and IV.
- **CKAD** — application development. Parts II and III.
- **CKS** — security. Assumes CKA; covers Parts V's material.

All three are practical exams against real clusters, which makes them more meaningful than most certifications.

Staying current

Kubernetes releases three times a year and each release deprecates something. Several claims in this book — the `CrashLoopBackOff/Error` reporting, `OOM reason` fields, Gateway API maturity, in-place pod resizing — are version-dependent and may change.

- **Release notes** — <https://kubernetes.io/docs/setup/release/notes/>. Read the deprecations section at minimum.
- **Last Week in Kubernetes Development** — <https://lwkd.info/>. Concise and technical.
- **KubeCon talks** on YouTube — the deep-dive and post-mortem talks especially.
- **CNCF landscape** — <https://landscape.cncf.io/>. A map, not a shortlist (Ch 31).

Verifying claims yourself

The habit worth keeping from this book: **measure rather than repeat**. Every number here came from running a command against a real three-node cluster, and several results contradicted what the documentation and the folklore say — a crash loop that reported `Error` instead of `CrashLoopBackOff`, an OOM kill that reported `Error` instead of `OOMKilled`, an Ingress failure that was really a scheduling problem, and eviction thresholds that kind disables entirely.

A disposable cluster costs a minute:

```
kind create cluster
```

When you read a claim about Kubernetes — including one in this book — check it against your own version. The answer will be authoritative for the cluster you actually run, which is the one that matters.

[Back to contents](#) · [Previous: Appendix B — Glossary](#)